LabVIEW™ Basics II: Development Course Manual

Course Software Version 8.5
September 2007 Edition
Part Number 320629R-01

Copyright

© 1993–2007 National Instruments Corporation. All rights reserved.
Under the copyright laws, this publication may not be reproduced or transmitted in any form, electronic or mechanical, including photocopying, recording, storing in an information retrieval system, or translating, in whole or in part, without the prior written consent of National Instruments Corporation.

National Instruments respects the intellectual property of others, and we ask our users to do the same. NI software is protected by copyright and other intellectual property laws. Where NI software may be used to reproduce software or other materials belonging to others, you may use NI software only to reproduce materials that you may reproduce in accordance with the terms of any applicable license or other legal restriction.

The following notices relate to certain components used in USI (Xerces C++, ICU, HDF5, Citadel 5, b64 library, and Stingray). For a listing of the conditions and disclaimers regarding these components, refer to the `USICopyrights.chm`.

Xerces C++. This product includes software developed by the Apache Software Foundation (`http://www.apache.org/`).
Copyright 1999 The Apache Software Foundation. All rights reserved.

ICU. Copyright 1995–2003 International Business Machines Corporation and others. All rights reserved.

HDF5. NCSA HDF5 (Hierarchical Data Format 5) Software Library and Utilities
Copyright 1998, 1999, 2000, 2001, 2003 by the Board of Trustees of the University of Illinois. All rights reserved.

Citadel 5. In regards to components that use Citadel 5, the following copyright applies. Copyright © 1994 Hewlett-Packard Company.

b64 library. Copyright © 2004–2006, Matthew Wilson and Synesis Software. All Rights Reserved.

Stingray. This software includes Stingray software developed by the Rogue Wave Software division of Quovadx, Inc.
Copyright 1995–2006, Quovadx, Inc. All Rights Reserved.

Trademarks

National Instruments, NI, ni.com, and LabVIEW are trademarks of National Instruments Corporation. Refer to the *Terms of Use* section on `ni.com/legal` for more information about National Instruments trademarks.

Other product and company names mentioned herein are trademarks or trade names of their respective companies.

Members of the National Instruments Alliance Partner Program are business entities independent from National Instruments and have no agency, partnership, or joint-venture relationship with National Instruments.

Patents

For patents covering National Instruments products, refer to the appropriate location: **Help»Patents** in your software, the `patents.txt` file on your CD, or `ni.com/legal/patents`.

Worldwide Technical Support and Product Information
ni.com

National Instruments Corporate Headquarters
11500 North Mopac Expressway Austin, Texas 78759-3504 USA Tel: 512 683 0100

Worldwide Offices

Australia 1800 300 800, Austria 43 662 457990-0, Belgium 32 (0) 2 757 0020, Brazil 55 11 3262 3599, Canada 800 433 3488, China 86 21 5050 9800, Czech Republic 420 224 235 774, Denmark 45 45 76 26 00, Finland 358 (0) 9 725 72511, France 01 57 66 24 24, Germany 49 89 7413130, India 91 80 41190000, Israel 972 3 6393737, Italy 39 02 41309277, Japan 0120-527196, Korea 82 02 3451 3400, Lebanon 961 (0) 1 33 28 28, Malaysia 1800 887710, Mexico 01 800 010 0793, Netherlands 31 (0) 348 433 466, New Zealand 0800 553 322, Norway 47 (0) 66 90 76 60, Poland 48 22 3390150, Portugal 351 210 311 210, Russia 7 495 783 6851, Singapore 1800 226 5886, Slovenia 386 3 425 42 00, South Africa 27 0 11 805 8197, Spain 34 91 640 0085, Sweden 46 (0) 8 587 895 00, Switzerland 41 56 2005151, Taiwan 886 02 2377 2222, Thailand 662 278 6777, Turkey 90 212 279 3031, United Kingdom 44 (0) 1635 523545

For further support information, refer to the *Additional Information and Resources* appendix. To comment on National Instruments documentation, refer to the National Instruments Web site at ni.com/info and enter the info code feedback.

Contents

Student Guide

A. Course Description vi
B. What You Need to Get Started vi
C. Installing the Course Software vii
D. Course Goals vii
E. Course Conventions viii

Lesson 1
Common Design Techniques

A. Single Loop Architectures 1-2
B. Parallelism 1-6
C. Multiple Loop Architectures 1-8
D. Timing a Design Pattern 1-12

Lesson 2
Communicating Among Multiple Loops

A. Variables 2-2
B. Functional Global Variables 2-12
Exercise 2-1 Variables VI 2-15
Exercise 2-2 Optional: Global Data Project 2-24
C. Race Conditions 2-30
Exercise 2-3 Concept: Bank VI 2-37
D. Synchronizing Data Transfer 2-40
Exercise 2-4 Queues vs Local Variables VI 2-45

Lesson 3
Event Programming

A. Event-Driven Programming 3-2
Exercise 3-1 Concept: Experiment with Event Structures 3-11
B. Event-Based Design Patterns 3-14
Exercise 3-2 Concept: Experiment with Event-Based Design Patterns 3-17

Lesson 4
Improving an Existing VI

A. Refactoring Inherited Code 4-2
B. Typical Refactoring Issues 4-4
Exercise 4-1 Concept: Typical Issues 4-8

Lesson 5
Controlling the User Interface

A. VI Server Architecture 5-2
B. Property Nodes 5-3
Exercise 5-1 Temperature Limit VI 5-5
C. Control References 5-10
Exercise 5-2 Set Plot Names 5-14
D. Invoke Nodes 5-23
Exercise 5-3 Front Panel Properties VI 5-24

Lesson 6
Advanced File I/O Techniques

A. File Formats 6-2
B. Binary Files 6-4
Exercise 6-1 Bitmap File Writer VI 6-12
C. TDMS Files 6-19
Exercise 6-2 TDMS Reader VI 6-27

Lesson 7
Creating and Distributing Applications

A. LabVIEW Features for Project Development 7-2
Exercise 7-1 Concept: LabVIEW Project Management Tools 7-5
B. Preparing the Application 7-7
C. Building the Application and Installer 7-8
Exercise 7-2 Concept: Creating a Stand-Alone Application 7-10

Appendix A
Additional Information and Resources

Course Evaluation

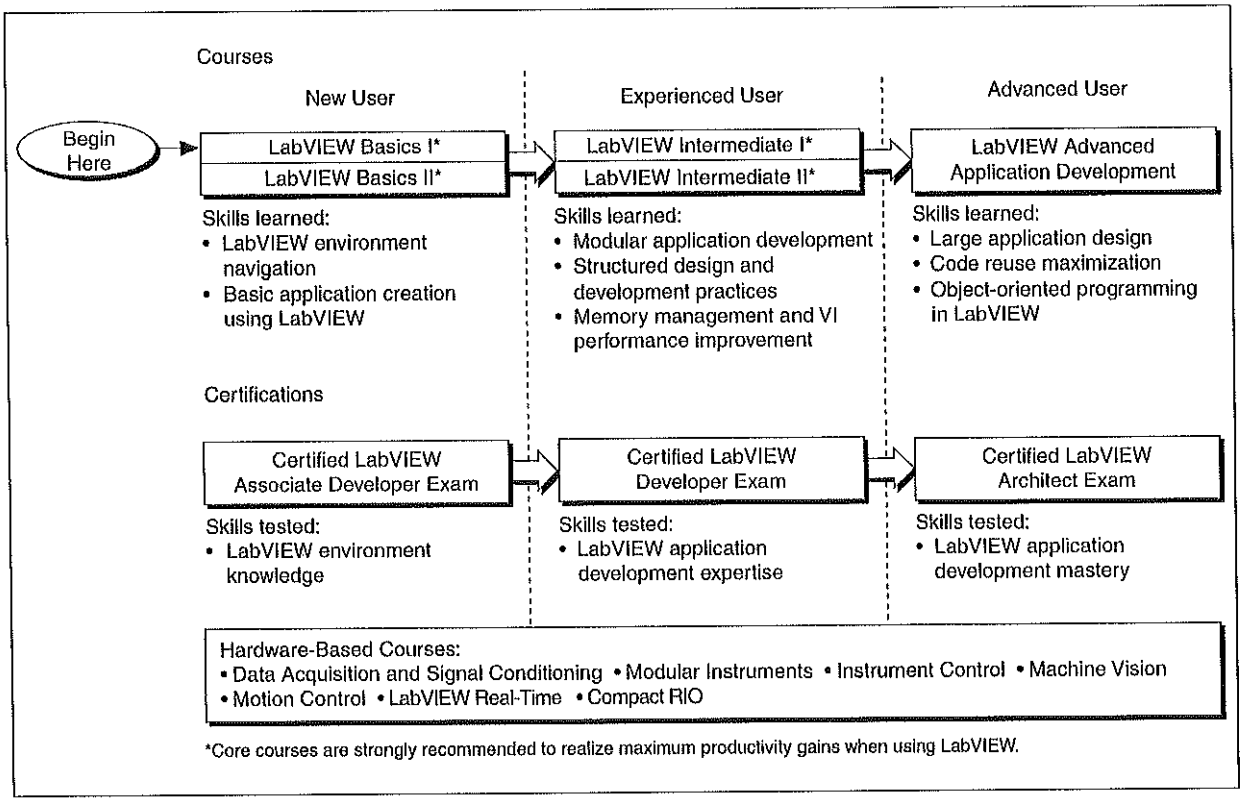
Student Guide

Thank you for purchasing the *LabVIEW Basics II: Development* course kit. You can begin developing an application soon after you complete the exercises in this manual. This course manual and the accompanying software are used in the two-day, hands-on *LabVIEW Basics II: Development* course.

You can apply the full purchase of this course kit toward the corresponding course registration fee if you register within 90 days of purchasing the kit. Visit `ni.com/training` for online course schedules, syllabi, training centers, and class registration.

Note For course manual updates and corrections, refer to `ni.com/info` and enter the info code `rdlvc2`.

The *LabVIEW Basics II: Development* course is part of a series of courses designed to build your proficiency with LabVIEW and help you prepare for exams to become an NI Certified LabVIEW Developer and NI Certified LabVIEW Architect. The following illustration shows the courses that are part of the LabVIEW training series. Refer to `ni.com/training` for more information about NI Certification.



A. Course Description

Use this manual to learn about LabVIEW programming concepts, techniques, features, VIs, and functions you can use to create test and measurement, data acquisition, instrument control, datalogging, measurement analysis, and report generation applications.

This course manual assumes that you are familiar with Windows, that you have experience writing algorithms in the form of flowcharts or block diagrams; and that you have taken the *LabVIEW Basics I: Introduction* course or have equivalent experience.

The course manual is divided into lessons, each covering a topic or a set of topics. Each lesson consists of the following:

- An introduction that describes the purpose of the lesson and what you will learn
- A description of the topics in the lesson
- A set of exercises to reinforce those topics
- A set of additional exercises to complete if time permits
- A summary that outlines important concepts and skills taught in the lesson

Several exercises in this manual use a plug-in multifunction data acquisition (DAQ) device connected to a DAQ Signal Accessory containing a temperature sensor, function generator, and LEDs.



Exercises that explicitly require hardware are indicated with an icon, shown at left. You also can substitute other hardware for those previously mentioned. For example, you can use another National Instruments DAQ device connected to a signal source, such as a function generator.

B. What You Need to Get Started

Before you use this course manual, make sure you have all of the following items:

- ❑ Windows 2000 or later installed on your computer; this course is optimized for Windows XP
- ❑ Multifunction DAQ device configured as device 1 using Measurement & Automation Explorer (MAX)
- ❑ DAQ Signal Accessory, wires, and cable
- ❑ LabVIEW Professional Development System 8.5 or later

- ❑ *LabVIEW Basics II: Development* course CD, from which you install the following files:

Folder Name	Description
`Exercises`	Folder containing VIs used in the course
`Solutions`	Folder containing completed course exercises

C. Installing the Course Software

Complete the following steps to install the course software.

1. Insert the course CD in your computer. The **LabVIEW Basics Course Material Setup** dialog box appears.
2. Click the **Next** button.
3. Click the **Next** button to select the default installation directory
4. Click the **Next** button to begin the installation.
5. Click **Finish** to exit the Setup Wizard.
6. The installer places the `Exercises` and `Solutions` folders at the top level of the root directory.

Exercise files are located in the `<Exercises>\LabVIEW Basics II\` folder.



Tip Folder names in angle brackets, such as `<Exercises>`, refer to folders on the root directory of your computer.

D. Course Goals

This course prepares you to do the following:

- Understand the VI development process
- Understand some common VI programming architectures
- Design effective user interfaces (front panels)
- Efficiently transfer data among parallel processes
- Use advanced file I/O techniques
- Use LabVIEW to create applications
- Use Property Nodes and Invoke Nodes throughout your VI

You will apply these concepts as you build a project that uses VIs you create throughout the course. While these VIs individually illustrate specific

concepts and features in LabVIEW, they constitute part of a larger project built throughout the course.

This course does *not* describe any of the following:

- LabVIEW programming methods covered in the *LabVIEW Basics I: Introduction* course
- Every built-in VI, function, or object; refer to the *LabVIEW Help* for more information about LabVIEW features not described in this course
- Developing a complete application for any student in the class; refer to the NI Example Finder, available by selecting **Help»Find Examples**, for example VIs you can use and incorporate into VIs you create

E. Course Conventions

The following conventions appear in this course manual:

» The » symbol leads you through nested menu items and dialog box options to a final action. The sequence **File»Page Setup»Options** directs you to pull down the **File** menu, select the **Page Setup** item, and select **Options** from the last dialog box.

 This icon denotes a tip, which alerts you to advisory information.

 This icon denotes a note, which alerts you to important information.

 This icon denotes a caution, which advises you of precautions to take to avoid injury, data loss, or a system crash.

 This icon indicates that an exercise requires a plug-in DAQ device.

bold Bold text denotes items that you must select or click in the software, such as menu items and dialog box options. Bold text also denotes parameter names, controls and buttons on the front panel, dialog boxes, sections of dialog boxes, menu names, and palette names.

italic Italic text denotes variables, emphasis, a cross-reference, or an introduction to a key concept. Italic text also denotes text that is a placeholder for a word or value that you must supply.

`monospace` Text in this font denotes text or characters that you should enter from the keyboard, sections of code, programming examples, and syntax examples. This font is also used for the proper names of disk drives, paths, directories, programs, subprograms, subroutines, device names, functions, operations, variables, filenames, and extensions.

monospace bold	Bold text in this font denotes the messages and responses that the computer automatically prints to the screen. This font also emphasizes lines of code that are different from the other examples.
Platform	Text in this font denotes a specific platform and indicates that the text following it applies only to that platform.

1

Common Design Techniques

You can develop better programs in LabVIEW and in other programming languages if you follow consistent programming techniques and architectures. This lesson discusses two different categories of programming architectures—single loops and multiple loops. Collectively, these architectures are known as design patterns.

Single loop architectures include the simple VI, the general VI, and the state machine design patterns.

Multiple loop architectures include the parallel loop VI, the master/slave, and the producer/consumer design patterns.

Understanding the appropriate use of each design pattern helps you create more efficient LabVIEW VIs.

Topics

A. Single Loop Architectures

B. Parallelism

C. Multiple Loop Architectures

D. Timing a Design Pattern

A. Single Loop Architectures

You learned to design three different types of architectures in the *LabVIEW Basics I: Introduction* course—the simple architecture, the general architecture, and the state machine.

Simple VI Design Patterns

When performing calculations or making quick lab measurements, you do not need a complicated architecture. Your program might consist of a single VI that takes a measurement, performs calculations, and either displays the results or records them to disk. The simple VI design pattern usually does not require a specific start or stop action from the user. The user just clicks the **Run** button. Use this architecture for simple applications or for functional components within larger applications. You can convert these simple VIs into subVIs that you use as building blocks for larger applications.

Figure 1-1 displays the block diagram of the Determine Warnings VI that was the course project in the *LabVIEW Basics I: Introduction* course. This VI performs a single task—it determines what warning to output dependent on a set of inputs. You can use this VI as a subVI whenever you must determine the warning level.

Notice that the VI in Figure 1-1contains no start or stop actions from the user. In this VI all block diagram objects are connected through data flow. You can determine the overall order of operations by following the flow of data. For example, the Not Equal function cannot execute until the Greater Than or Equal function, the Less Than or Equal function, and both Select functions have executed.

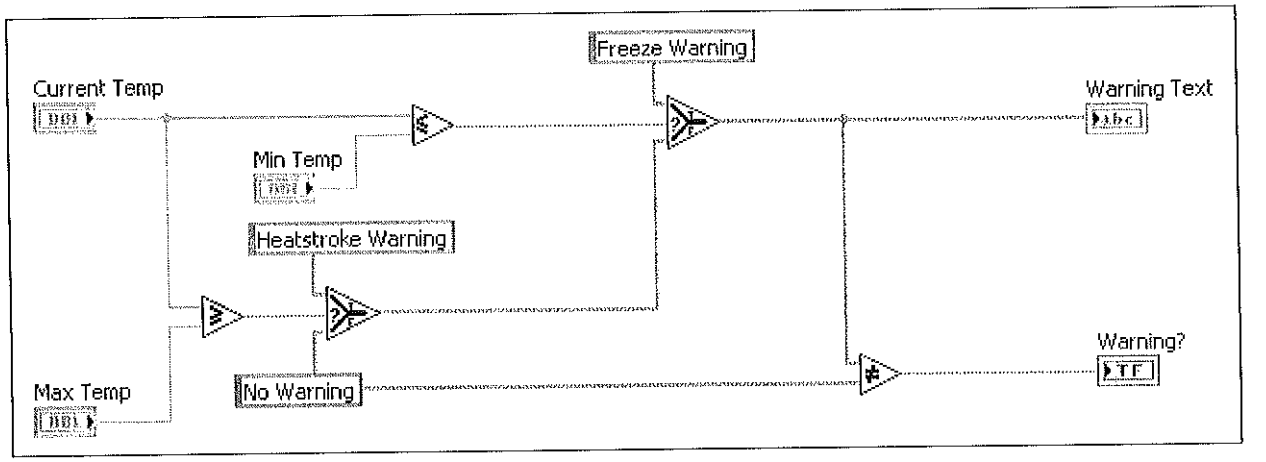


Figure 1-1. Simple VI Architecture

General VI Design Patterns

A general VI design pattern has three main phases—startup, main application, and shutdown. Each of the following phases may contain code that uses another type of design pattern.

- Startup—Initializes hardware, reads configuration information from files, or prompts the user for data file locations.
- Main Application—Consists of at least one loop that repeats until the user decides to exit the program or the program terminates for other reasons such as I/O completion.
- Shutdown—Closes files, writes configuration information to disk, or resets I/O to the default state.

Figure 1-2 shows the general VI design pattern.

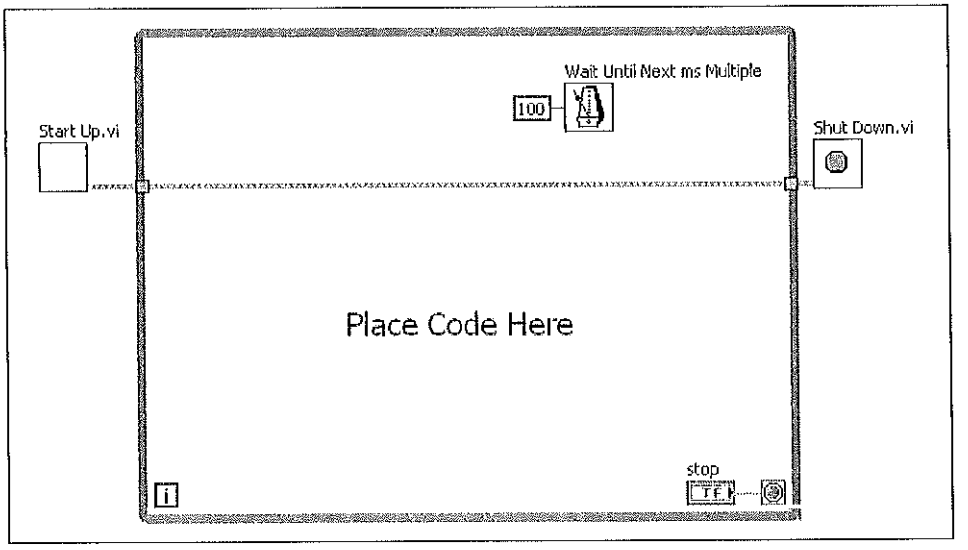


Figure 1-2. General VI Design Pattern

In Figure 1-2, the error cluster wires control the execution order of the three sections. The While Loop does not execute until the Start Up VI finishes running and returns the error cluster data. Consequently, the Shut Down VI cannot run until the main application in the While Loop finishes and the error cluster data leaves the loop.

Tip Most loops require a Wait function, especially if that loop monitors user input on the front panel. Without the Wait function, the loop might run continuously and use all of the computer system resources. The Wait function forces the loop to run asynchronously even if you specify 0 milliseconds as the wait period. If the operations inside the main loop react to user inputs, you can increase the wait period to a level acceptable for reaction times. A wait of 100–200 ms is usually good because most users cannot detect that amount of delay between clicking a button on the front panel and the subsequent event execution.

For simple applications, the main application loop is obvious and contains code that uses the simple VI design pattern. When the application incudes complicated user interfaces or multiple tasks such as user actions, I/O triggers, and so on, the main application phase gets more complicated.

State Machine Design Pattern

The state machine design pattern is a modification of the general design pattern. It usually has a start up and shut down phase. However, the main application phase consists of a Case structure embedded in the loop. This architecture allows you to run different code each time the loop executes, depending upon some condition. Each case defines a state of the machine, hence the name, state machine. Use this design pattern for VIs that are easily divided into several simpler tasks, such as VIs that act as a user interface.

A state machine in LabVIEW consists of a While Loop, a Case structure, and a shift register. Each state of the state machine is a separate case in the Case structure. You place VIs and other code that the state should execute within the appropriate case. A shift register stores the state that should execute upon the next iteration of the loop. The block diagram of a state machine VI with five states appears in Figure 1-3. Figure 1-4 shows the other cases, or states, of the state machine.

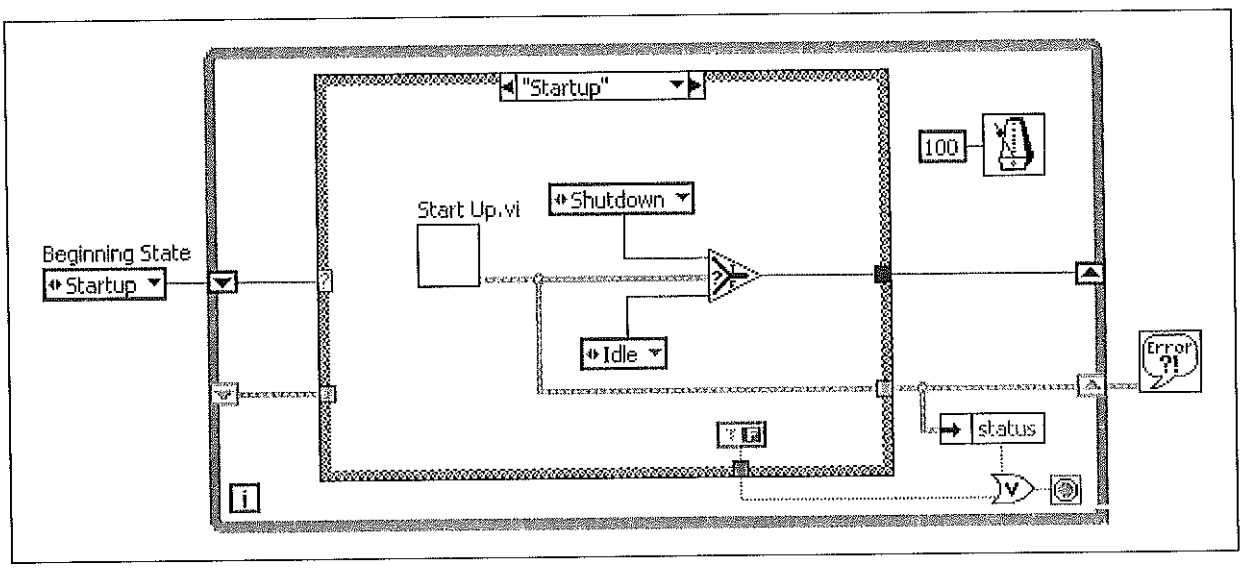


Figure 1-3. State Machine with Startup State

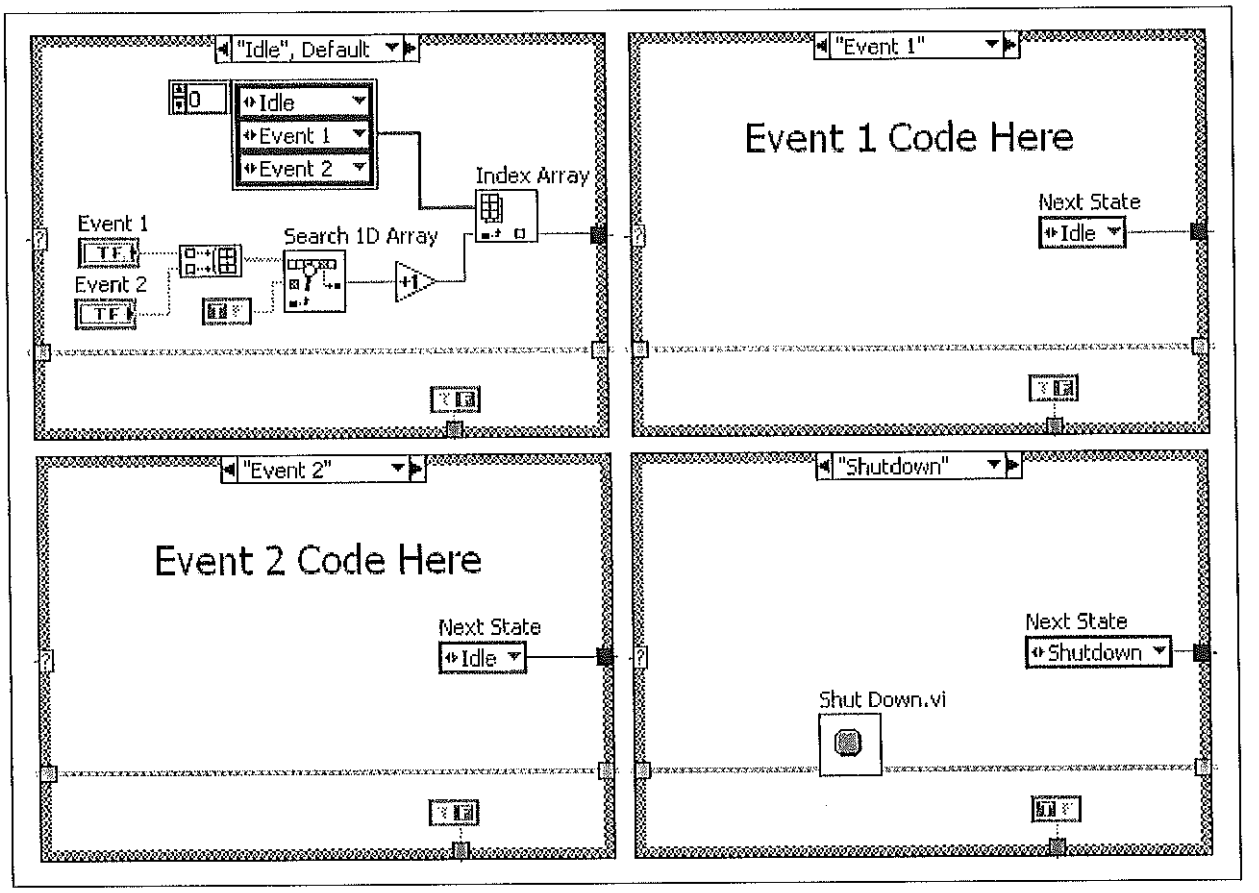


Figure 1-4. Idle (Default), Event 1, Event 2, and Shutdown States

In the state machine design pattern, you design the list of possible tasks, or states, and then map them to each case. For the VI in the previous example, the possible states are Startup, Idle, Event 1, Event 2, and Shutdown. An enumerated constant stores the states. Each state has its own case in the Case structure. The outcome of one case determines which case to execute next. The shift register stores the value that determines which case to execute next.

The state machine design pattern can make the block diagram much smaller, and therefore, easier to read and debug. Another advantage of the state machine architecture is that each case determines the next state, unlike Sequence structures that must execute every frame in sequence.

A disadvantage of the state machine design pattern is that with the approach in the previous example, it is possible to skip states. If two states in the structure are called at the same time, this model handles only one state, and the other state does not execute. Skipping states can lead to errors that are difficult to debug because they are difficult to reproduce. More complex versions of the state machine design pattern contain extra code that creates a queue of events, or states, so that you do not miss a state. Refer to

Lesson 2, *Communicating Among Multiple Loops*, for more information about queues.

B. Parallelism

In this course, parallelism refers to executing multiple tasks at the same time. Consider the example of creating and displaying two sine waves at a different frequencies. Using parallelism, you place one sine wave in a loop, and the second sine wave in a different loop.

A challenge in programming parallel tasks is passing data among multiple loops without creating a data dependency. For example, if you pass the data using a wire, the loops are no longer parallel. In the multiple sine wave example, you may want to share a single stop mechanism between the loops, as shown in Figure 1-5.

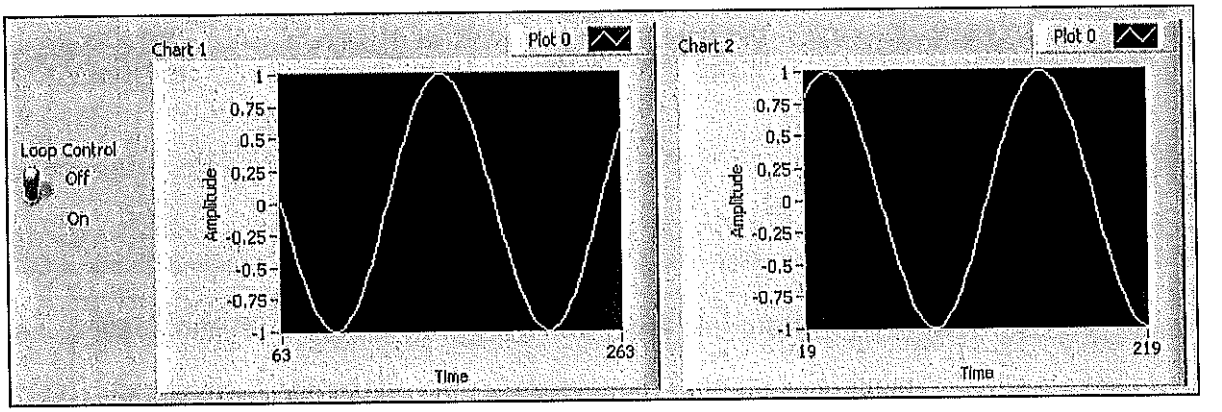


Figure 1-5. Parallel Loops Front Panel

Examine what happens when you try to share data among parallel loops with a wire using those different methods.

Method 1 (Incorrect)

Place the **Loop Control** terminal outside of both loops and wire it to each conditional terminal, as shown in Figure 1-6. The Loop control is a data input to both loops, therefore the **Loop Control** terminal is read only once, before either While Loop begins executing. If False is passed to the loops, the While Loops run indefinitely. Turning off the switch does not stop the VI because the switch is not read during the iteration of either loop.

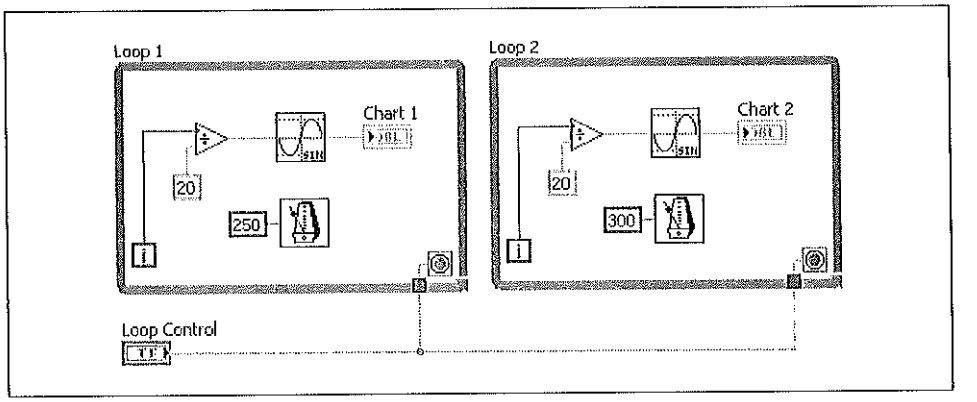


Figure 1-6. Parallel Loops Method 1 Example

Method 2 (Incorrect)

Move the **Loop Control** terminal inside Loop 1 so that it is read in each iteration of Loop 1, as shown in the following block diagram. Although Loop 1 terminates properly, Loop 2 does not execute until it receives all its data inputs. Loop 1 does not pass data out of the loop until the loop stops, so Loop 2 must wait for the final value of the **Loop Control**, available only after Loop 1 finishes. Therefore, the loops do not execute in parallel. Also, Loop 2 executes for only one iteration because its conditional terminal receives a True value from the **Loop Control** switch in Loop 1.

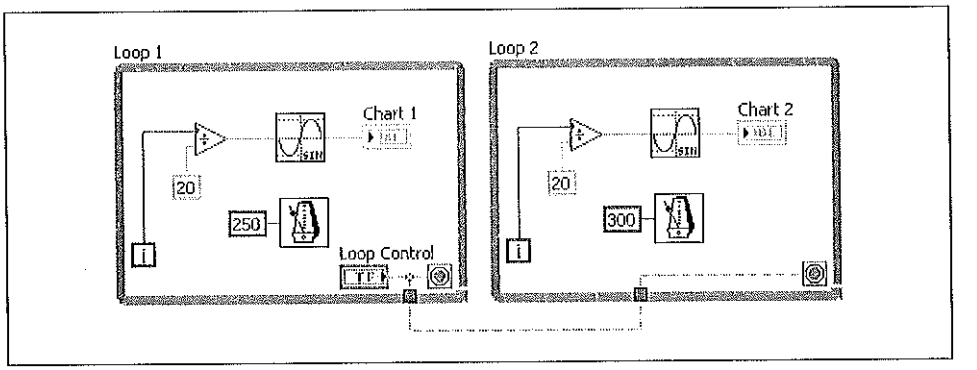


Figure 1-7. Parallel Loops Method 2 Example

Method 3 (Solution)

If you could read the value of the loop control from a file, you would no longer have a dataflow dependency between the loops, as each loop can independently access the file. However, reading and writing to files can be time consuming, at least in processor time. Another way to accomplish this task is to find the location where the loop control data is stored in memory and read that memory location directly. Refer to Lesson 2, *Communicating Among Multiple Loops*, for information on methods for solving this problem.

C. Multiple Loop Architectures

You learned a few different reasons for using parallelism in the previous section. This section describes the following multiple loop architectures—parallel loop, master/slave, and producer/consumer data.

Parallel Loop Design Pattern

Some applications must respond to and run several tasks concurrently. One way to improve parallelism is to assign a different loop to each task. For example, you might have a different loop for each button on the front panel and for every other kind of task, such as a menu selection, I/O trigger, and so on. Figure 1-8 shows this parallel loop design pattern.

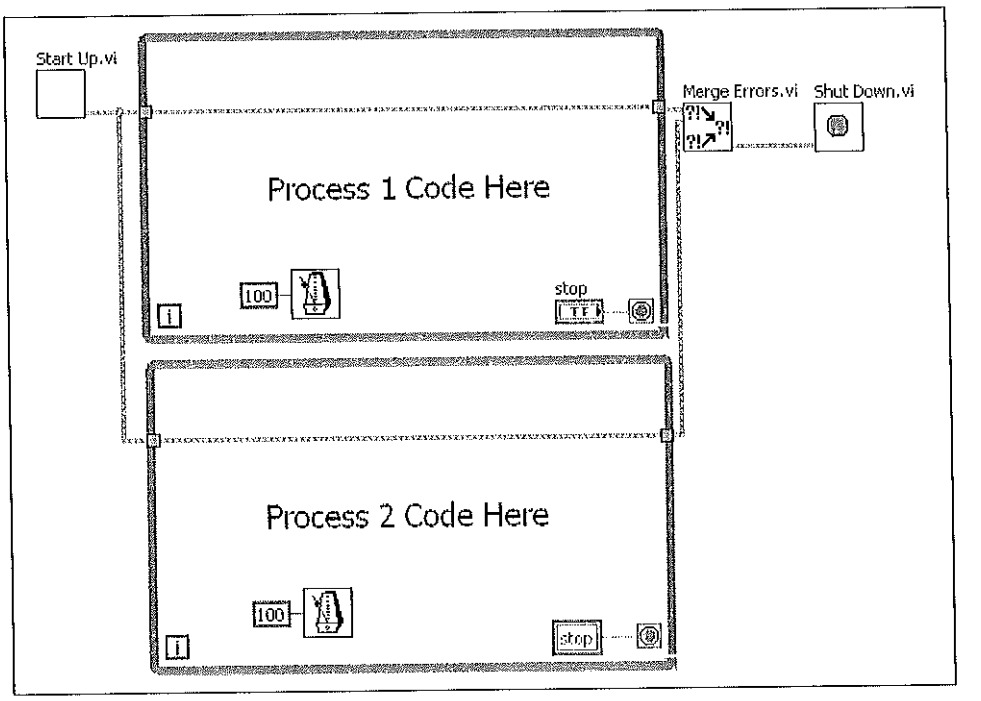


Figure 1-8. Parallel Loop Design Pattern

This structure is straightforward and appropriate for some simple menu VIs, where you expect a user to select from one of several buttons that perform different actions. The parallel loop design pattern lets you handle multiple, simultaneous, independent tasks. In this design pattern, responding to one action does not prevent the VI from responding to another action. For example, if a user clicks a button that displays a dialog box, parallel loops can continue to respond to I/O tasks.

However, the parallel loop design pattern requires you to coordinate and communicate between different loops. The **Stop** button for the second loop in Figure 1-8 is a local variable. You cannot use wires to pass data between loops because doing so prevents the loops from running in parallel. Instead,

you must use a messaging technique for passing information among processes. Refer to Lesson 2, *Communicating Among Multiple Loops*, for more information about using local variables, notifiers, or queues to message among parallel loops.

Master/Slave Design Pattern

The master/slave design pattern consists of multiple parallel loops. Each of the loops may execute tasks at different rates. One loop acts as the master, and the other loops act as slaves. The master loop controls all the slave loops and communicates with them using messaging techniques, as shown in Figure 1-9.

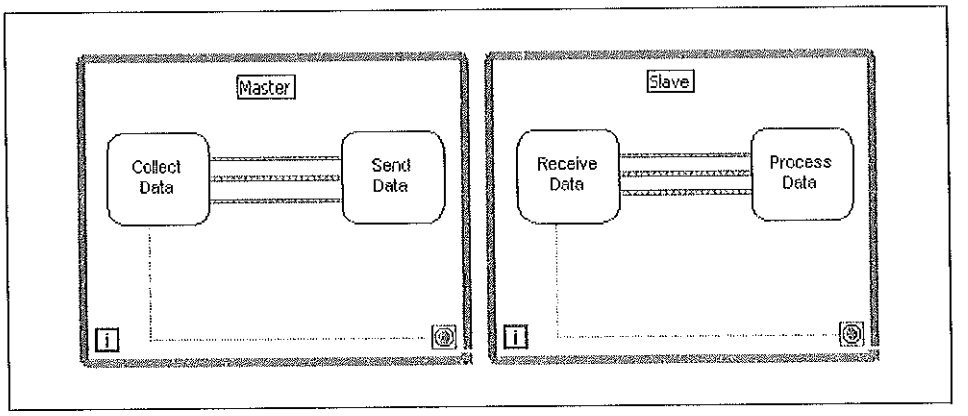


Figure 1-9. Master/Slave Design Pattern

Use the master/slave design pattern when you need a VI to respond to user interface controls while simultaneously collecting data. For example, you want to create a VI that measures and logs a slowly changing voltage once every five seconds. The VI acquires a waveform from a transmission line and displays it on a graph every 100 ms. The VI also provides a user interface that allows the user to change parameters for each acquisition. The master/slave design pattern is well suited for this acquisition application. For this application, the master loop contains the user interface. The voltage acquisition occurs in one slave loop, while the graphing occurs in another slave loop.

Using the standard master/slave design pattern approach to this VI, you would put the acquisition processes in two separate While Loops, both of them driven by a master loop that receives inputs from the user interface controls. This ensures that the separate acquisition processes do not affect each other, and that any delays caused by the user interface, such as displaying a dialog box, do not delay any iterations of the acquisition processes.

VIs that involve control also benefit from the use of master/slave design patterns. Consider a VI where a user controls a free motion robotic arm

using buttons on a front panel. This type of VI requires efficient, accurate, and responsive control because of the physical damage to the arm or surroundings that might occur if control is mishandled. For example, if the user instructs the arm to stop its downward motion, but the program is occupied with the arm swivel control, the robotic arm might collide with the support platform. Apply the master/slave design pattern to the application to avoid these problems. In this case, the master loop handles the user interface, and each controllable section of the robotic arm has its own slave loop. Because each controllable section of the arm has its own loop and its own piece of processing time, the user interface has more responsive control of the robotic arm.

With a master/slave design pattern, it is important that no two While Loops write to the same shared data. Ensure that no more than one While Loop may write to any given piece of shared data. Refer to Lesson 2, *Communicating Among Multiple Loops*, for more information about shared data.

The slave must not take too long to respond to the master. If the slave is processing a signal from the master and the master sends more than one message to the slave, the slave receives only the latest message. This use of the master/slave architecture could cause a loss of data. Use a master/slave architecture only if you are certain that each slave task takes less time to execute than the master loop.

Producer/Consumer Design Pattern

The producer/consumer design pattern is based on the master/slave design pattern and improves data sharing among multiple loops running at different rates. Similar to the master/slave design pattern, the producer/consumer design pattern separates tasks that produce and consume data at different rates. The parallel loops in the producer/consumer design pattern are separated into two categories—those that produce data and those that consume the data produced. Data queues communicate data among the loops. The data queues also buffer data among the producer and consumer loops.



Tip A buffer is a memory device that stores temporary data among two devices, or in this case, multiple loops.

Use the producer/consumer design pattern when you must acquire multiple sets of data that must be processed in order. Suppose you want to create a VI that accepts data while processing the data sets in the order they were received. The producer/consumer pattern is ideal for this type of VI because queuing (producing) the data occurs much faster than the data can be processed (consumed). You could put the producer and consumer in the same loop for this application, but the processing queue could not receive

additional data until the first piece of data was completely processed. The producer/consumer approach to this VI queues the data in the producer loop and processes the data in the consumer loop, as shown in Figure 1-10.

Tip Queue functions allow you to store a set of data that can be passed among multiple loops running simultaneously or among VIs. Refer to Lesson 2, *Communicating Among Multiple Loops*, for more information about queues.

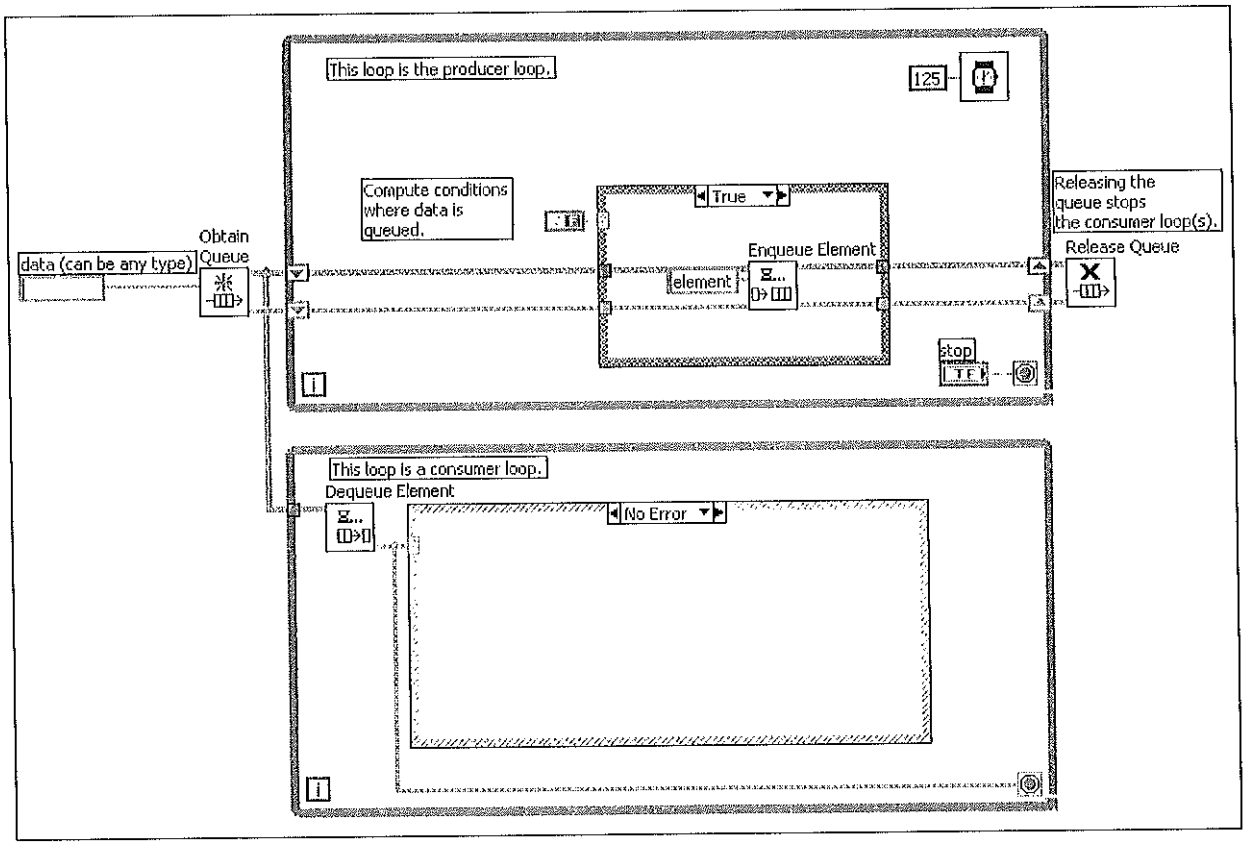


Figure 1-10. Producer/Consumer Design Pattern

This design pattern allows the consumer loop to process the data at its own pace, while the producer loop continues to queue additional data.

You also can use the producer/consumer design pattern to create a VI that analyzes network communication. This type of VI requires two processes to operate at the same time and at different speeds. The first process constantly polls the network line and retrieves packets. The second process analyzes the packets retrieved by the first process.

In this example, the first process acts as the producer because it supplies data to the second process, which acts as the consumer. The producer/consumer design pattern is an effective architecture for this VI. The parallel producer

and consumer loops handle the retrieval and analysis of data off the network, and the queued communication between the two loops allows buffering of the network packets retrieved. Buffering can become important when network communication is busy. With buffering, packets can be retrieved and communicated faster than they can be analyzed.

D. Timing a Design Pattern

This section discusses two forms of timing—execution timing and software control timing. Execution timing uses timing functions to give the processor time to complete other tasks. Software control timing involves timing a real-world operation to perform within a set time period.

Execution Timing

Execution timing involves timing a design pattern explicitly or based on events that occur within the VI. Explict timing uses a function that specifically allows the processor time to complete other tasks, such as the Wait Until Next ms Multiple function. When timing is based on events, the design pattern waits for some action to occur before continuing and allows the processor to complete other tasks while it waits.

Use explicit timing for design patterns such as the master/slave, producer/consumer, and state machine. These design patterns perform some type of polling while they execute.



Tip Polling is the process of making continuous requests for data from another device. In LabVIEW, this generally means that the block diagram continuously asks if there is data available, usually from the user interface.

For example, the master/slave design pattern shown in Figure 1-11 uses a While Loop and a Case structure to implement the master loop. The master executes continuously and polls for an event of some type, such as the user clicking a button. When the event occurs, the master sends a message to the slave. You need to time the master so it does not take over the execution of the processor. In this case, you typically use the Wait (ms) function to regulate how frequently the master polls.



Tip Always use a timing function such as the Wait (ms) function or the Wait Until Next ms Multiple function in any design pattern that continually executes and needs to be regulated. If you do not use a timing function in a continuously executing structure, LabVIEW uses all the processor time, and background processes may not run.

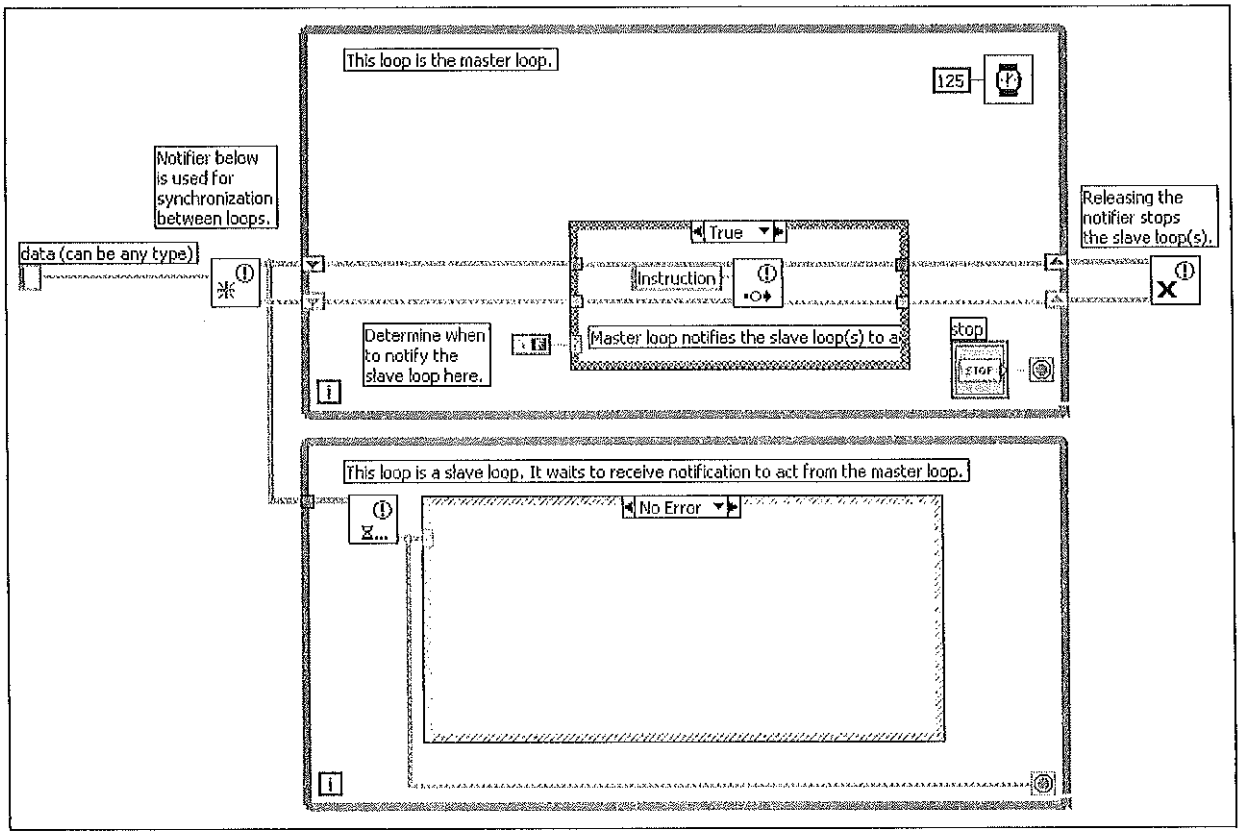


Figure 1-11. Master/Slave Design Pattern

Notice that the slave loop does not contain any form of timing. The use of Synchronization functions, such as queues and notifiers, to pass messages provides an inherent form of timing in the slave loop because the slave loop waits for the Notifier function to receive a message. After the Notifier function receives a message, the slave executes on the message. This creates an efficient block diagram that does not waste processor cycles by polling for messages. This is an example of execution timing by waiting for an event.

When you implement design patterns where the timing is based on the occurrence of events, you do not have to determine the correct timing frequency because the execution of the design pattern occurs only when an event occurs. In other words, the design pattern executes only when it receives an event.

Software Control Timing

Many applications that you create must execute an operation for a specified amount of time. Consider implementing a state machine design pattern for a temperature data acquisition system. If the specifications require that the system acquire temperature data for 5 minutes, you could remain in the acquisition state for 5 minutes. However, during that time you cannot

process any user interface actions such as stopping the VI. To process user interface actions, you must implement timing so that the VI continually executes for specified time. Implementing this type of timing involves keeping the application executing while monitoring a real-time clock.

In the *LabVIEW Basics I: Introduction* course, you implemented software control timing to monitor the time until the VI should acquire the next piece of data, as shown in Figure 1-12. Notice the use of the Elapsed Time Express VI to keep track of a clock.

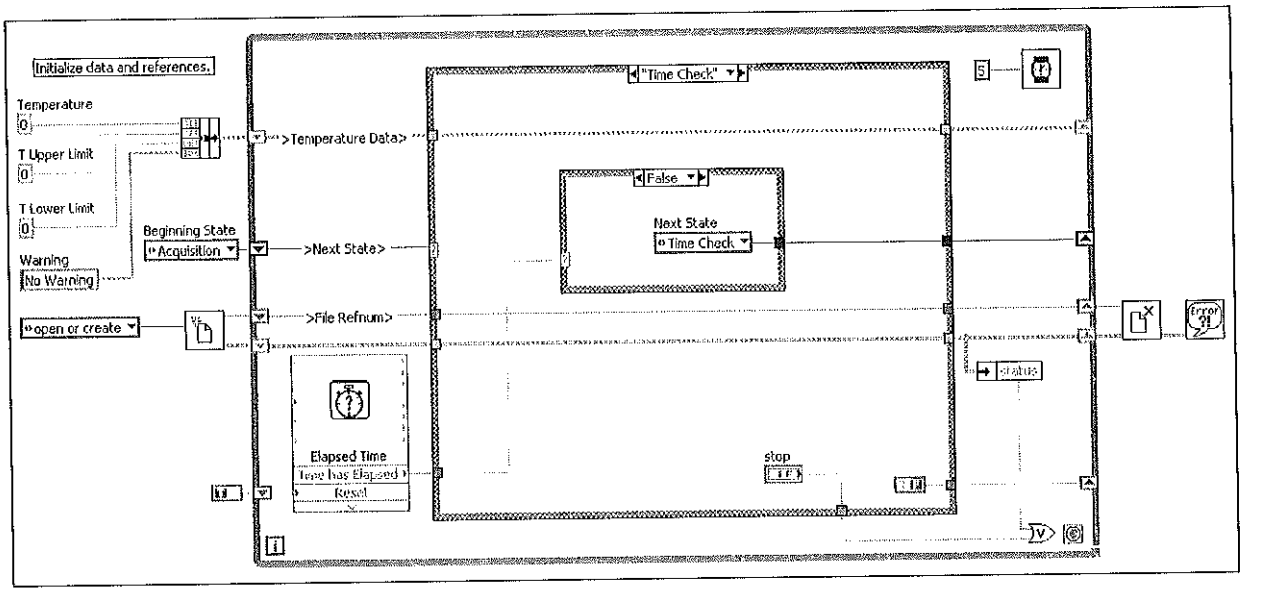


Figure 1-12. Use of the Elapsed Time Express VI

If you use the Wait (ms) function or the Wait Until Next ms Multiple function to perform software timing, the execution of the function you are timing does not occur until the wait function finishes. These timing functions are not the preferred method for performing software control timing, especially for VIs where the system must continually execute. A good pattern to use for software control timing cycles the current time throughout the VI, as shown in Figure 1-13.

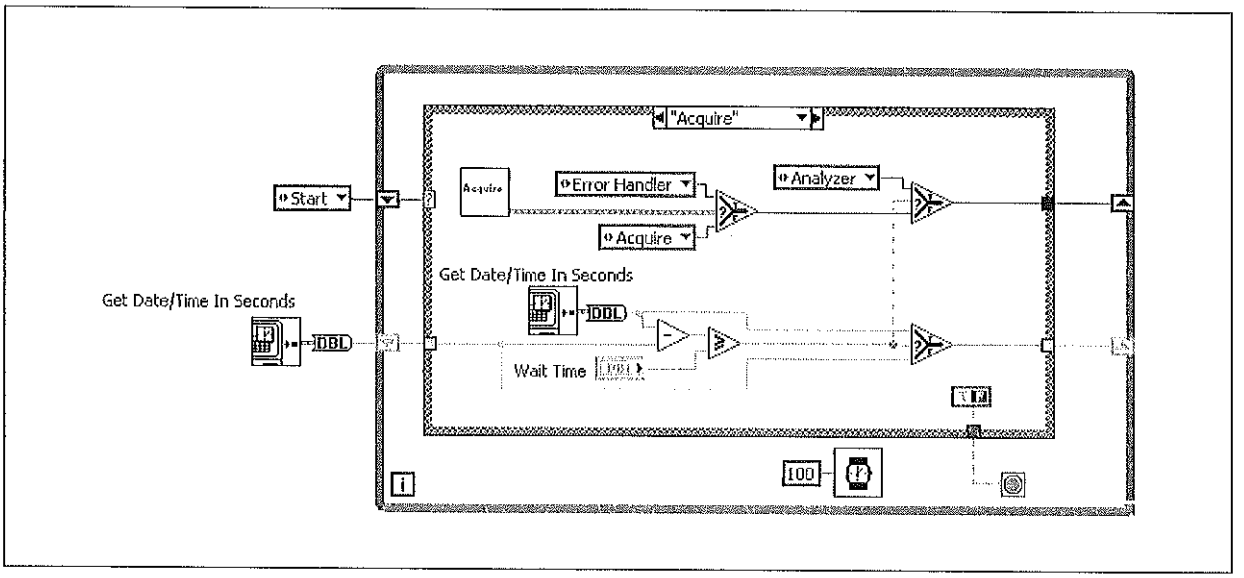


Figure 1-13. Software Timing Using the Get Date/Time In Seconds Function

The Get Date/Time In Seconds function, connected to the left terminal of the shift register, initializes the shift register with the current system time. Each state uses another Get Date/Time In Seconds function and compares the current time to the start time. If the difference in these two times is greater or equal to the wait time, the state finishes executing and the rest of the application executes.

Tip Always use the Get Date/Time In Seconds function instead of the Tick Count function for this type of comparison because the value of the Tick Count function can rollover to 0 during execution.

Refer to Lesson 2, *Communicating Among Multiple Loops*, for more information on creating a timing functional global variable to make the timing functionality modular and reusable.

Self-Review: Quiz

1. Software control timing allows the processor time to complete other tasks.
   a. True
   b. False
2. Execution timing is a method for allowing the processor time to complete other tasks.
   a. True
   b. False
3. You can use a wire to pass data between two parallel loops.
   a. True
   b. False

Self-Review: Quiz Answers

1. FALSE: Software control timing is a method for monitoring a real-time clock.
2. TRUE: Execution timing is a method for allowing the processor time to complete other tasks.
3. FALSE: If you pass data between two loops with a wire, the loops no longer run in parallel.

Notes

2

Communicating Among Multiple Loops

In Lesson 1, *Common Design Techniques*, you learned the difficulties of transferring data among multiple loops while maintaining parallel execution in the loops. This lesson describes messaging techniques for transferring data among multiple loops. These messaging techniques include variables, notifiers, and queues. You also learn about the programming issues involved in using these techniques and methods for overcoming these challenges.

Topics

A. Variables

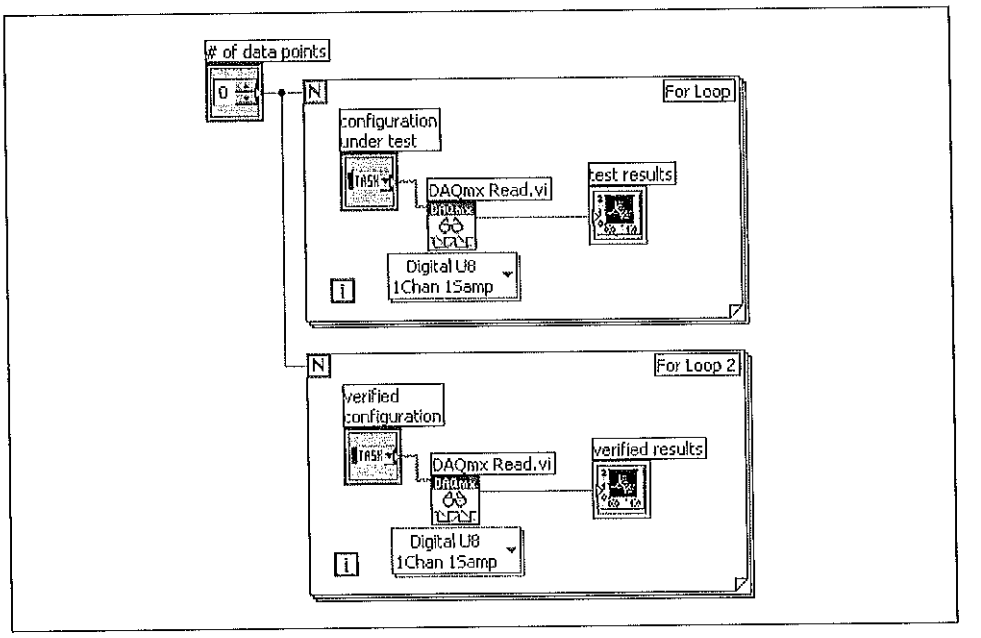
B. Functional Global Variables

C. Race Conditions

D. Synchronizing Data Transfer

A. Variables

In LabVIEW, the flow of data rather than the sequential order of commands determines the execution order of block diagram elements. Therefore, you can create block diagrams that have simultaneous operations. For example, you can run two For Loops simultaneously and display the results on the front panel, as shown in the following block diagram.



However, if you use wires to pass data between parallel block diagrams, they no longer operate in parallel. Parallel block diagrams can be two parallel loops on the same block diagram without any data flow dependency, or two separate VIs that are called at the same time.

The block diagram in Figure 2-1 does not run the two loops in parallel because of the wire between the two subVIs.

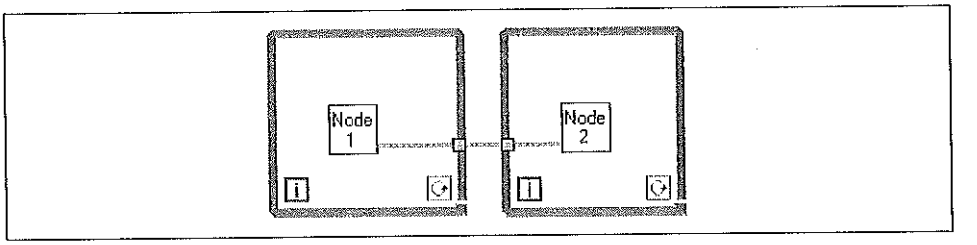


Figure 2-1. Data Dependency Imposed by Wire

The wire creates a data dependency, because the second loop does not start until the first loop finishes and passes the data through its tunnel. To make the two loops run concurrently, remove the wire. To pass data between the subVIs, use another technique, such as a variable.

In LabVIEW, *variables* are block diagram elements that allow you to access or store data in another location. The actual location of the data varies depending on the type of the variable. Local variables store data in front panel controls and indicators. Global variables and single-process shared variables store data in special repositories that you can access from multiple VIs. Functional global variables store data in While Loop shift registers. Regardless of where the variable stores data, all variables allow you to circumvent normal data flow by passing data from one place to another without connecting the two places with a wire. For this reason, variables are useful in parallel architectures, but also have certain drawbacks, such as race conditions.

Using Variables in a Single VI

Local variables transfer data within a single VI.

In LabVIEW, you read data from or write data to a front panel object using its block diagram terminal. However, a front panel object has only one block diagram terminal, and your application might need to access the data in that terminal from more than one location.

Local and global variables pass information between locations in the application that you cannot connect with a wire. Use local variables to access front panel objects from more than one location in a single VI. Use global variables to access and pass data among several VIs.

Creating Local Variables

Right-click an existing front panel object or block diagram terminal and select **Create»Local Variable** from the shortcut menu to create a local variable. A local variable icon for the object appears on the block diagram.

You also can select a local variable from the **Functions** palette and place it on the block diagram. The local variable node, shown as follows, is not yet associated with a control or indicator.



To associate a local variable with a control or indicator, right-click the local variable node and select **Select Item** from the shortcut menu. The expanded shortcut menu lists all the front panel objects that have owned labels.

LabVIEW uses owned labels to associate local variables with front panel objects, so label the front panel controls and indicators with descriptive owned labels.

Reading and Writing to Variables

After you create a local or global variable, you can read data from a variable or write data to it. By default, a new variable receives data. This kind of variable works as an indicator and is a write local or global. When you write new data to the local or global variable, the associated front panel control or indicator updates to the new data.

You also can configure a variable to behave as a data source, or a read local or global. Right-click the variable and select **Change To Read** from the shortcut menu to configure the variable to behave as a control. When this node executes, the VI reads the data in the associated front panel control or indicator.

To change the variable to receive data from the block diagram rather than provide data, right-click the variable and select **Change To Write** from the shortcut menu.

On the block diagram, you can distinguish read locals or globals from write locals or globals the same way you distinguish controls from indicators. A read local or global has a thick border similar to a control. A write local or global has a thin border similar to an indicator.

Local Variable Example

In the *Parallelism* section of Lesson 1, *Common Design Techniques*, you saw an example of a VI that used parallel loops. The front panel contained a single switch that stopped the data generation displayed on two graphs. On the block diagram, the data for each chart is generated within an individual While Loop to allow for separate timing of each loop. The Loop Control terminal stopped both While Loops. In this example, the two loops must share the switch to stop both loops at the same time.

For both charts to update as expected, the While Loops must operate in parallel. Connecting a wire between While Loops to pass the switch data makes the While Loops execute serially, rather than in parallel. Figure 2-2 shows a block diagram of this VI using a local variable to pass the switch data.

Loop 2 reads a local variable associated with the switch. When you set the switch to False on the front panel, the switch terminal in Loop 1 writes a False value to the conditional terminal in Loop 1. Loop 2 reads the **Loop Control** local variable and writes a False to the Loop 2 conditional terminal. Thus, the loops run in parallel and terminate simultaneously when you turn off the single front panel switch.

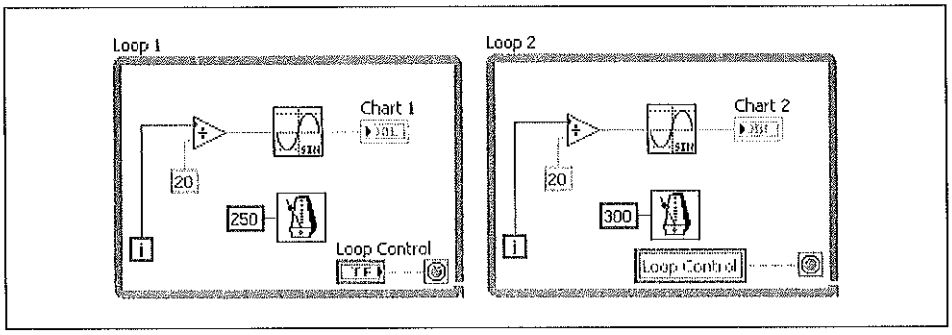


Figure 2-2. Local Variable Used to Stop Parallel Loops

With a local variable, you can write to or read from a control or indicator on the front panel. Writing to a local variable is similar to passing data to any other terminal. However, with a local variable you can write to it even if it is a control or read from it even if it is an indicator. In effect, with a local variable, you can access a front panel object as both an input and an output.

For example, if the user interface requires users to log in, you can clear the `Login` and `Password` prompts each time a new user logs in. Use a local variable to read from the **Login** and **Password** string controls when a user logs in and to write empty strings to these controls when the user logs out.

Using Variables Among VIs

You also can use variables to access and pass data among several VIs that run simultaneously. A local variable shares data within a VI. A global variable also shares data, but it shares data among multiple VIs. For example, suppose you have two VIs running simultaneously. Each VI contains a While Loop and writes data points to a waveform chart. The first VI contains a Boolean control to terminate both VIs. You can use a global variable to terminate both loops with a single Boolean control. If both loops were on a single block diagram within the same VI, you could use a local variable to terminate the loops.

You also can use a single-process shared variable in the same way you use a global variable. A shared variable is similar to a local variable or a global variable, but allows you to share data across a network. A shared variable can be single-process or network-published. Although network-published shared variables are beyond the scope of this course, by using the single-process shared variable, you can later change to a network-published shared variable.

Use a global variable to share data among VIs on the same computer, especially if you do not use a project file. Use a single-process shared variable if you may need to share the variable information among VIs on multiple computers in the future.

Creating Global Variables

Use global variables to access and pass data among several VIs that run simultaneously. Global variables are built-in LabVIEW objects. When you create a global variable, LabVIEW automatically creates a special global VI, which has a front panel but no block diagram. Add controls and indicators to the front panel of the global VI to define the data types of the global variables it contains. In effect, this front panel is a container from which several VIs can access data.

For example, suppose you have two VIs running simultaneously. Each VI contains a While Loop and writes data points to a waveform chart. The first VI contains a Boolean control to terminate both VIs. You must use a global variable to terminate both loops with a single Boolean control. If both loops were on a single block diagram within the same VI, you could use a local variable to terminate the loops.

Complete the following steps to create a global variable.

1. Select a global variable, shown as follows, from the **Functions** palette and place it on the block diagram.



2. Double-click the global variable node to display the front panel of the global VI. Place controls and indicators on this front panel the same way you do on a standard front panel.
3. LabVIEW uses owned labels to identify global variables, so label the front panel controls and indicators with descriptive owned labels.

   You can create several single global VIs, each with one front panel object, or if you want to group similar variables together, you can create one global VI with multiple front panel objects.

   You can create several single global variables, each with one front panel object, or you can create one global variable with multiple front panel objects.

   A global variable with multiple objects is more efficient because you can group related variables together. The block diagram of a VI can include several global variable nodes that are associated with controls and indicators on the front panel of a global variable. These global variable nodes are either copies of the first global variable node that you placed on the block diagram of the global VI, or they are the global variable nodes of global VIs that you placed on the current VI. You place global VIs on other VIs the same way you place subVIs on other VIs. Each time you place a new global variable node on a block diagram, LabVIEW creates a new VI associated only with that global variable node and copies of it.

Figure 2-3 shows a global variable front panel window with a numeric, a string, and a cluster containing a numeric and a Boolean control. The toolbar does not show the **Run**, **Stop**, or related buttons as a normal front panel window.

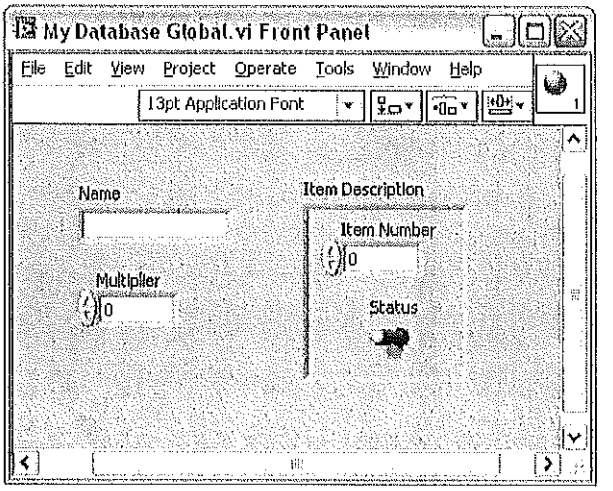


Figure 2-3. Global Variable Front Panel Window

4. After you finish placing objects on the global VI front panel, save it and return to the block diagram of the original VI. You must then select the object in the global VI that you want to access. Click the global variable node and select a front panel object from the shortcut menu. The shortcut menu lists all the front panel objects in the global VI that have owned labels. You also can right-click the global variable node and select a front panel object from the **Select Item** shortcut menu.

   You also can use the Operating tool or Labeling tool to click the global variable node and select the front panel object from the menu that displays.

   If you want to use this global variable in other VIs, select the **Select a VI** option on the **Functions** palette. By default, the global variable is associated with the first front panel object with an owned label that you placed in the global VI. Right-click the global variable node you placed on the block diagram and select a front panel object from the **Select Item** shortcut menu to associate the global variable with the data from another front panel object.

Creating Single-Process Shared Variables

You must use a project file to use a shared variable. To create a single-process shared variable, right-click **My Computer** in the **Project Explorer** window and select **New»Variable**. The **Shared Variable Properties** dialog box appears, as shown in Figure 2-4.

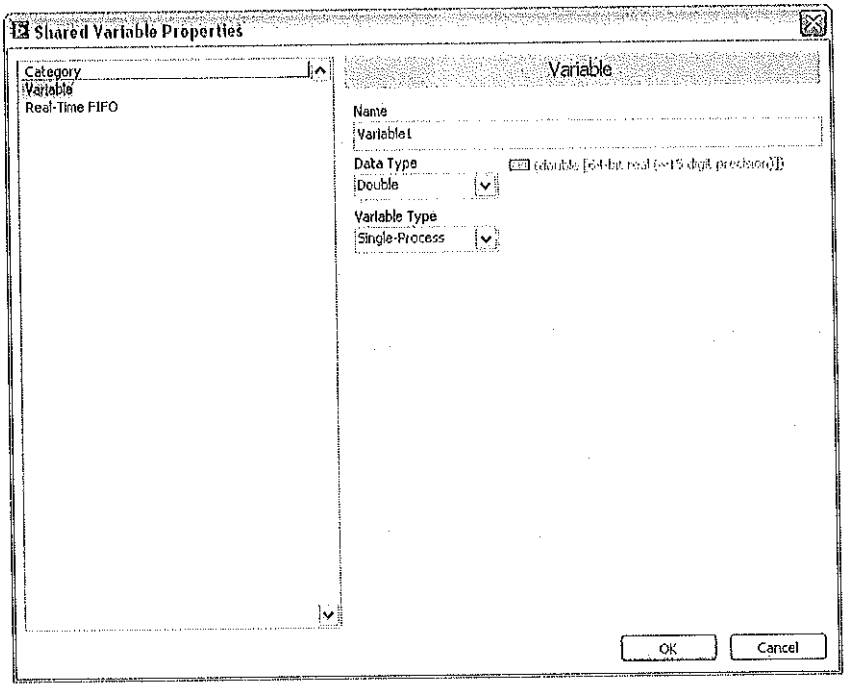


Figure 2-4. Shared Variable Properties Dialog Box

Under **Variable Type**, select **Single Process**. Give the variable a name and a data type. After you create the global variable, it automatically appears in a new library in your project file. Save the library. You can add additional global variables to this library as needed. You can drag and drop the variable from the listing in the **Project Explorer** window directly to the block diagram. Use the short-cut menu to switch between writing or reading. Use the error clusters on the variable to impose data flow.

Using Variables Carefully

Local and global variables are advanced LabVIEW concepts. They are inherently not part of the LabVIEW dataflow execution model. Block diagrams can become difficult to read when you use local and global variables, so you should use them carefully. Misusing local and global variables, such as using them instead of a connector pane or using them to access values in each frame of a sequence structure, can lead to unexpected behavior in VIs. Overusing local and global variables, such as using them to avoid long wires across the block diagram or using them instead of data flow, slows performance.

Variables often are used unnecessarily. The example in Figure 2-5 shows a traffic light application implemented as a state machine. Each state updates the lights for the next stage of the light sequence. In the state shown, the east and west traffic has a green light, while the north and south traffic has a red light. This stage waits for 4 seconds, as shown by the Wait (ms) function.

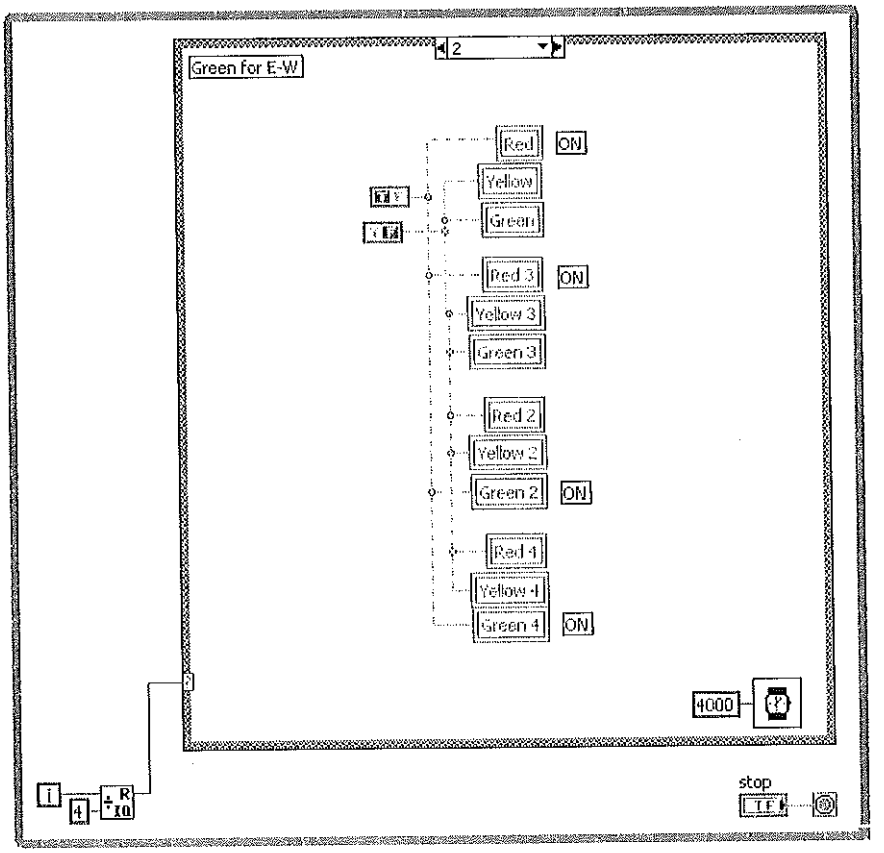


Figure 2-5. Too Many Variables Used

The example shown in Figure 2-6 accomplishes the same task, but more efficiently and using a better design. Notice that this example is much easier to read and understand than the previous example, mostly by reducing variable use. By placing the indicators in the While Loop outside the Case structure, the indicators can update after every state without using a variable. This example is less difficult to modify for further functionality, such as adding left turn signals, than the previous example.

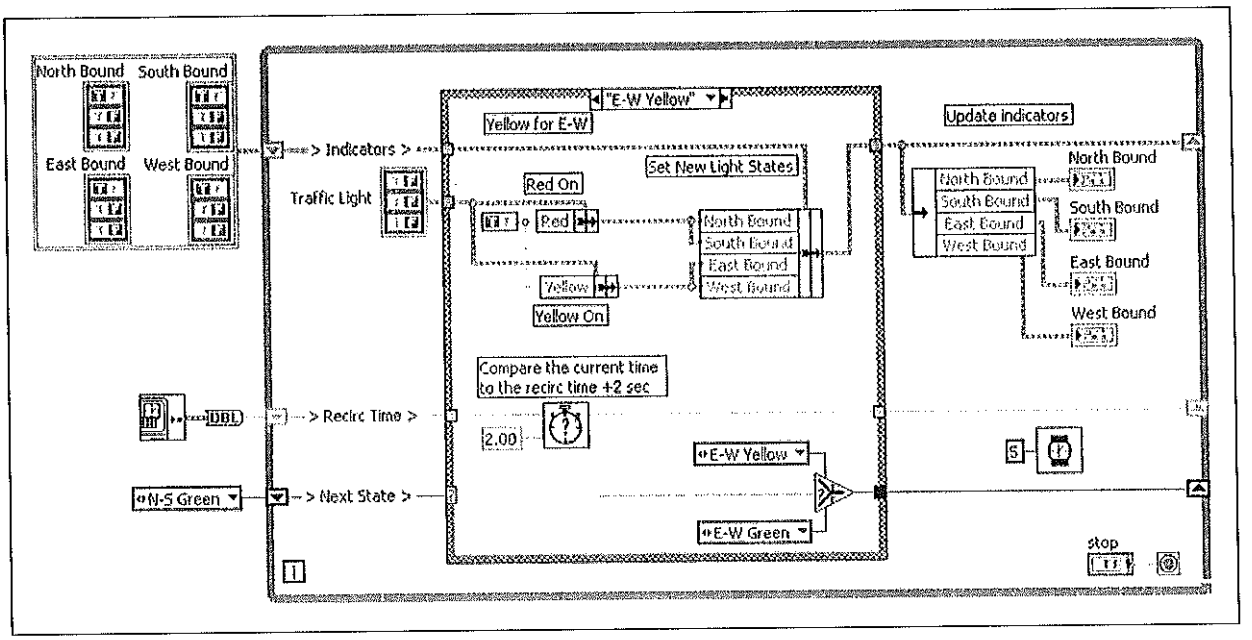


Figure 2-6. Reduced Variables

Initializing Variables

To initialize a local or global variable, verify that the variable contains known data values before the VI runs. Otherwise, the variables might contain data that causes the VI to behave incorrectly. If the variable relies on a computation result for the initial value, make sure LabVIEW writes the value to the variable before it attempts to access the variable for any other action. Wiring the write action in parallel with the rest of the VI can cause a race condition.

To make sure it executes first, you can isolate the code that writes the initial value for the variable to the first frame of a Sequence structure or to a subVI and wire the subVI to execute first in the data flow of the block diagram.

If you do not initialize the variable before the VI reads the variable for the first time, the variable contains the default value of the associated front panel object.

Figure 2-7 shows a common mistake when using variables. A shared variable synchronizes the stop conditions for two loops. This example operates the first time it runs, because the default value of a Boolean is False. However, each time this VI runs, the **Stop** control writes a True value to the variable. Therefore, the second and subsequent times that this VI runs, the lower loop stops after only a single iteration unless the first loop updates the variable quickly enough.

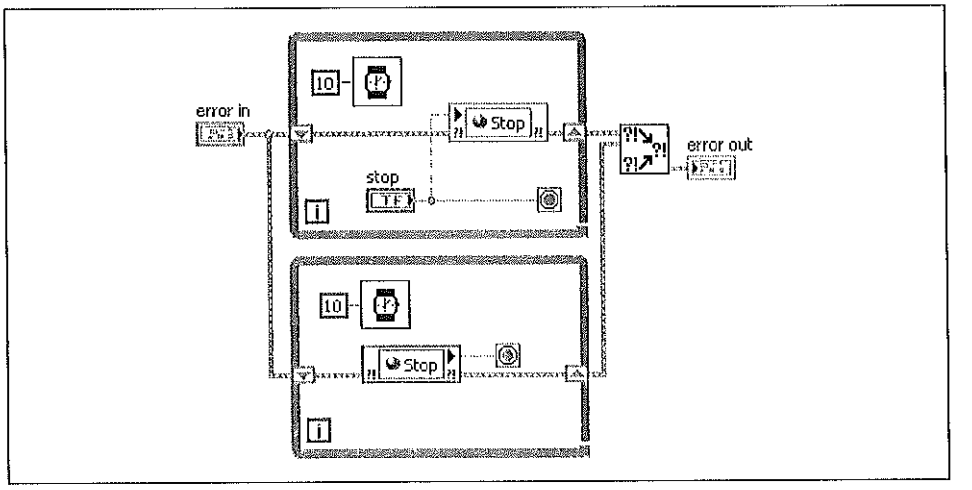


Figure 2-7. Failing to Initialize a Shared Variable

Figure 2-8 shows the VI with code added to initialize the shared variable. Initialize the variable before the loops begin to insure that the second loop does not immediately stop.

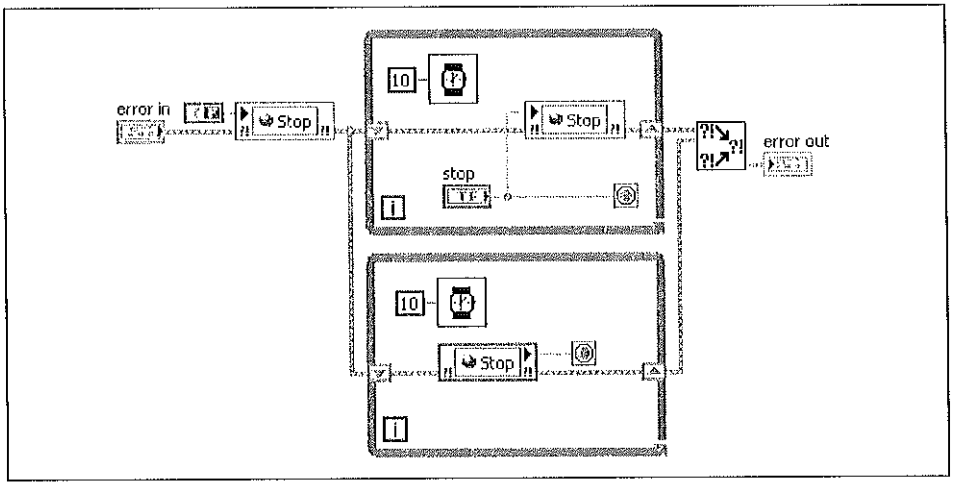


Figure 2-8. Initializing a Shared Variable Properly

B. Functional Global Variables

You can use uninitialized shift registers in For Loops or While Loops to store data as long as the VI is in memory. The shift register holds the last state of the shift register. Place a While Loop within a subVI and use the shift registers to store data that can be read from or written to. Using this technique is similar to using a global variable. This method is often called a functional global variable. The advantage to this method over a global variable is that you can control access to the data in the shift register. The general form of a functional global variable includes an uninitialized shift register with a single iteration For Loop or While Loop, as shown in Figure 2-9.

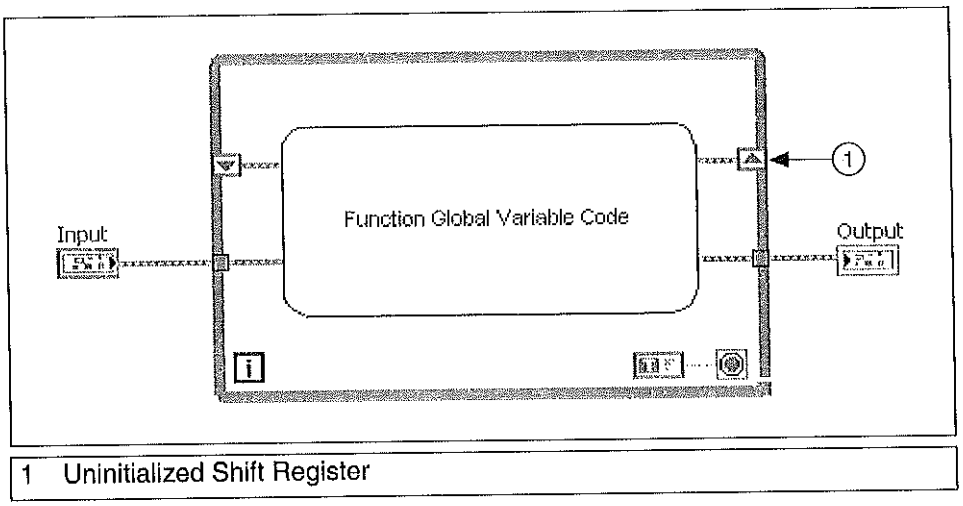


Figure 2-9. Functional Global Variable Format

A functional global variable usually has an action input parameter that specifies which task the VI performs. The VI uses an uninitialized shift register in a While Loop to hold the result of the operation.

Figure 2-10 shows a simple functional global variable with set and get functionality.

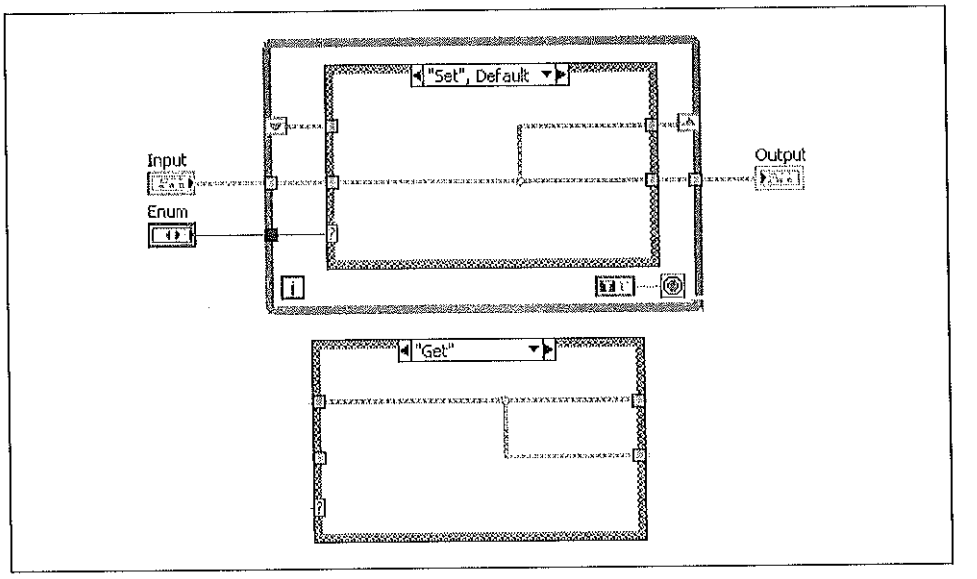


Figure 2-10. Functional Global Variable with Set and Get Functionality

In this example, data passes into the VI and the shift register stores the data if you configure the enumerated data type to Set. Data is retrieved from the shift register if the enumerated data type is configured to Get.

Although you can use functional global variables to implement simple global variables, as shown in the previous example, they are especially useful when implementing more complex data structures, such as a stack or a queue buffer. You also can use functional global variables to protect access to global resources, such as files, instruments, and data acquisition devices, that you cannot represent with a global variable.

Note A functional global variable is a subVI that is not reentrant. This means that when the subVI is called from multiple locations, the same copy of the subVI is used. Therefore, only one call to the subVI can occur at a time.

Using Functional Global Variables for Timing

One powerful application of functional global variables is to perform timing in your VI. Many VIs that perform measurement and automation require some form of timing. Often an instrument or hardware device needs time to initialize, and you must build explicit timing into your VI to take into account the physical time required to initialize a system. You can create a functional global variable that measures the elapsed time between each time the VI is called, as shown in Figure 2-11.

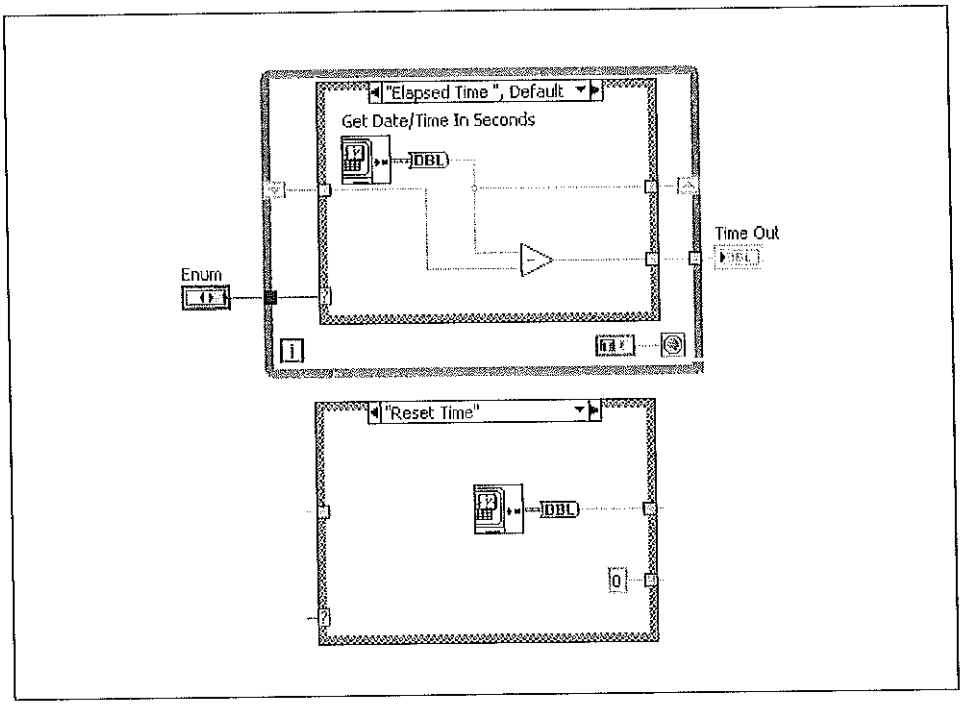


Figure 2-11. Elapsed Time Functional Global Variable

The Elapsed Time case gets the current date and time in seconds and subtracts it from the time that is stored in the shift register. The Reset Time case initializes the functional global variable with a known time value.

The Elapsed Time Express VI implements the same functionality as this functional global variable. The benefit of using the functional global variable is that you can customize the implementation easily, such as adding a pause option.

Exercise 2-1 Variables VI

Goal

Use variables to write to and read from a control

Scenario

You have a LabVIEW Project that implements a temperature weather station. The weather station acquires a temperature every half a second, analyzes each temperature to determine if the temperature is too high or too low, then alerts the user if there is a danger of a heat stroke or freeze. The VI logs the data if a warning occurs.

Two front panel controls determine the setpoints—the temperature upper limit and the temperature lower limit. However, nothing prevents the user from setting a lower limit that is higher than the upper limit.

Use variables to set the lower limit equal to the upper limit if the user sets a lower limit that is higher than the upper limit.

Design

The VIs in this project have already been written. Your only task is to modify the VIs so that the lower limit is set equal to the upper limit when necessary.

State Definitions

The following table describes the states in the state machine.

State	Description	Next State
Acquisition	Set time to zero, acquire data from the temperature sensor, and read front panel controls	Analysis
Analysis	Determine warning level	Data Log if a warning occurs, Time Check if no warning occurs
Data Log	Log the data in a tab-delimited ASCII file	Time Check
Time Check	Check whether time is greater than or equal to .5 seconds	Acquisition if time has elapsed, Time Check if time has not elapsed

Changing the value of the lower temperature limit control should happen after the user has entered the value but before the value determines the warning level. Therefore, make the modifications to the VI in the Acquisition or Analysis state, or place a new state between the two.

1. Before determining which option to use, take a closer look at the content of the Acquisition and Analysis states:

   ❑ Open the `Weather Station` project located in the `<Exercises>\LabVIEW Basics II\Variables` directory.



Tip Folder names in angle brackets, such as `<Exercises>`, refer to folders on the root directory of your computer.

   ❑ Open `Weather Station UI.vi`.

   ❑ Review the contents of the Acquisition and Analysis states, which correspond to the Acquisition and Analysis cases of the Case structure.

Design Options

You have three different design options for modifying this project.

Option	Description	Benefits/Drawbacks
1	Insert a Case structure in the Acquisition state to reset the controls before a local variable writes the values to the cluster.	Poor design: the acquisition state has another task added, rather than focusing only on acquisition.
2	Insert a new state in the state machine that checks the controls and resets them if necessary.	Ability to control when the state occurs.
3	Modify the Determine Warnings subVI to reset the controls.	Easy to implement because functionality is already partially in place. However, if current functionality is used, one set of data always is lost when resetting the lower limit control.

This exercise implements Option 2 as a solution.

New State Definitions for Option 2

The following table describes the new state definitions to implement.

State	Description	Next State
Acquisition	Acquire data from the temperature sensor on channel AI0 and read front panel controls	Range Check
Range Check	Read front panel controls and set the lower limit equal to the upper limit if upper limit less than the lower limit	Analysis
Analysis	Determine warning level	Data Log if a warning occurs, Time Check if no warning occurs
Data Log	Log the data in a tab-delimited ASCII file	Time Check
Time Check	Check whether time is greater than or equal to .5 seconds	Acquisition if time has elapsed, Time Check if time has not elapsed

Implementation

1. If the `Weather Station.lvproj` is not already open, open it from the `<Exercises>\LabVIEW Basics II\Variables` directory.



Note If you do not have a data acquisition device and a DAQ Signal Accessory available, use the files located in the `<Exercises>\LabVIEW Basics II\No Hardware Required\Variables` directory.

2. Add the Range Check state to the state machine.

   - ❑ From the **Project Explorer** window, open the `Weather Station States.ctl` by double-clicking the listing. This is the type-defined enumerated control that defines the states for the state machine.
   - ❑ Right-click the control and select **Edit Items** from the shortcut menu.

- ❑ Insert an item and modify to match Table 2-1. Be careful not to add an empty listing.

Table 2-1. States Enumerated Control

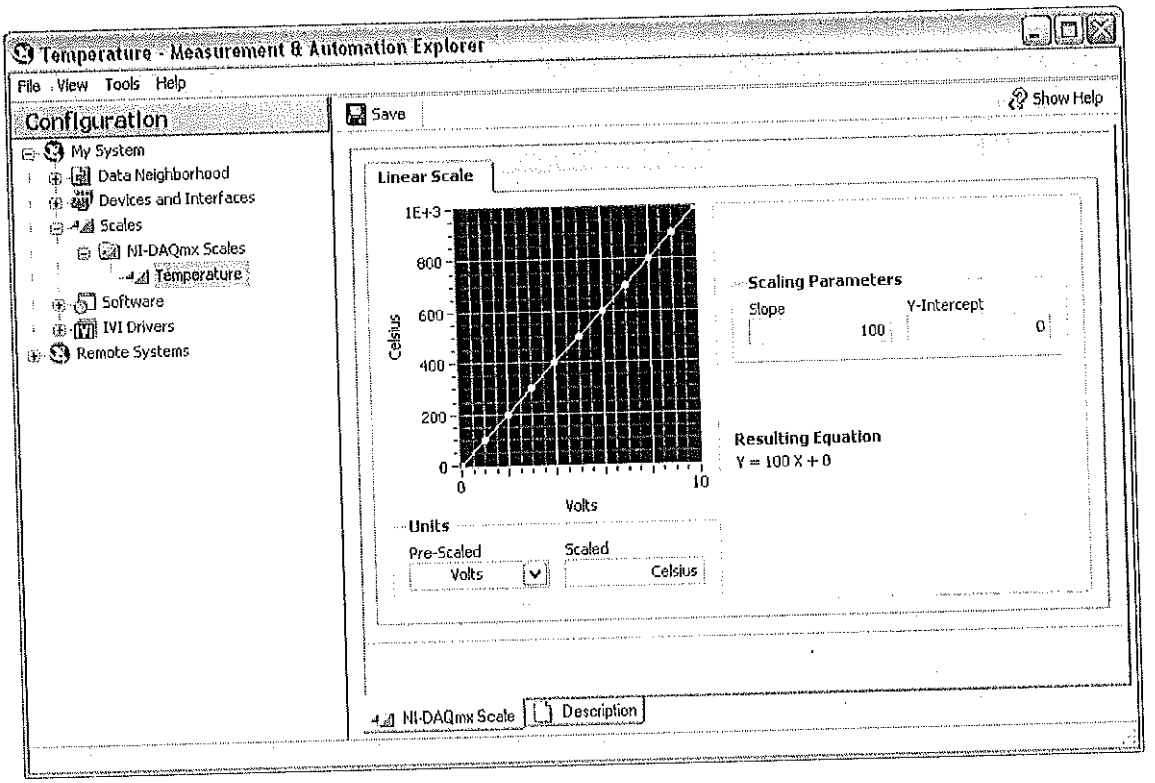
Item	Digital Display
Acquisition	0
Range Check	1
Analysis	2
Data Log	3
Time Check	4

- ❑ Save and close the control.
- ❑ If the `Weather Station UI.vi` is not open, open it by double-clicking the listing in the **Project Explorer** window.
- ❑ Open the block diagram.
- ❑ Right-click the state machine Case structure and select **Add Case for Every Value** from the shortcut menu. Because the enumerated control has a new value, a new case appears in the Case structure.



Note You may have already completed this step in the *LabVIEW Basics I: Introduction* course.

3. In Measurement & Automation Explorer (MAX), create a custom scale for the temperature sensor on the DAQ Signal Accessory. The sensor conversion is linear, and the formula is Voltage × 100 = Celsius.



- ❑ Launch MAX by double-clicking the icon on the desktop or by selecting **Tools»Measurement & Automation Explorer** in LabVIEW.
- ❑ Right-click the **Scales** section and select **Create New** from the shortcut menu.
- ❑ Select **NI-DAQmx Scale**.
- ❑ Click **Next**.
- ❑ Select **Linear**.
- ❑ Name the scale `Temperature`.
- ❑ Click **Finish**.
- ❑ Change the Scaling Parameters **Slope** to `100`.
- ❑ Enter `Celsius` as the **Scaled Units**.
- ❑ Click the **Save** button on the toolbar to save the scale.
- ❑ Close MAX by selecting **File»Exit**.

4. Read the upper and lower limit controls in the Range Check state, instead of the Acquisition state.

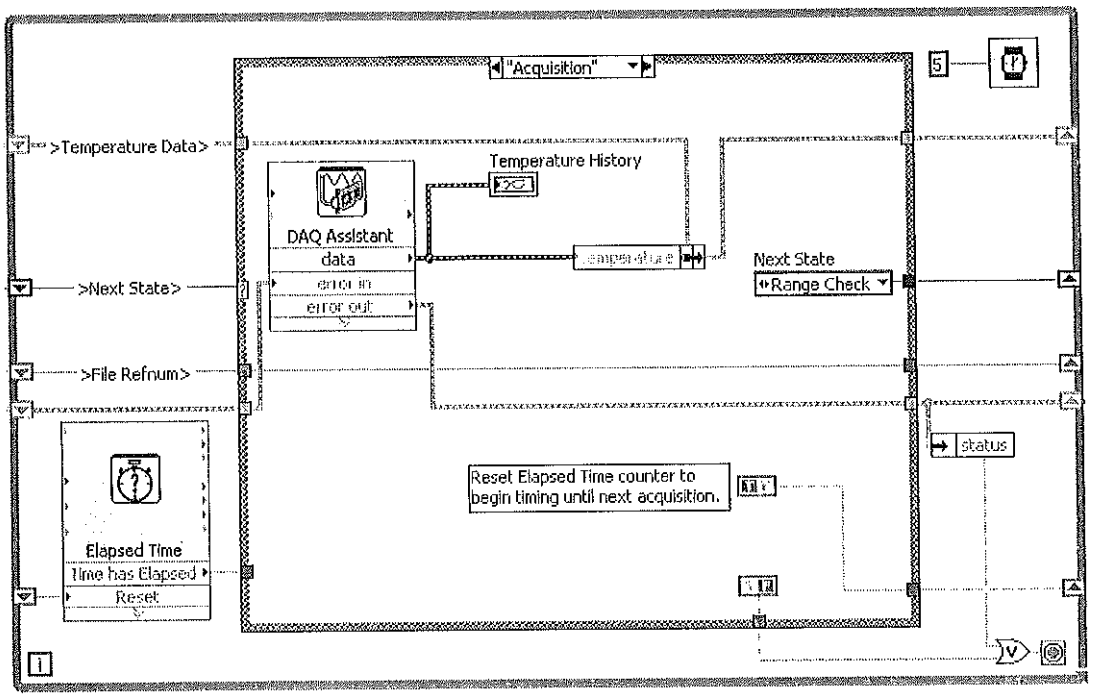


Figure 2-12. Completed Acquisition State

- ❑ Return to the block diagram of the Weather Station UI VI. Select the Acquisition case in the state machine Case structure.
- ❑ Inside the Acquisition case, change the **Next State** enum to Range Check.
- ❑ Make a copy of the **Next State** enum by pressing <Ctrl> and dragging a copy outside the While Loop.
- ❑ Move the Upper Limit and Lower Limit numeric controls outside the While Loop.
- ❑ Resize the Bundle by Name function to one element, as shown in Figure 2-12.
- ❑ Select the **Range Check** case in the state machine Case structure.
- ❑ Move the Upper Limit and Lower Limit numeric controls and the Next State enum into the Range Check state.

5. Set the Range Check state to transition to the Analysis state.

   - ❑ In the Range Check case, wire the **Next State** enum to the **Next State** output tunnel.
   - ❑ Change the **Next State** enum to Analysis.

6. If the Upper Limit is less than the Lower Limit, use a local variable to write the Upper Limit value to the Lower Limit control.

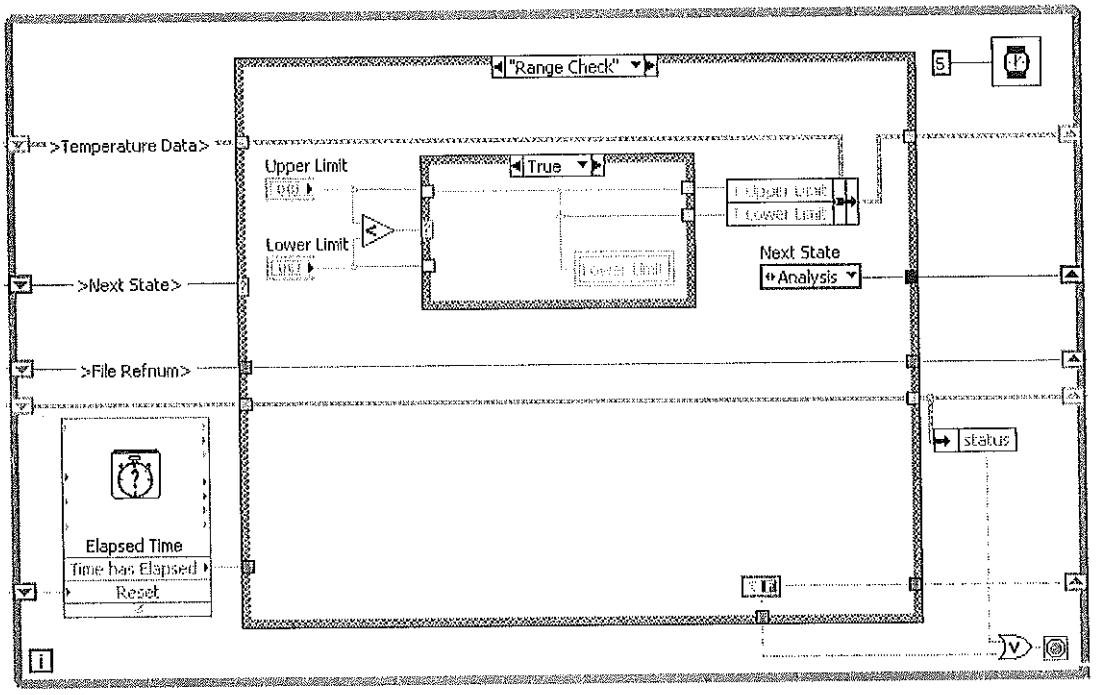


Figure 2-13. Completed Range Check State—True



- ❑ Add a Less? function to the Range Check state.



- ❑ Add a Case structure to the right of the Less? function.
- ❑ Wire the Upper Limit and Lower Limit controls to the Less? function and the Case structure as shown in Figure 2-13.
- ❑ Right-click the Lower Limit control and select **Create»Local Variable** from the shortcut menu.
- ❑ Move the local variable inside the True case of the Case structure.



- ❑ Add a Bundle By Name function to the right of the Case structure.
- ❑ Wire the Temperature Data cluster to the **input cluster** input of the Bundle By Name function.

- ❑ Expand the Bundle By Name function to two elements.
- ❑ Select T Upper Limit in the first element and T Lower Limit in the second element.
- ❑ Add a False constant to the outer Case structure.
- ❑ Wire the case as shown in Figure 2-13.

7. If the Upper Limit is equal to or greater than the Lower Limit, pass the values of the controls to the temperature cluster.

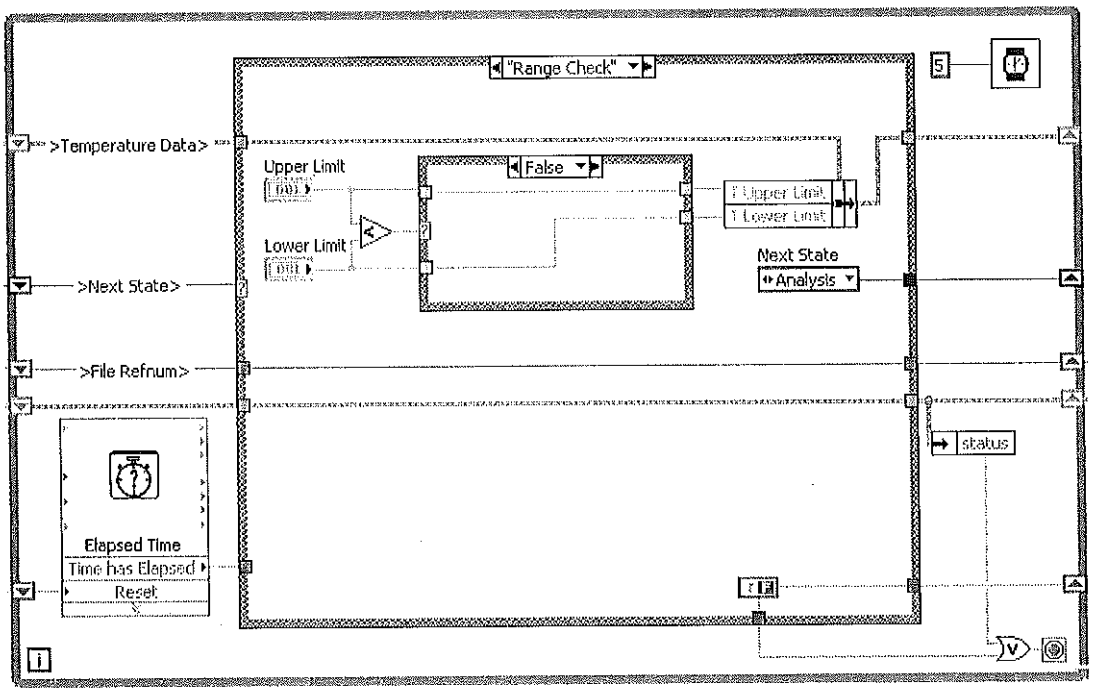


Figure 2-14. Completed Range Check State—False

- ❑ Switch to the False case of the interior Case structure.
- ❑ Wire the Upper Limit and Lower Limit data through the case.

8. Save the VI.
9. Save the Project.

Test

1. Run the VI.
   - ❑ Name the log file when prompted.
   - ❑ Enter a value in the **Upper Limit** control that is less than the value in the **Lower Limit** control. Does the VI behave as expected?
2. Stop the VI when you are finished.
3. Close the VI and the project.

End of Exercise 2-1

Exercise 2-2 Optional: Global Data Project

Goal

Create a project containing multiple VIs that share data using a single-process shared variable.

Scenario

Create a VI that generates a sine wave. Create a second VI that displays the sine wave and allows the user to modify the time between each acquisition of the sine wave data. Use one stop button to stop both VIs.

Design

Two VIs and two pieces of global data are necessary to implement the VI:

- First VI: generate sine, write sine to Data shared variable, read Stop shared variable to stop loop
- Second VI: read Data shared variable, display on chart, write Stop button to Stop shared variable
- First shared variable: Stop (Boolean data type)
- Second shared variable: Data (Numeric data type)

Implementation

1. Open a blank project.
2. Save the project as `Global Data.lvproj` in the `<Exercises>\LabVIEW Basics II\Global Data` directory.
3. Create the Stop shared variable.
   - ❑ Give the variable the following properties.
     - Name: `Stop`
     - Data Type: Boolean
     - Variable Type: Single-process
   - ❑ Click **OK** to close the **Shared Variable Properties** dialog box. Notice that a new library is created in the **Project Explorer** window to hold the variable.
4. Save the library.
   - ❑ Right-click the library and select **Save** from the shortcut menu.
   - ❑ Save the library as `Global Data.lvlib` in the `<Exercises>\LabVIEW Basics II\Global Data` directory.

5. Create the Data shared variable.
   - ❑ Switch to the **Project Explorer** window.
   - ❑ Right-click **Global Data.lvlib** and select **New»Variable** from the shortcut menu.
   - ❑ Give the new variable the following properties:
     - Name: `Data`
     - Data Type: Double
     - Variable Type: Single-process
   - ❑ Click **OK** to close the **Shared Variable Properties** dialog box.

Generate Data VI

1. Open a blank VI.
2. Save the VI as `Generate Data.vi` in the `<Exercises>\LabVIEW Basics II\Global Data` directory.
3. Add a Numeric Indicator to the front panel window.
4. Name the Numeric Indicator `Data`.
5. Switch to the block diagram of the VI.
6. Create the block diagram shown in Figure 2-15. No implementation instructions are given. Labels are shown to assist you.

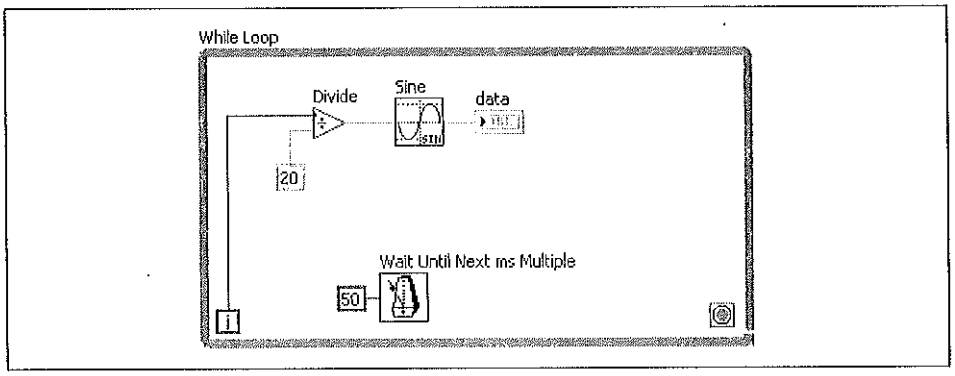


Figure 2-15. Generate Data Block Diagram without Variables

7. Save the VI.

8. Write the data generated to the Data shared variable.

   - ❑ Select the **Data** shared variable from the **Project Explorer** window and drag it inside the While Loop of the Generate Data VI block diagram.
   - ❑ Right-click the global variable and select **Change to Write** from the shortcut menu.
   - ❑ Wire the **Sin(x)** output of the Sine function to the **Data** shared variable.

9. Read the Stop shared variable to stop the While Loop.

   - ❑ Switch to the **Project Explorer** window.
   - ❑ Select the **Stop** shared variable and drag it inside the While Loop of the `Generate Data.vi` block diagram.
   - ❑ Wire the **Stop** shared variable to the **Loop Condition** terminal.

10. Initialize the Stop shared variable.

    - ❑ Switch to the **Project Explorer** window.
    - ❑ Select the **Stop** shared variable and drag it to the left of the While Loop of the `Generate Data.vi` block diagram.
    - ❑ Right-click the Stop shared variable and select **Change to Write** from the shortcut menu.
    - ❑ Right-click the input of the **Stop** shared variable and select **Create» Constant** from the shortcut menu to create a False constant.
    - ❑ Use the Operating tool to change the constant to False if necessary.

11. Use the shared variable error clusters to ensure the order of operations. Refer to Figure 2-16 for assistance wiring this block diagram.

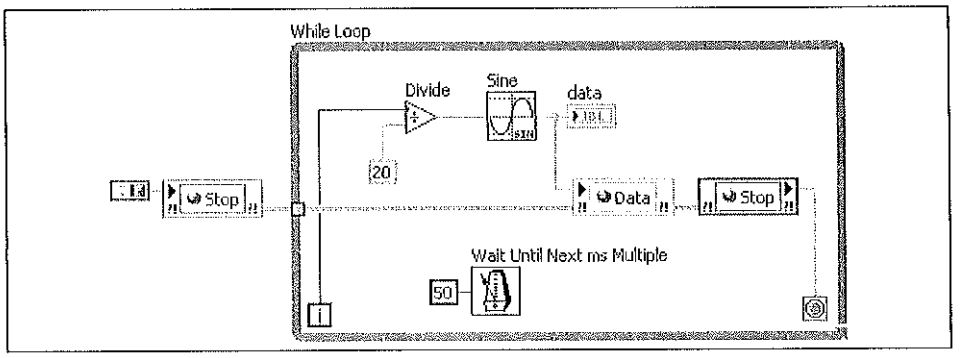


Figure 2-16. Generate Data Block Diagram with Shared Variables

12. Save the VI.

13. Close the block diagram, but leave the front panel open.

Read Data VI

1. Open a blank VI.

2. Save the VI as `Read Data.vi` in the `<Exercises>\LabVIEW Basics II\Global Data` directory.

3. Create the front panel shown in Figure 2-17.

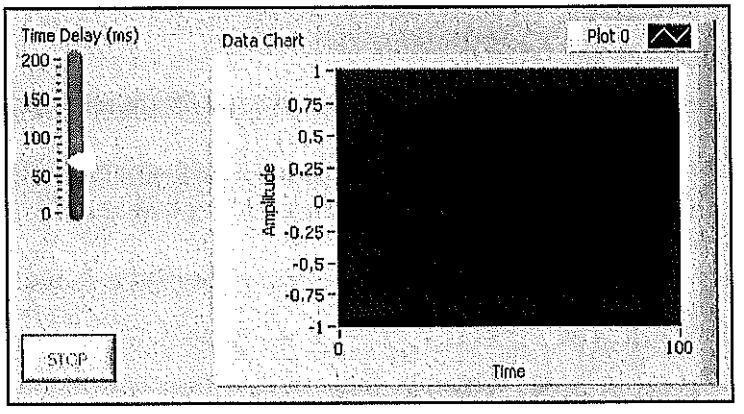


Figure 2-17. Read Data Front Panel



4. Add a vertical pointer slide and rename it `Time Delay (ms)`.

   - ❑ Change the range of the slide by entering `200` in the top value shown.

   - ❑ Right-click the slide and select **Representation»U32** from the shortcut menu.



- ❑ Add a waveform chart and rename it `Data Chart`.
- ❑ Change the *x*-scale and *y*-scale ranges and labels of the chart to the values shown in Figure 2-17.



- ❑ Add a Stop button and hide the label.

5. Open the block diagram.
6. Create the block diagram shown in Figure 2-18. Labels are shown to assist you.

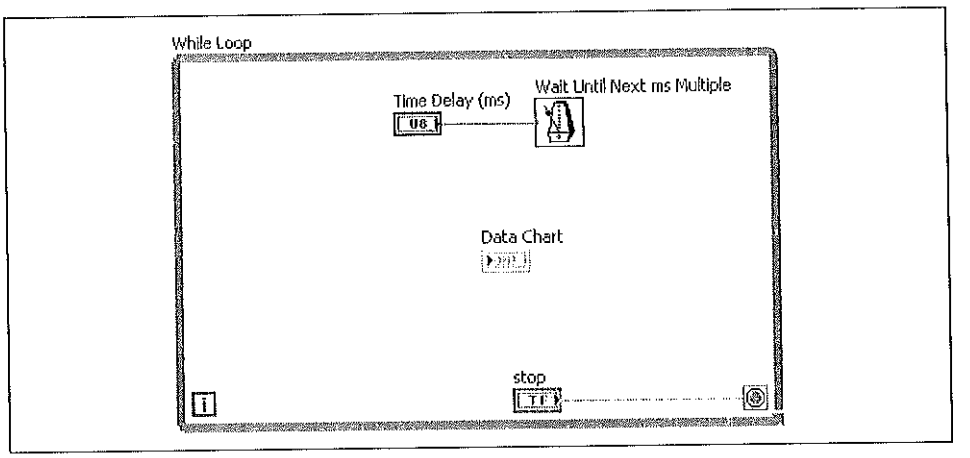


Figure 2-18. Read Data Block Diagram w/o Shared Variables

7. Read the data from the Data shared variable and display it on the waveform chart.
   - ❑ Switch to the Project Explorer window.
   - ❑ Select the **Data** shared variable and drag it inside the While Loop of the Read Data VI block diagram.
   - ❑ Wire the output of the **Data** shared variable to the Data Chart indicator.
8. Write the value of the Stop control to the Stop shared variable.
   - ❑ Switch to the **Project Explorer** window.
   - ❑ Select the **Stop** shared variable and drag it inside the While Loop of the `Read Data.vi` block diagram.
   - ❑ Right-click the **Stop** shared variable and select **Change to Write** from the shortcut menu.
   - ❑ Wire the **Stop** control to the **Stop** shared variable.

9. Use the shared variable error clusters to ensure the order of operations. Refer to Figure 2-19 for assistance wiring this block diagram.

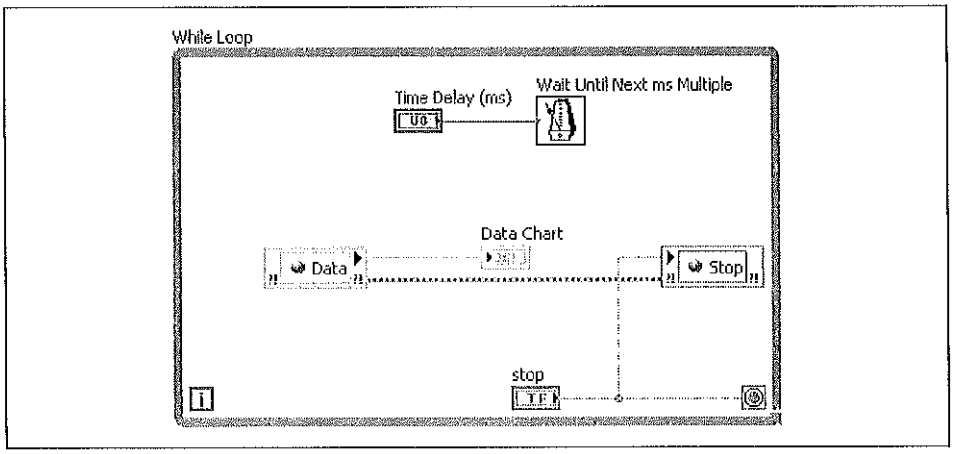


Figure 2-19. Read Data Block Diagram with Shared Variables

10. Save the VI.

11. Close the block diagram.

12. Save the project.

Test

1. Run the Generate Data VI.

2. Run the Read Data VI.

3. Modify the value of the Time Delay (ms) control.

   The Time Delay (ms) control determines how often the shared variable is read. What happens if you set the Time Delay to zero? When accessing global data, you may read the value more than once before it is updated to a new value, or you may miss a new value altogether, depending on the value of the Time Delay.

4. Stop and close the VIs and the project when you are finished.

Challenge

Create a functional global variable to handle the Stop data and use it in the Generate Data VI and the Read Data VI to share the stop button between the two VIs.

End of Exercise 2-2

C. Race Conditions

A race condition occurs when the timing of events or the scheduling of tasks unintentionally affects an output or data value. Race conditions are a common problem for programs that execute multiple tasks in parallel and share data between them. Consider the following example in Figures 2-20 and 2-21.

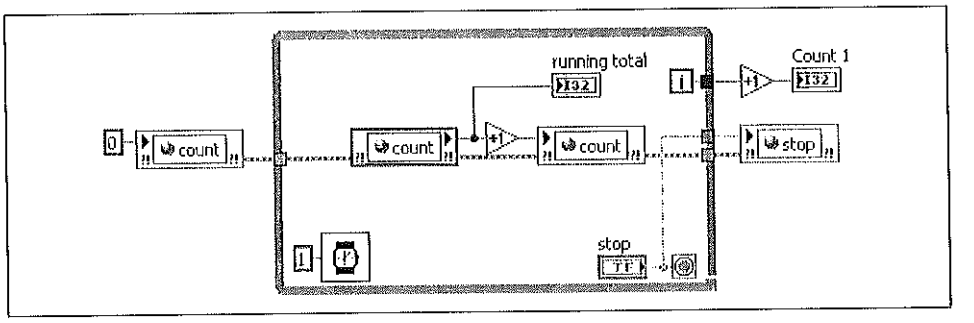


Figure 2-20. Race Condition Example: Loop 1

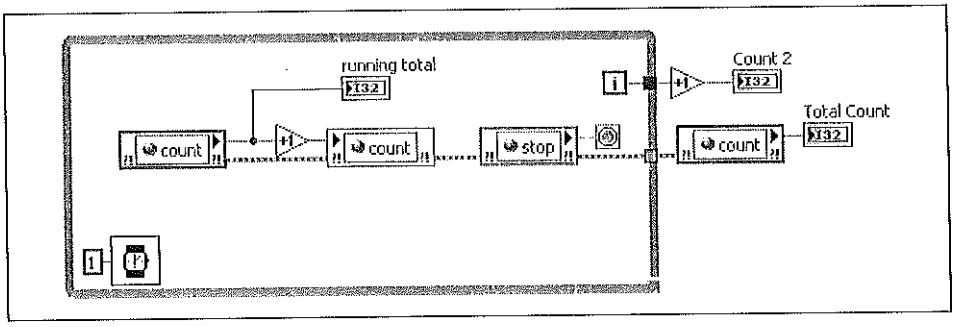


Figure 2-21. Race Condition Example: Loop 2

The two loops both increment a shared variable during each iteration. If you run this VI, the expected result after clicking the **Stop** button is that the **Total Count** is equal to the sum of **Count 1** and **Count 2**. If you run the VI for a short period of time, you generally see the expected result. However, if you run the VI for a longer period of time, the **Total Count** is less than the sum of **Count 1** and **Count 2**, because this VI contains a race condition.

On a single processor computer, actions in a multi-tasking program like this example actually happen sequentially, but LabVIEW and the operating system rapidly switch tasks so that the tasks effectively execute at the same time. The race condition in this example occurs when the switch from one task to the other occurs at a certain time. Notice that both of the loops perform the following operations:

- Read the shared variable.
- Increment the value read.
- Write the incremented value to the shared variable.

9. Use the shared variable error clusters to ensure the order of operations. Refer to Figure 2-19 for assistance wiring this block diagram.

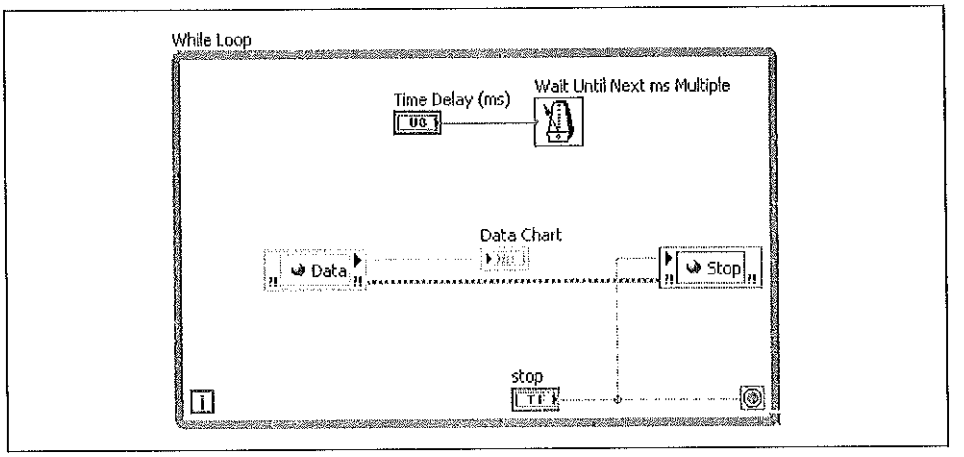


Figure 2-19. Read Data Block Diagram with Shared Variables

10. Save the VI.

11. Close the block diagram.

12. Save the project.

Test

1. Run the Generate Data VI.

2. Run the Read Data VI.

3. Modify the value of the Time Delay (ms) control.

   The Time Delay (ms) control determines how often the shared variable is read. What happens if you set the Time Delay to zero? When accessing global data, you may read the value more than once before it is updated to a new value, or you may miss a new value altogether, depending on the value of the Time Delay.

4. Stop and close the VIs and the project when you are finished.

Challenge

Create a functional global variable to handle the Stop data and use it in the Generate Data VI and the Read Data VI to share the stop button between the two VIs.

End of Exercise 2-2

C. Race Conditions

A race condition occurs when the timing of events or the scheduling of tasks unintentionally affects an output or data value. Race conditions are a common problem for programs that execute multiple tasks in parallel and share data between them. Consider the following example in Figures 2-20 and 2-21.

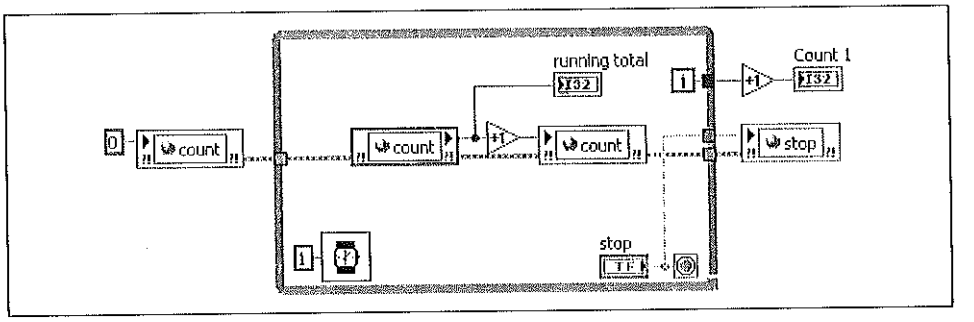


Figure 2-20. Race Condition Example: Loop 1

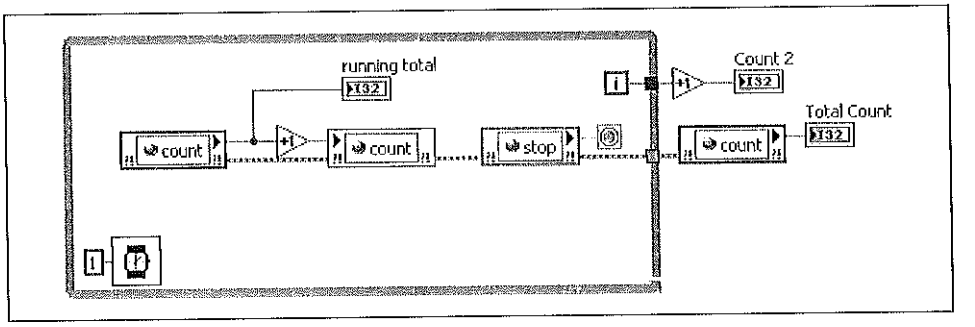


Figure 2-21. Race Condition Example: Loop 2

The two loops both increment a shared variable during each iteration. If you run this VI, the expected result after clicking the **Stop** button is that the **Total Count** is equal to the sum of **Count 1** and **Count 2**. If you run the VI for a short period of time, you generally see the expected result. However, if you run the VI for a longer period of time, the **Total Count** is less than the sum of **Count 1** and **Count 2**, because this VI contains a race condition.

On a single processor computer, actions in a multi-tasking program like this example actually happen sequentially, but LabVIEW and the operating system rapidly switch tasks so that the tasks effectively execute at the same time. The race condition in this example occurs when the switch from one task to the other occurs at a certain time. Notice that both of the loops perform the following operations:

- Read the shared variable.
- Increment the value read.
- Write the incremented value to the shared variable.

Now consider what happens if the loop operations happen to be scheduled in the following order:

1. Loop 1 reads the shared variable.
2. Loop 2 reads the shared variable.
3. Loop 1 increments the value it read.
4. Loop 2 increments the value it read.
5. Loop 1 writes the incremented value to the shared variable.
6. Loop 2 writes the incremented value to the shared variable.

In this example, both loops write the same value to the variable, and the increment of the first loop is effectively overwritten by Loop 2. This generates a race condition, which can cause serious problems if you intend the program to calculate an exact count.

In this particular example, there are few instructions between when the shared variable is read and when it is written. Therefore, the VI is less likely to switch between the loops at the wrong time. This explains why this VI runs accurately for short periods and only loses a few counts for longer periods.

Race conditions are difficult to identify and debug, because the outcome depends upon the order in which the operating system executes scheduled tasks and the timing of external events. The way tasks interact with each other and the operating system, as well as the arbitrary timing of external events, make this order essentially random. Often, code with a race condition can return the same result thousands of times in testing, but still can return a different result, which can appear when the code is in use.

The best way to avoid race conditions is by using the following techniques:

- Controlling and limiting shared resources.
- Identifying and protecting critical sections within your code.
- Specifying execution order.

Controlling and Limiting Shared Resources

Race conditions are most common when two tasks have both read and write access to a resource, as is the case in the previous example. A resource is any entity that is shared between the processes. When dealing with race conditions, the most common shared resources are data storage, such as variables. Other examples of resources include files and references to hardware resources.

Allowing a resource to be altered from multiple locations often introduces the possibility for a race condition. Therefore, an ideal way to avoid race

conditions is to minimize shared resources and the number of writers to the remaining shared resources. In general, it is not harmful to have multiple readers or monitors for a shared resource. However, try to use only one writer or controller for a shared resource. Most race conditions only occur when a resource has multiple writers.

In the previous example, you can reduce the dependency upon shared resources by having each loop maintain its count locally. Then, share the final counts after clicking the **Stop** button. This involves only a single read and a single write to a shared resource and eliminates the possibility of a race condition. If all shared resources have only a single writer or controller, and the VI has a well sequenced instruction order, then race conditions do not occur.

Protecting Critical Sections

A critical section of code is code that must behave consistently in all circumstances. When you use multi-tasking programs, one task may interrupt another task as it is running. In nearly all modern operating systems, this happens constantly. Normally, this does not have any effect upon running code, however, when the interrupting task alters a shared resource that the interrupted task assumes is constant, then a race condition occurs.

Figures 2-20 and 2-21 contain critical code sections. If one of the loops interrupts the other loop while it is executing the code in its critical section, then a race condition can occur. One way to eliminate race conditions is to identify and protect the critical sections in your code. There are many techniques for protecting critical sections. Two of the most effective are functional global variables and semaphores.

Functional Global Variables

One way to protect critical sections is to place them in subVIs. You can only call a non-reentrant subVI from one location at a time. Therefore, placing critical code in a non-reentrant subVI keeps the code from being interrupted by other processes calling the subVI. Using the functional global variable architecture to protect critical sections is particularly effective, because shift registers can replace less protected storage methods like global or single-process shared variables. Functional global variables also encourage the creation of multi-functional subVIs that handle all tasks associated with a particular resource.

After you identify each section of critical code in your VI, group the sections by the resources they access, and create one functional global variable for each resource. Critical sections performing different operations each can become a command for the functional global variable, and you can group

critical sections that perform the same operation into one command, thereby re-using code.

You can use functional global variables to protect critical sections of code in Figure 2-20 and Figure 2-21. To remove the race condition, replace the shared variables with a functional global variable and place the code to increment the counter within the functional global variable, as shown in Figure 2-22, Figure 2-23, and Figure 2-24.

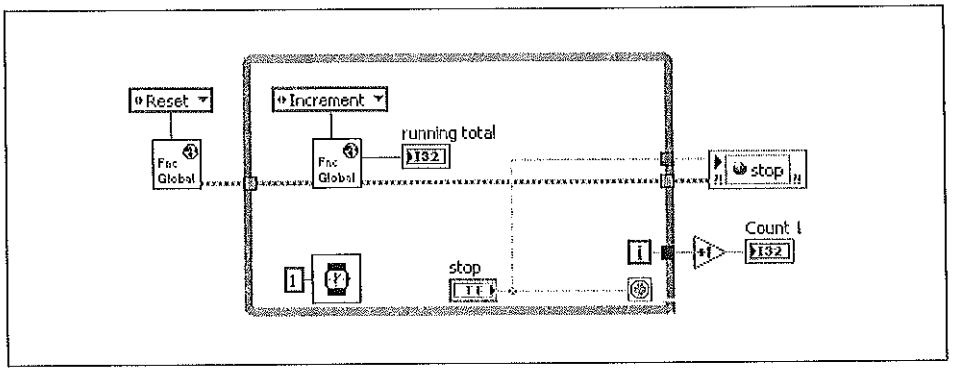


Figure 2-22. Using Functional Global Variables to Protect the Critical Section in Loop 1

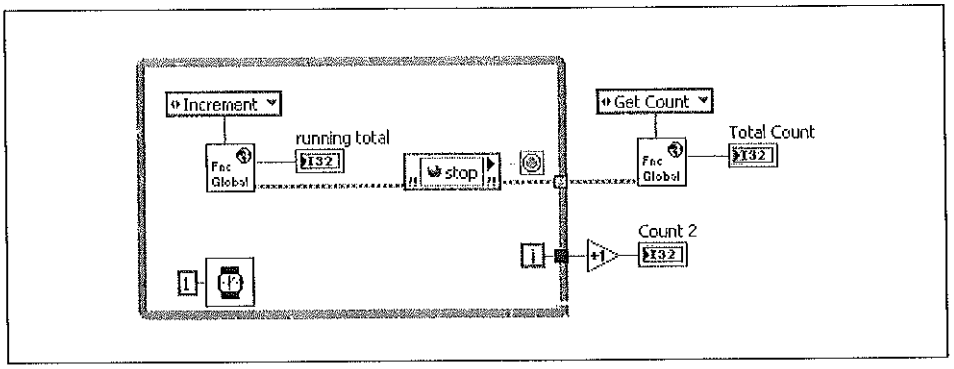


Figure 2-23. Using Functional Global Variables to Protect the Critical Section in Loop 2

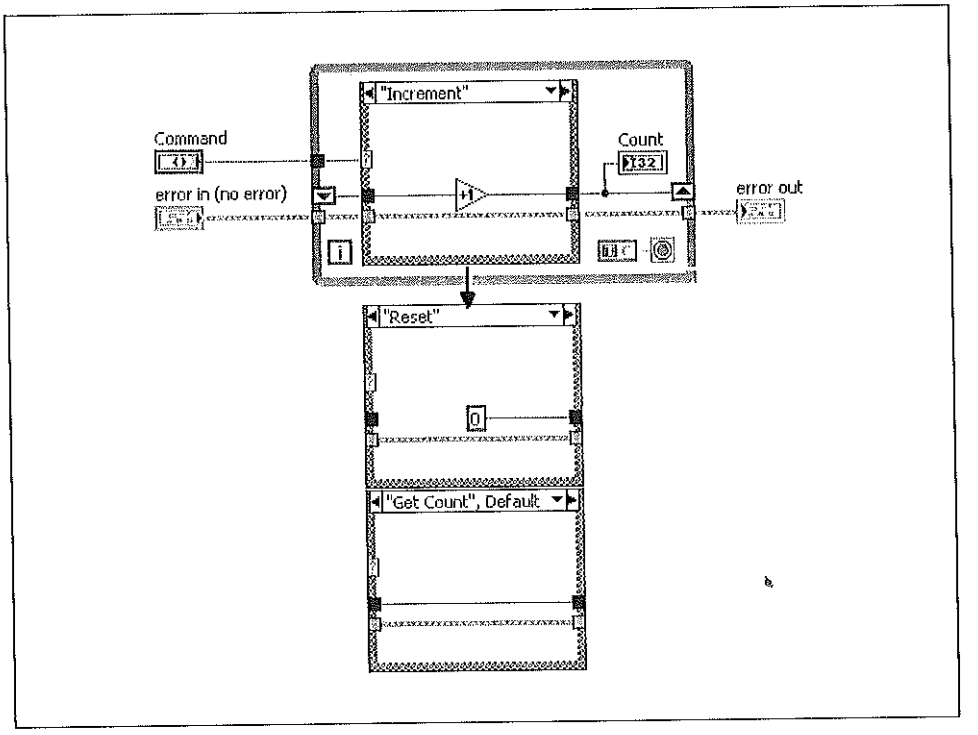


Figure 2-24. Functional Global Variable Eliminates the Race Condition

Semaphores

Semaphores are synchronization mechanisms specifically designed to protect resources and critical sections of code. You can prevent critical sections of code from interrupting each other by enclosing each between an Acquire Semaphore and Release Semaphore VI. By default, a semaphore only allows one task to acquire it at a time. Therefore, after one of the tasks enters a critical section, the other tasks cannot enter their critical sections until the first task completes. When done properly, this eliminates the possibility of a race condition.

You can use semaphores to protect the critical sections of the VIs, as shown in Figure 2-20 and Figure 2-21. A named semaphore allows you to share the semaphore between VIs. You must open the semaphore in each VI, then acquire it just before the critical section and release it after the critical section. Figure 2-25 and Figure 2-26 show a solution to the race condition using semaphores.

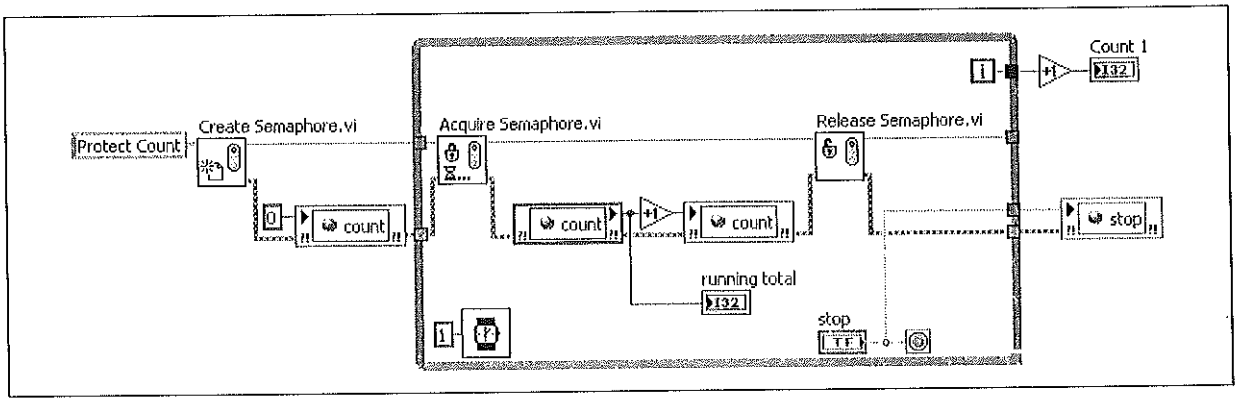


Figure 2-25. Protecting the Critical Section with a Semaphore in Loop 1

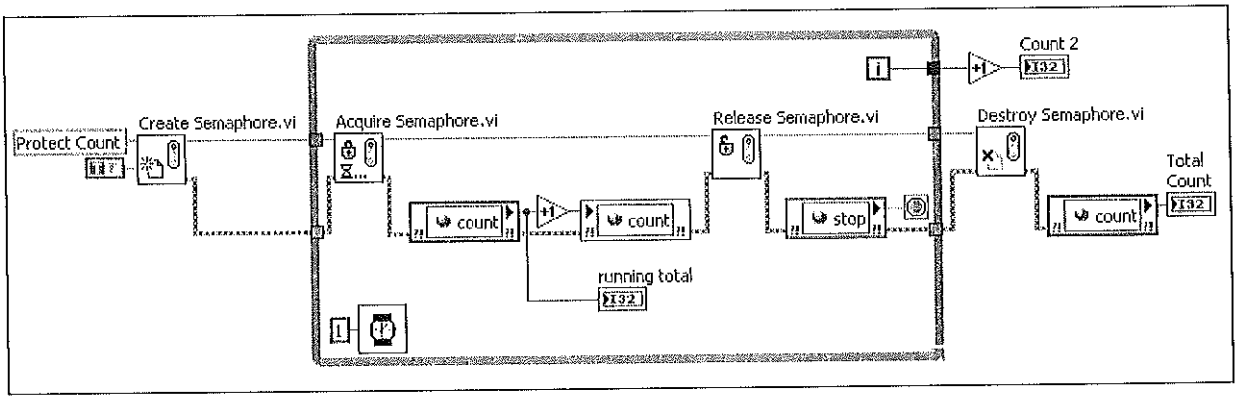


Figure 2-26. Protecting the Critical Section with a Semaphore in Loop 2

Specifying Execution Order

Code in which data flow is not properly used to control the execution order can cause some race conditions. When a data dependency is not established, LabVIEW can schedule tasks in any order, which creates the possibility for race conditions if the tasks depend upon each other. Consider the example in Figure 2-27.

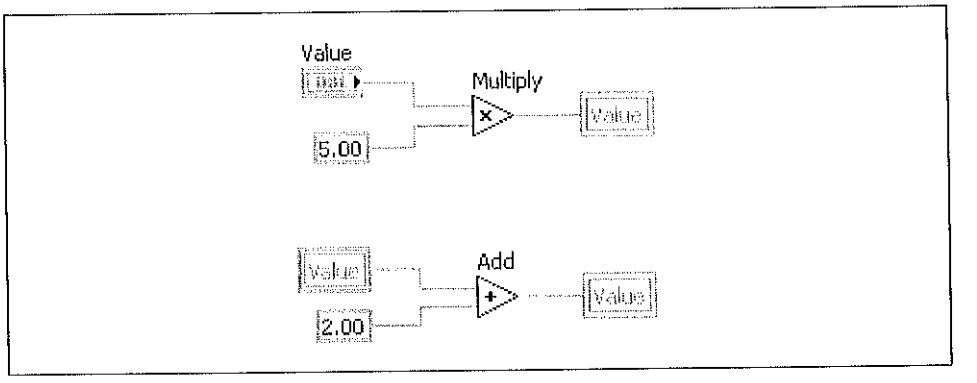


Figure 2-27. Simple Race Condition

The code in this example has four possible outcomes, depending on the order in which the operations execute.

Outcome 1: Value = (Value × 5) + 2

1. Terminal reads Value.
2. Value × 5 is stored in Value.
3. Local variable reads Value × 5.
4. (Value × 5) + 2 is stored in Value.

Outcome 2: Value = (Value + 2) × 5

1. Local variable reads Value.
2. Value + 2 is stored in Value.
3. Terminal reads Value + 2.
4. (Value + 2) × 5 is stored in Value.

Outcome 3: Value = Value × 5

1. Terminal reads Value.
2. Local variable reads Value.
3. Value + 2 is stored in Value.
4. Value × 5 is stored in Value.

Outcome 4: Value = Value + 2

1. Terminal reads Value.
2. Local variable reads Value.
3. Value × 5 is stored in Value.
4. Value + 2 is stored in Value.

Although this code is considered a race condition, the code generally behaves less randomly than the first race condition example. This is because LabVIEW usually assigns a consistent order to the operations. However, avoid situations such as this because the order and the behavior of the VI can vary. For example, the order could change when running the VI under different conditions or when upgrading the VI to a newer version of LabVIEW. Fortunately, race conditions of this nature are easily remedied by controlling the data flow.

Exercise 2-3 Concept: Bank VI

Goal

Eliminate a race condition in a VI.

Description

You must identify and fix a problem with the server software in a bank. The bank server handles requests from many sources and must process the requests quickly. In order to increase its efficiency, the server uses two parallel loops—one to handle deposits to the account and another to handle withdrawals. The problem with the server is that some deposit or withdrawal requests are lost, thereby resulting in incorrect balances.

Identify Race Condition

1. Open `Bank.vi` in the `<Exercises>\LabVIEW Basics II\Bank` directory.
2. Run the VI.
3. Perform a deposit, a withdrawal, and a simultaneous transaction to familiarize yourself with the program.
4. Set the **Deposit Amount** to `20` and the **Withdrawal Amount** to `10`.
5. Open the block diagram of the Bank VI while it is still running.
6. Arrange the block diagram of the Bank VI so that you can see it while operating the user interface.
7.  Enable execution highlighting on the block diagram by clicking **Highlight Execution**.
8. Click the **Simultaneous Transactions** button and watch the code as it executes. The balance should increase by 10.

   Notice that either the deposit or the withdrawal is lost, causing the balance to increase by 20 or decrease by 10.
9. Stop the VI.

You tracked the problem down to a race condition in a section of a code handling deposits and withdrawals for a single account. Although you can see the issue with execution highlighting enabled, during regular operation, the issue would occur sporadically.

Remove Race Condition

Remove the race condition by protecting the critical section of code using a semaphore. In the VI, the critical sections of code are those enclosed by a Sequence structure.

1. Save the VI as `Bank with Semaphores.vi` in the `<Exercises>\LabVIEW Basics II\Bank` directory.

2. Use semaphores to protect the critical sections of code, as shown in Figure 2-28.

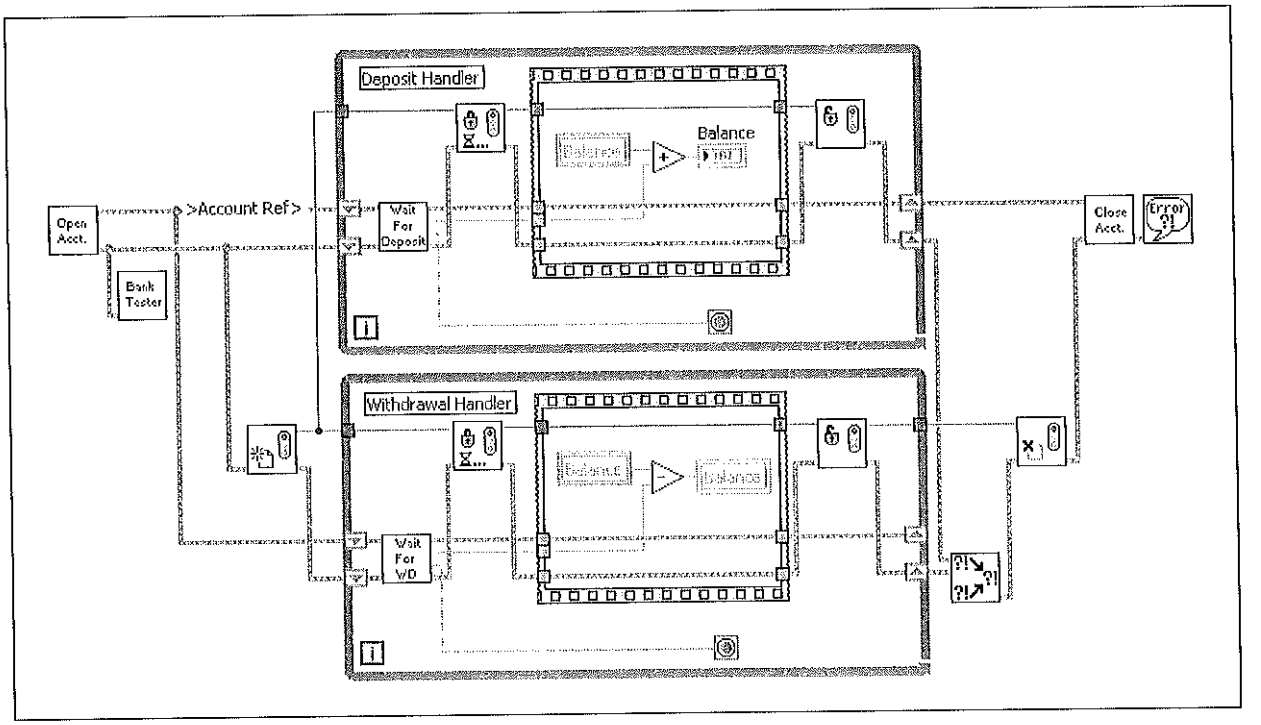


Figure 2-28. Bank with Semaphore



- ❑ Add a Create Semaphore VI to the left of the While Loops.
- ❑ Wire the Create Semaphore VI as shown in Figure 2-28.



- ❑ Add an Acquire Semaphore VI to the Deposit Handler loop, to the left of the Sequence structure.
- ❑ Add a second Acquire Semaphore VI to the Withdrawal Handler loop to the left of the Sequence structure.
- ❑ Wire the Acquire Semaphore VIs as shown in Figure 2-28.



- ❑ Add a Release Semaphore VI to the Deposit Handler loop, to the right of the Sequence structure.

- ❑ Add a second Release Semaphore VI to the Withdrawal Handler loop, to the right of the Sequence structure.
- ❑ Wire the Release Semaphore VIs as shown in Figure 2-28.



- ❑ Add a Destroy Semaphore VI to the right of the While Loops.
- ❑ Wire the Destroy Semaphore VI as shown in Figure 2-28. Notice that the Destroy Semaphore VI requires only the reference to the semaphore.

3. Save the VI.
4. Repeat the steps detailed in the *Test* section to test the modification to this VI.
5. Close the VI when you are finished.

End of Exercise 2-3

D. Synchronizing Data Transfer

Variables are very useful in LabVIEW for passing data between parallel processes. Notifiers and queues are methods for passing data between parallel processes that have advantages over using variables because of the ability to synchronize the transfer of data.

Variables

For parallel loops to communicate, you must use some form of globally available shared data. Using a global variable breaks the LabVIEW dataflow paradigm, allows for race conditions, and incurs more overhead than passing the data by wire.

The example shown in Figure 2-29 is a less effective implementation of a master/slave design pattern. This example uses a variable, which causes two problems—there is no timing between the master and the slave, and the variable can cause race conditions. The master cannot signal the slave that data is available, so the slave loop must continually poll the variable to determine if the data changes.

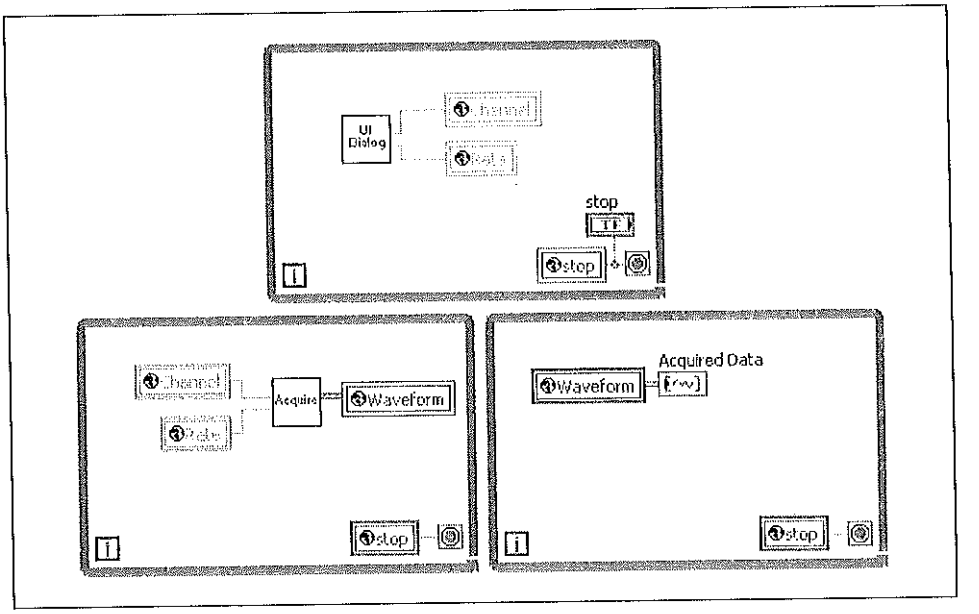


Figure 2-29. Master/Slave Architecture Using Global Variables

Notifiers

A more effective implementation of the master/slave design pattern uses notifiers to synchronize data transfer. A notifier sends data along with a notification that the data is available. Using a notifier to pass data from the master to the slave removes any issues with race conditions. Using notifiers also provides a synchronization advantage because the master and slave are timed when data is available, providing for an elegant implementation of the master/slave design pattern. Figure 2-30 shows the master/slave design pattern using notifiers.

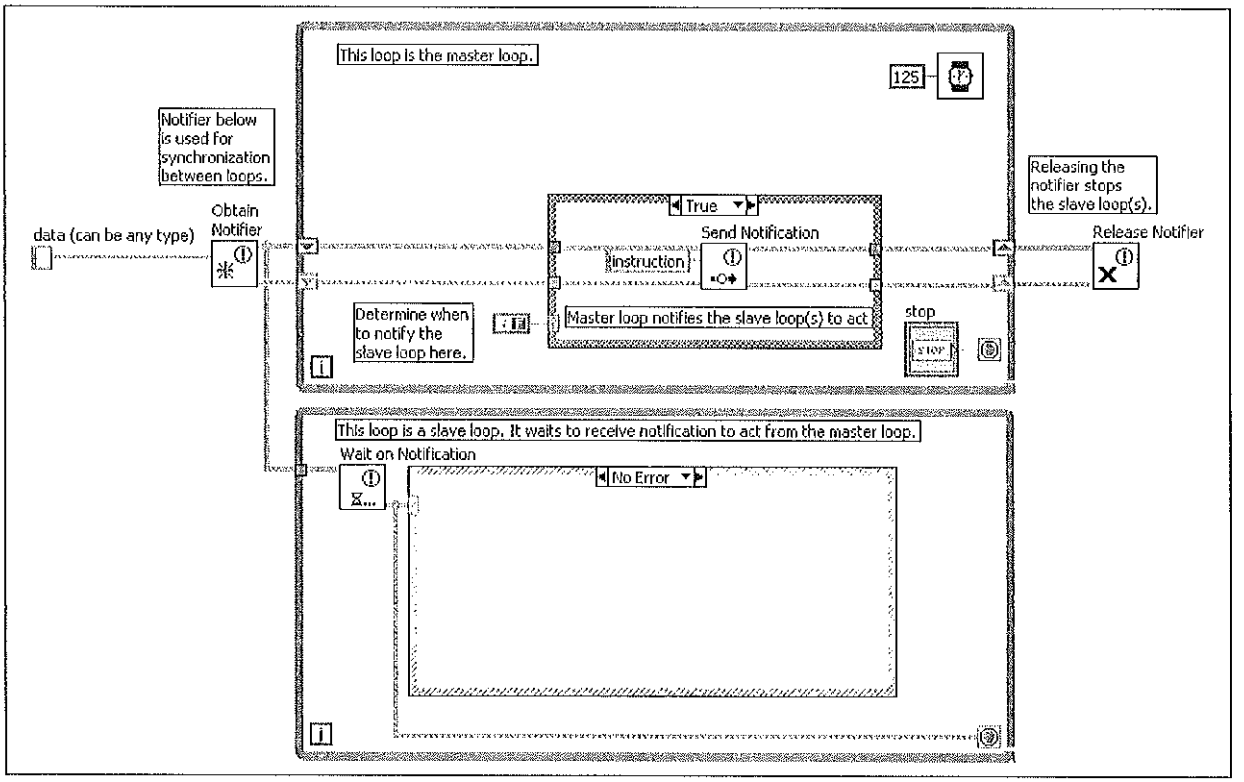


Figure 2-30. Master/Slave Design Pattern Using Notifiers

The notifier is created before the loops begin using the Obtain Notifier function. The master loop uses the Send Notification function to notify the slave loop through the Wait on Notification function. After the VI has finished using the notifiers, the Release Notifier function releases the notifiers.

The following benefits result from using notifiers in the master/slave design pattern:

- Both loops are synchronized to the master loop. The slave loop only executes when the master loop sends a notification.

- You can use notifiers to create globally available data. Thus, you can send data with a notification. For example, in Figure 2-30, the Send Notification function sends the string `instruction`.
- Using notifiers creates efficient code. You need not use polling to determine when data is available from the master loop.

However, using notifiers can have drawbacks. A notifier does not buffer the data. If the master loop sends another piece of data before the slave loop(s) reads the first piece of data, that data is overwritten and lost.

Queues

Queues are similar to notifiers, except that a queue can store multiple pieces of data. By default, queues work in a first in, first out (FIFO) manner. Therefore, the first piece of data inserted into the queue is the first piece of data that is removed from the queue. Use a queue when you want to process all data placed in the queue. Use a notifier if you want to process only the current data.

When used, the producer/consumer design pattern, queues pass data and synchronize the loops.

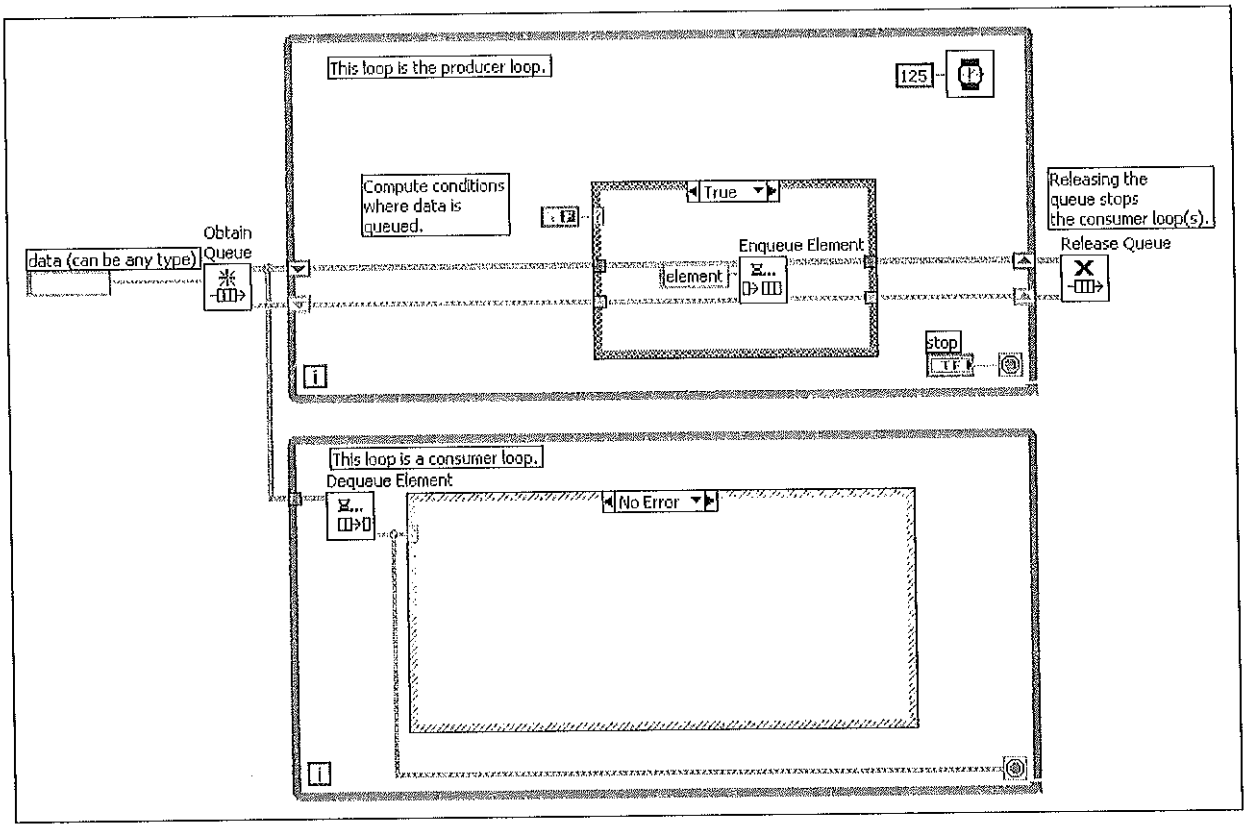


Figure 2-31. Producer/Consumer Design Pattern Using Queues

The queue is created before the loops begin using the Obtain Queue function. The producer loop uses the Enqueue Element function to add data to the queue. The consumer loop removes data from the queue using the Dequeue Element function. The consumer loop does not execute until data is available in the queue. After the VI has finished using the queues, the Release Queue function releases the queues. When the queue releases, the Dequeue Element function generates an error, effectively stopping the consumer loop. This eliminates the need to use a variable to stop the loops.

The following benefits result from using queues in the producer/consumer design pattern:

- Both loops are synchronized to the producer loop. The consumer loop only executes when data is available in the queue.
- You can use queues to create globally available data that is queued, removing the possibility of losing the data in the queue when new data is added to the queue.
- Using queues creates efficient code. You need not use polling to determine when data is available from the producer loop.

Queues are also useful for holding state requests in a state machine. In the implementation of a state machine that you have learned, if two states are requested simultaneously, you might lose one of the state requests. A queue stores the second state request and executes it when the first has finished.

Case Study: Course Project

The course project acquires temperature and wind speed data, and analyzes it to determine if the situation requires a warning. If the temperature is too high or too low, it alerts the user to a danger of heatstroke or freeze. It also monitors the wind speed to generate a high wind warning when appropriate.

The block diagram consists of two parallel loops, which are synchronized using queues. One loop acquires data for temperature and wind speed and the other loop analyzes the data. The loops in the block diagram use the producer/consumer design pattern and pass the data through the queue. Queues help process every reading acquired from the DAQ Assistant.

Code for acquiring temperature and wind speed is placed in the producer loop. Code containing the state machine for analysis of temperature-weather conditions is within the no error case of the consumer loop. The code using a queue is more readable and efficient than the code using only state machine architecture. The Obtain Queue function creates the queue reference. The producer loop uses the Enqueue Element function to add data obtained from the DAQ Assistant to the queue. The consumer loop uses the Dequeue Element function to get the data from the queue and provide it to the state machine for analysis. The Release Queue function marks the end

of queue by destroying it. The use of queues also eliminates the need for a shared variable to stop the loops because the Dequeue Element function stops the consumer loop when the queue is released.

Figure 2-32 shows the block diagram consisting of a producer and a consumer loop. Data transfer and synchronization between the loops is achieved by the queue functions.

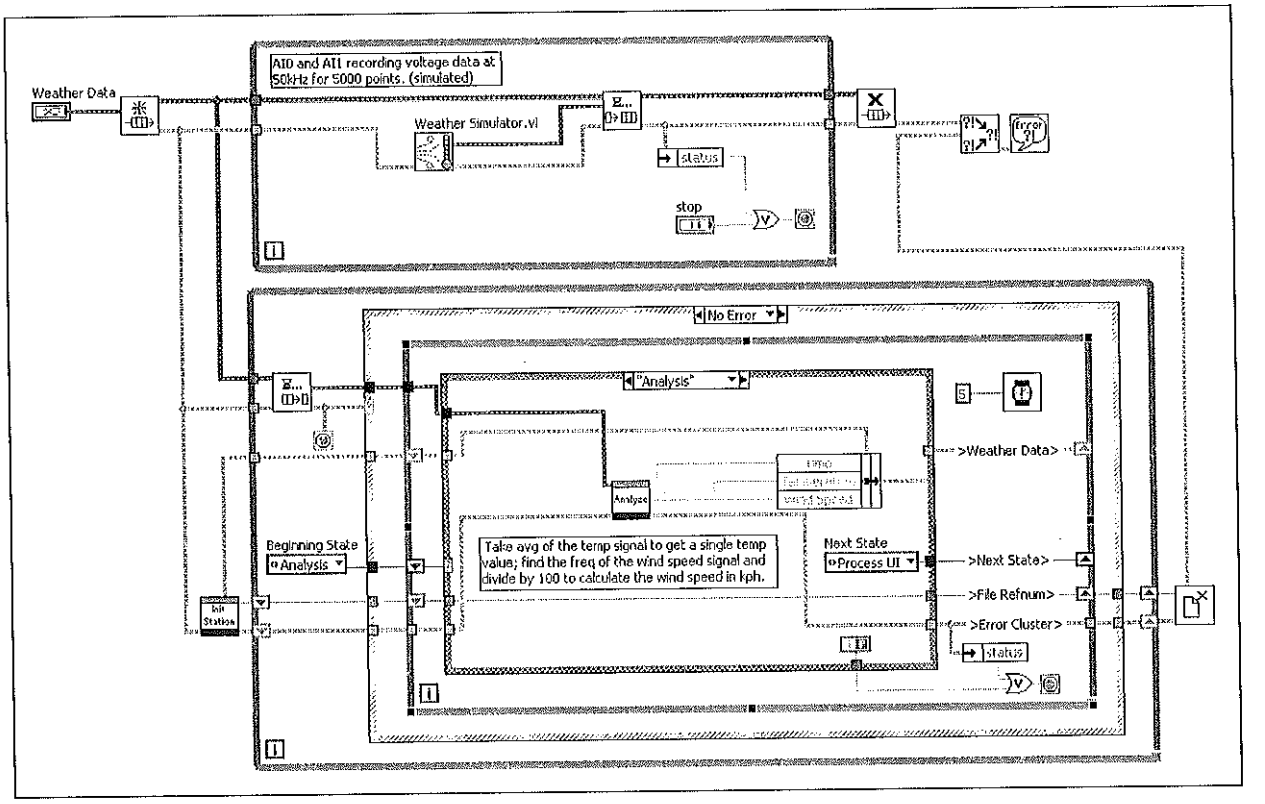


Figure 2-32. Data Transfer and Synchronization of Parallel Loops Using Queues

Exercise 2-4 Queues vs Local Variables VI

Goal

In this exercise, examine a built-in producer/consumer design pattern VI that uses queues to avoid race conditions and synchronize the data transfer between two independent parallel loops.

Description

Complete the following steps to run and examine a built-in producer/consumer design pattern VI that transfers data generated by the producer loop to each of the consumer loops using local variables and queues.

1. Open `Queues vs Local Variables.vi` in the `<Exercises>\LabVIEW Basics II\Queues versus Local Variables` directory. The front panel of this VI is shown in Figure 2-33.

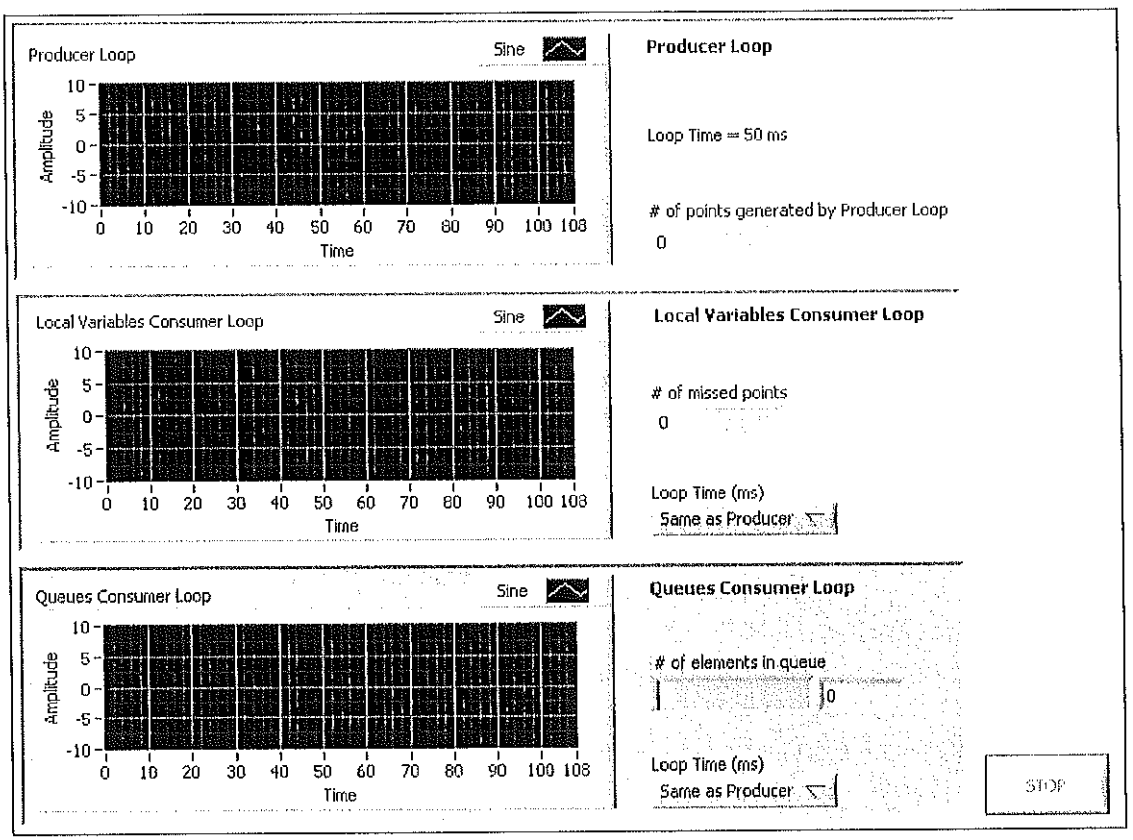


Figure 2-33. Front Panel of the Queues vs Local Variables VI

2. Run the VI. The Producer Loop generates data and transfers it to each consumer loop using a local variable and a queue.

3. Display and examine the block diagram for this VI.

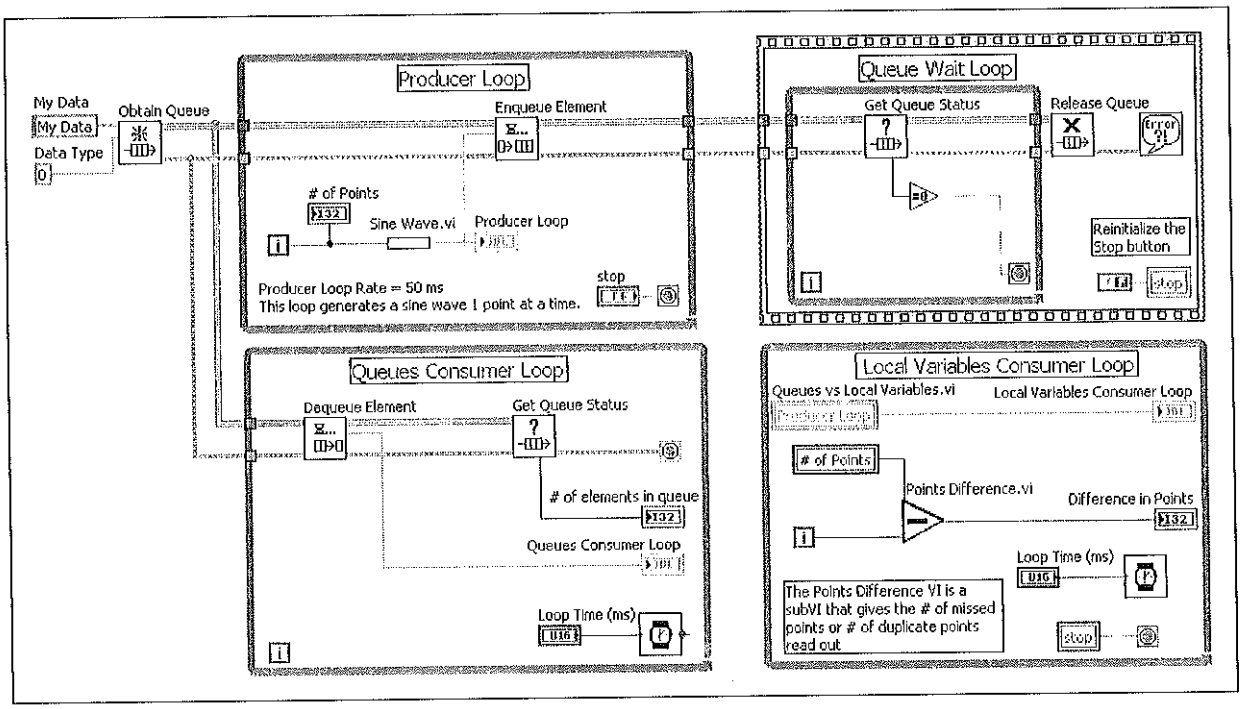


Figure 2-34. Block Diagram of the Queues vs Local Variables VI

Creating a Queue



The Obtain Queue function, placed to the left of the Producer Loop, creates the queue.

The My Data string constant, wired to the **name (unnamed)** input of the Obtain Queue function, assigns a name to the queue you want to obtain or create.

The Data Type numeric constant, wired to the **element data type** input of the Obtain Queue function, specifies the type of data that you want the queue to contain.

Queuing Data Generated by the Producer Loop Generated



The Enqueue Element function inside the Producer Loop adds each data element generated by the Sine Wave subVI to the back of the queue.

Dequeuing Data from the Producer Loop inside the Queues Consumer Loop



The Dequeue Element function inside the Queues Consumer Loop removes an element from the front of the queue and outputs the data element to the Queues Consumer Loop waveform graph.



The Get Queue Status function inside the Queues Consumer Loop indicates how many elements remain in the queue. In order to process these data elements, you must execute the Queues Consumer Loop faster than the Producer Loop, or continue to process after the Producer Loop has stopped.

Waiting for the Queue to Empty



The While Loop inside the Flat Sequence structure waits for the queue to empty before stopping the VI. Refer to this While Loop as the Queue Wait Loop.



The Get Queue Status function inside the Queue Wait Loop returns information about the current state of the queue, such as the number of data elements currently in the queue.



The Equal To 0? function wired to the stop condition of the Queue Wait Loop checks if the queue is empty.



The Release Queue function to the right of the Queue Wait Loop releases and clears reference to the queue.



The Simple Error Handler to the right of the Release Queue function reports any error at the end of execution.

Local Variables Consumer Loop

The Producer Loop generates sine wave data and writes it to a local variable while the Local Variables Consumer Loop periodically reads out the sine wave data from the same local variable. The Points Difference VI inside the Local Variables Consumer Loop outputs the number of missed points or number of duplicate points read out.

Switch to the front panel of this VI.

1. Select the loop time speed of the Local Variables Consumer Loop and observe the Local Variables Consumer Loop waveform graph and the results generated on the **# of missed points** indicator.

   ❑ Ensure that the Loop Time (ms) selected is Same as Producer and observe the waveform graphs for both the Producer Loop and the Local Variables Consumer Loop. A race condition may occur resulting in missed points or duplicated data.

- ❑ Select **Maximum Speed** from the pull-down menu of the **Loop Time (ms)** control and observe the waveform graph of the Local Variables Consumer Loop. A race condition occurs because data is consumed faster than it is produced, allowing the local variable to read the same value multiple times.

- ❑ Select **1/2 as Producer** from the pull-down menu of the **Loop Time (ms)** control and observe the waveform graph of the Local Variables Consumer Loop. A race condition occurs because data is produced faster than it is consumed. The data changes before the local variable has a chance to read it.

- ❑ Select the remaining options available from the pull-down menu of the **Loop Time (ms)** control and observe the data retrieval.

- ❑ Stop the VI.

Data transfer between two non-synchronized parallel loops using local variables causes a race condition. This occurs when the Producer Loop is writing a value to a local variable while the Local Variables Consumer Loop is periodically reading out the value from the same local variable. Because the parallel loops are not synchronized, the value can be written before it has actually been read or vice-versa resulting in data starvation or data overflow.

2. Run the VI. Select the loop time speed of the Queues Consumer Loop and observe the Queues Consumer Loop waveform graph and the results generated on the **# of elements in queue** indicator.

   - ❑ Ensure that the Loop Time (ms) selected is **Same as Producer** and observe the value of the **# of elements in queue** indicator. The value should remain zero. Hence with queues, you will not lose data when the producer and consumer loops are executing at the same rate.

   - ❑ Select **Maximum Speed** from the pull-down menu of the Loop Time (ms) control and observe the value of the # of elements in queue. The value should remain zero. Hence with queues, you will not lose data if the consumer loop is executing much faster than the producer loop.

   - ❑ Select **1/2 as Producer** from the pull-down menu of the Loop Time (ms) control and observe the value of the # of elements in queue. The data points will accumulate in the queue. You will need to process the accumulated elements in the queue before reaching the maximum size of the queue to avoid data loss.

- ❑ Select the remaining options available from the pull-down menu of the **Loop Time (ms)** control and observe the synchronization of data transfer between the producer loop and the consumer loop using queues.
- ❑ Stop the VI.

When the Producer Loop and Queues Consumer Loop run at the same speed, the number of elements in the queue remain unchanged. When the Queues Consumer Loop runs slower, the queue quickly fills up and the Producer Loop must wait for the Queue Consumer Loop to remove the elements. When the Queue Consumer Loop runs faster, the queue is quickly emptied and the consumer loop must wait for the Producer loop to insert elements. Hence queues synchronize the data transfer between the two independent parallel loops and thus avoid loss or duplication of data.

3. Close the VI. Do not save changes.

End of Exercise 2-4

Self-Review: Quiz

1. Use variables frequently in your VIs.
   a. True
   b. False
2. Which of the following cannot transfer data?
   a. Semaphores
   b. Functional global variables
   c. Notifiers
   d. Queues
3. Which of the following must be used within a project?
   a. Local variable
   b. Global variable
   c. Functional global variable
   d. Single-process shared variable
4. Which of the following cannot be used to pass data between multiple VIs?
   a. Local variable
   b. Global variable
   c. Functional global variable
   d. Single-process shared variable

Self-Review: Quiz Answers

1. Use variables frequently in your VI.
   a. True
   b. **False**
2. Which of the following cannot transfer data?
   a. **Semaphores**
   b. Functional global variables
   c. Notifiers
   d. Queues
3. Which of the following must be used within a project?
   a. Local variable
   b. Global variable
   c. Functional global variable
   d. **Single-process shared variable**
4. Which of the following cannot be used to pass data between multiple VIs?
   a. **Local variable**
   b. Global variable
   c. Functional global variable
   d. Single-process shared variable

Notes

3

Event Programming

Event-based design patterns allow you to create more efficient and flexible applications. Event-based design patterns use the Event structure to respond directly to the user or other events. This lesson describes event-driven programming using the Event structure and design patterns that use the Event structure.

Topics

A. Event-Driven Programming

B. Event-Based Design Patterns

A. Event-Driven Programming

LabVIEW is a dataflow programming environment where the flow of data determines the execution order of block diagram elements. Event-driven programming features extend the LabVIEW dataflow environment to allow the user's direct interaction with the front panel and other asynchronous activity to further influence block diagram execution.

Note Event-driven programming features are available only in the LabVIEW Full and Professional Development Systems. You can run a VI built with these features in the LabVIEW Base Package, but you cannot reconfigure the event-handling components.

What Are Events?

An event is an asynchronous notification that something has occurred. Events can originate from the user interface, external I/O, or other parts of the program. User interface events include mouse clicks, key presses, and so on. External I/O events include hardware timers or triggers that signal when data acquisition completes or when an error condition occurs. Other types of events can be generated programmatically and used to communicate with different parts of the program. LabVIEW supports user interface and programmatically generated events but does not support external I/O events.

In an event-driven program, events that occur in the system directly influence the execution flow. In contrast, a procedural program executes in a predetermined, sequential order. Event-driven programs usually include a loop that waits for an event to occur, executes code to respond to the event, and reiterates to wait for the next event. How the program responds to each event depends on the code written for that specific event. The order in which an event-driven program executes depends on which events occur and on the order in which they occur. Some sections of the program might execute frequently because the events they handle occur frequently, and other sections of the program might not execute at all because the events never occur.

Why Use Events?

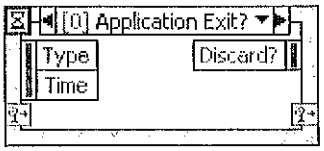
Use user interface events in LabVIEW to synchronize user actions on the front panel with block diagram execution. Events allow you to execute a specific event-handling case each time a user performs a specific action. Without events, the block diagram must poll the state of front panel objects in a loop, checking to see if any change has occurred. Polling the front panel requires a significant amount of CPU time and can fail to detect changes if they occur too quickly. By using events to respond to specific user actions, you eliminate the need to poll the front panel to determine which actions the user performed. Instead, LabVIEW actively notifies the block diagram each time an interaction you specified occurs. Using events reduces the CPU

requirements of the program, simplifies the block diagram code, and guarantees that the block diagram can respond to all interactions the user makes.

Use programmatically generated events to communicate among different parts of the program that have no dataflow dependency. Programmatically generated events have many of the same advantages as user interface events and can share the same event handling code, making it easy to implement advanced architectures, such as queued state machines using events.

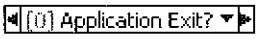
Event Structure Components

Use the Event structure, shown as follows, to handle events in a VI.



The Event structure works like a Case structure with a built-in Wait on Notification function. The Event structure can have multiple cases, each of which is a separate event-handling routine. You can configure each case to handle one or more events, but only one of these events can occur at a time. When the Event structure executes, it waits until one of the configured events occurs, then executes the corresponding case for that event. The Event structure completes execution after handling exactly one event. It does not implicitly loop to handle multiple events. Like a Wait on Notification function, the Event structure can time out while waiting for notification of an event. When this occurs, a specific Timeout case executes.

The event selector label at the top of the Event structure, shown as follows, indicates which events cause the currently displayed case to execute.



View other event cases by clicking the down arrow next to the case name and selecting another case from the shortcut menu.

The Timeout terminal at the top left corner of the Event structure, shown as follows, specifies the number of milliseconds to wait for an event before timing out.



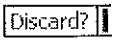
The default is –1, which specifies to wait indefinitely for an event to occur. If you wire a value to the Timeout terminal, you must provide a Timeout case.

The Event Data Node, shown as follows, behaves similarly to the Unbundle By Name function.



This node is attached to the inside left border of each event case. The node identifies the data LabVIEW provides when an event occurs. You can resize this node vertically to add more data items, and you can set each data item in the node to access any event data element. The node provides different data elements in each case of the Event structure depending on which event(s) you configure that case to handle. If you configure a single case to handle multiple events, the Event Data Node provides only the event data elements that are common to all the events configured for that case.

The Event Filter Node, shown as follows, is similar to the Event Data Node.



This node is attached to the inside right border of filter event cases. The node identifies the subset of data available in the Event Data Node that the event case can modify. The node displays different data depending on which event(s) you configure that case to handle. By default, these items are inplace to the corresponding data items in the Event Data Node. If you do not wire a value to a data item of an Event Filter Node, that data item remains unchanged.

Refer to the *Notify and Filter Events* section of this lesson for more information about filter events.

The dynamic event terminals, shown as follows, are available by right-clicking the Event structure and selecting **Show Dynamic Event Terminals** from the shortcut menu.



These terminals are used only for dynamic event registration.

Refer to the *Dynamic Event Registration* and the *Modifying Registration Dynamically* topics of the *LabVIEW Help* for more information about using these terminals.

Note Like a Case structure, the Event structure supports tunnels. However, by default you do not have to wire Event structure output tunnels in every case. All unwired tunnels use the default value for the tunnel data type. Right-click a tunnel and deselect **Use Default If Unwired** from the shortcut menu to revert to the default Case structure behavior where tunnels must be wired in all cases.

Refer to the *LabVIEW Help* for information about the default values for data types.

Notify and Filter Events

There are two types of user interface events—notify and filter.

Notify events are an indication that a user action has already occurred, such as when the user has changed the value of a control. Use notify events to respond to an event after it has occurred and LabVIEW has processed it. You can configure any number of Event structures to respond to the same notify event on a specific object. When the event occurs, LabVIEW sends a copy of the event to each Event structure configured to handle the event in parallel.

Filter events inform you that the user has performed an action before LabVIEW processes it, which allows you to customize how the program responds to interactions with the user interface. Use filter events to participate in the handling of the event, possibly overriding the default behavior for the event. In an Event structure case for a filter event, you can validate or change the event data before LabVIEW finishes processing it, or you can discard the event entirely to prevent the change from affecting the VI. For example, you can configure an Event structure to discard the Panel Close? event, preventing the user from interactively closing the front panel of the VI. Filter events have names that end with a question mark, such as Panel Close?, to help you distinguish them from notify events. Most filter events have an associated notify event of the same name, but without the question mark, which LabVIEW generates after the filter event if no event case discarded the event.

As with notify events, you can configure any number of Event structures to respond to the same filter event on a specific object. However, LabVIEW sends filter events sequentially to each Event structure configured for the event. The order in which LabVIEW sends the event to each Event structure depends on the order in which the events were registered. Each Event structure must complete its event case for the event before LabVIEW can notify the next Event structure. If an Event structure case changes any of the event data, LabVIEW passes the changed data to subsequent Event structures in the chain. If an Event structure in the chain discards the event, LabVIEW does not pass the event to any Event structures remaining in the chain. LabVIEW completes processing the user action which triggered the event only after all configured Event structures handle the event without discarding it.



Note National Instruments recommends you use filter events only when you want to take part in the handling of the user action, either by discarding the event or by modifying

the event data. If you only want to know that the user performed a particular action, use notify events.

Event structure cases that handle filter events have an Event Filter Node. You can change the event data by wiring new values to these terminals. If you do not wire a data item, that item remains unchanged. You can completely discard an event by wiring a TRUE value to the **Discard?** terminal.

Note A single case in the Event structure cannot handle both notify and filter events. A case can handle multiple notify events but can handle multiple filter events only if the event data items are identical for all events.

Refer to the *Using Events in LabVIEW* section of this lesson for more information about event registration.

Tip In the **Edit Events** dialog box, notify events are signified by a green arrow, and filter events are signified by a red arrow.

Using Events in LabVIEW

LabVIEW can generate many different events. To avoid generating unwanted events, use event registration to specify which events you want LabVIEW to notify you about. LabVIEW supports two models for event registration—static and dynamic.

Static registration allows you to specify which events on the front panel of a VI you want to handle in each Event structure case on the block diagram of that VI. LabVIEW registers these events automatically when the VI runs, so the Event structure begins waiting for events as soon as the VI begins running. Each event is associated with a control on the front panel of the VI, the front panel window of the VI as a whole, or the LabVIEW application. You cannot statically configure an Event structure to handle events for the front panel of a different VI. Configuration is static because you cannot change at run time which events the Event structure handles.

Dynamic event registration avoids the limitations of static registration by integrating event registration with the VI Server, which allows you to use Application, VI, and control references to specify at run time the objects for which you want to generate events. Dynamic registration provides more flexibility in controlling what events LabVIEW generates and when it generates them. However, dynamic registration is more complex than static registration because it requires using VI Server references with block diagram functions to explicitly register and unregister for events rather than handling registration automatically using the information you configured in the Event structure.

Note In general, LabVIEW generates user interface events only as a result of direct user interaction with the active front panel. LabVIEW does not generate events, such as Value Change, when you use shared variables, global variables, local variables, and so on. However, you can use the Value (Signaling) property to generate a Value Change event programmatically. In many cases, you can use programmatically generated events instead of queues and notifiers.

The event data provided by a LabVIEW event always includes a time stamp, an enumeration that indicates which event occurred, and a VI Server reference to the object that triggered the event. The time stamp is a millisecond counter you can use to compute the time elapsed between two events or to determine the order of occurrence. The reference to the object that generated the event is strictly typed to the VI Server class of that object. Events are grouped into classes according to what type of object generates the event, such as Application, VI, or Control. If a single case handles multiple events for objects of different VI Server classes, the reference type is the common parent class of all objects. For example, if you configure a single case in the Event structure to handle events for a numeric control and a color ramp control, the type of the control reference of the event source is Numeric because the numeric and color ramp controls are in the Numeric class. If you register for the same event on both the VI and Control class, LabVIEW generates the VI event first.

Note Clusters are the only container objects for which you can generate events. LabVIEW generates Control events for clusters, before it generates events for the objects they contain, except in the case of the Value Change event. The Value Change event generates the event on an element in the cluster, then on the cluster itself. If the Event structure case for a VI event or for a Control event on a container object discards the event, LabVIEW does not generate further events.

Each Event structure and Register For Events function on the block diagram owns a queue that LabVIEW uses to store events. When an event occurs, LabVIEW places a copy of the event into each queue registered for that event. An Event structure handles all events in its queue and the events in the queues of any Register For Events functions that you wired to the dynamic event terminals of the Event structure. LabVIEW uses these queues to ensure that events are reliably delivered to each registered Event structure in the order the events occur.

By default, when an event enters a queue, LabVIEW locks the front panel that contains the object that generated that event. LabVIEW keeps the front panel locked until all Event structures finish handling the event. While the front panel is locked, LabVIEW does not process front panel activity but places those interactions in a buffer and handles them when the front panel is unlocked.

For example, a user might anticipate that an event case launches an application that requires text entry. Because the user already knows text entry is needed, he might begin typing before the application appears on the front panel. If the **Lock front panel until the event case for this event completes** option is enabled, after the application launches and appears on the front panel, it processes the key presses in the order in which they occurred. If the **Lock front panel until the event case for this event completes** option is disabled, the key presses might be processed elsewhere on the front panel, because LabVIEW does not queue their execution to depend on the completion of the event case.

Front panel locking does not affect certain actions, such as moving the window, interacting with the scroll bars, and clicking the **Abort** button.

LabVIEW can generate events even when no Event structure is waiting to handle them. Because the Event structure handles only one event each time it executes, place the Event structure in a While Loop to ensure that an Event structure can handle all events that occur.



Caution If no Event structure executes to handle an event and front panel locking is enabled, the user interface of the VI becomes unresponsive. If this occurs, click the **Abort** button to stop the VI. You can disable front panel locking by right-clicking the Event structure and removing the checkmark from the **Lock front panel until the event case for this event completes** checkbox in the **Edit Events** dialog box. You cannot turn off front panel locking for filter events.

Static Event Registration

Static event registration is available only for user interface events. Use the **Edit Events** dialog box to configure an Event structure to handle a statically registered event. Select the event source, which can be the application, the VI, or an individual control. Select a specific event the event source can generate, such as Panel Resize, Value Change, and so on. Edit the case to handle the event data according to the application requirements.

LabVIEW statically registers events automatically and transparently when you run a VI that contains an Event structure. LabVIEW generates events for a VI only while that VI is running or when another running VI calls the VI as a subVI.

When you run a VI, LabVIEW sets that top-level VI and the hierarchy of subVIs the VI calls on its block diagram to an execution state called reserved. You cannot edit a VI or click the **Run** button while the VI is in the reserved state because the VI can be called as a subVI at any time while its parent VI runs. When LabVIEW sets a VI to the reserved state, it automatically registers the events you statically configured in all Event structures on the block diagram of that VI. When the top-level VI finishes

running, LabVIEW sets it and its subVI hierarchy to the idle execution state and automatically unregisters the events.

Refer to the `labview\examples\general\uievents.llb` for examples of using static event registration.

Configuring Events

Before you configure events for the Event structure to handle, refer to the *Caveats and Recommendations when Using Events in LabVIEW* section of the *LabVIEW Help*.

Complete the following steps to configure an Event structure case to handle an event.

1. (Optional) If you want to configure the Event structure to handle a user event, a Boolean control, or a user interface event that is generated based on a reference to an application, VI, or control, you must first dynamically register that event. Refer to the *Dynamically Registered Events* topic of the *LabVIEW Help* for more information about using dynamic events.
2. Right-click the border of the Event structure and select **Edit Events Handled by This Case** from the shortcut menu to display the **Edit Events** dialog box to edit the current case. You also can select **Add Event Case** from the shortcut menu to create a new case.
3. Specify an event source in the **Event Sources** section.
4. Select the event you want to configure for the event source, such as **Key Down**, **Timeout**, or **Value Change** from the **Events** list. When you select a dynamic event source from the **Event Sources** list, the **Events** list displays that event. This is the same event you selected when you registered the event. If you have registered for events dynamically and wired **event reg refnum out** to the dynamic event terminal, the sources appear in the **Dynamic** section.
5. If you want to add additional events for the current case to handle, click the + button and repeat steps 3 and 4 to specify each additional event. The **Event Specifiers** section at the top of the dialog box lists all the events for the case to handle. When you click an item in this list, the **Event Sources** section updates to highlight the event source you selected. You can repeat steps 3 and 4 to redefine each event or click the **X** button to remove the selected event.
6. Click the **OK** button to save the configuration and close the dialog box. The event cases you configured appear as selection options in the event selector label at the top of the Event structure and the Event Data node displays the data common to all events handled in that case.

7. (Optional) You can use a Timeout event to configure an Event structure to wait a specified amount of time for an event to occur. Wire a value to the Timeout terminal at the top left of the Event structure to specify the number of milliseconds the Event structure should wait for an event to occur before generating a Timeout event. The default value for the Timeout terminal is –1, which specifies to wait indefinitely for an event to occur.
8. Repeat steps 1 through 6 for each event case you want to configure.

Refer to the following VIs for examples of using events:

`labview\examples\general\dynamicevents.llb`

`labview\examples\general\uievents.llb`

Event Example

Figure 3-1 shows an Event structure configured with the Menu Selection (User) event. This VI uses the Event structure to capture menu selections made using the user-defined menu named `sample.rtm`. The ItemTag returns the menu item that was selected and the MenuRef returns the refnum to the menubar. This information is passed to the Get Menu Item Info function. Refer to `examples\general\uievents.llb` for more examples of using events.

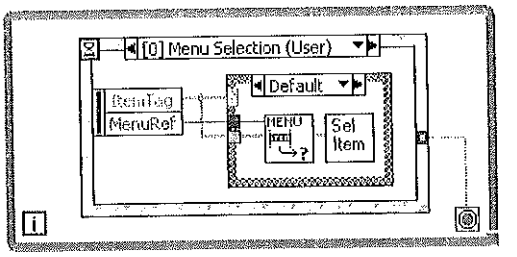


Figure 3-1. Menu Selection (User) Event

Note f you use the Get Menu Selection function with an Event structure configured to handle the same menu item, the Event structure takes precedence, and LabVIEW ignores the Get Menu Selection function. In any given VI, use the Event structure or the Get Menu Selection function to handle menu events, not both.

Exercise 3-1 Concept: Experiment with Event Structures

Goal

Experiment with the operation of the Event structure in a VI.

Description

Use and modify a VI that contains an Event structure.

1. Open the NI Example Finder.
   - ❑ Select **Help»Find Examples**.
2. Open the New Event Handler VI example.
   - ❑ Navigate to **Building User Interfaces»Acquiring User Input» General** and double-click `New Event Handler.vi`.
3. Enable execution highlighting on the block diagram.
4. Run the VI.
5. Observe the operation of the VI when you click the buttons on the front panel.
6. Stop the VI.
7. Modify the VI to respond to a Value Change event with a new control on the front panel.
   - ❑ Switch to the front panel of the VI.
   - ❑ Create a copy of a Boolean button on the front panel.
   - ❑ Change the button text and label of the button to `New Button`.
   - ❑ Right-click the new button and verify that the **Mechanical Action** is set to **Latch When Released**.
   - ❑ Switch to the block diagram of the VI.
   - ❑ Right-click the border of the Event structure and select **Add Event Case** from the shortcut menu to open the **Edit Events** dialog box.

- ❑ Select **New Button** in the **Event Sources** section and select **Value Change** in the **Events** section as shown in Figure 3-2.

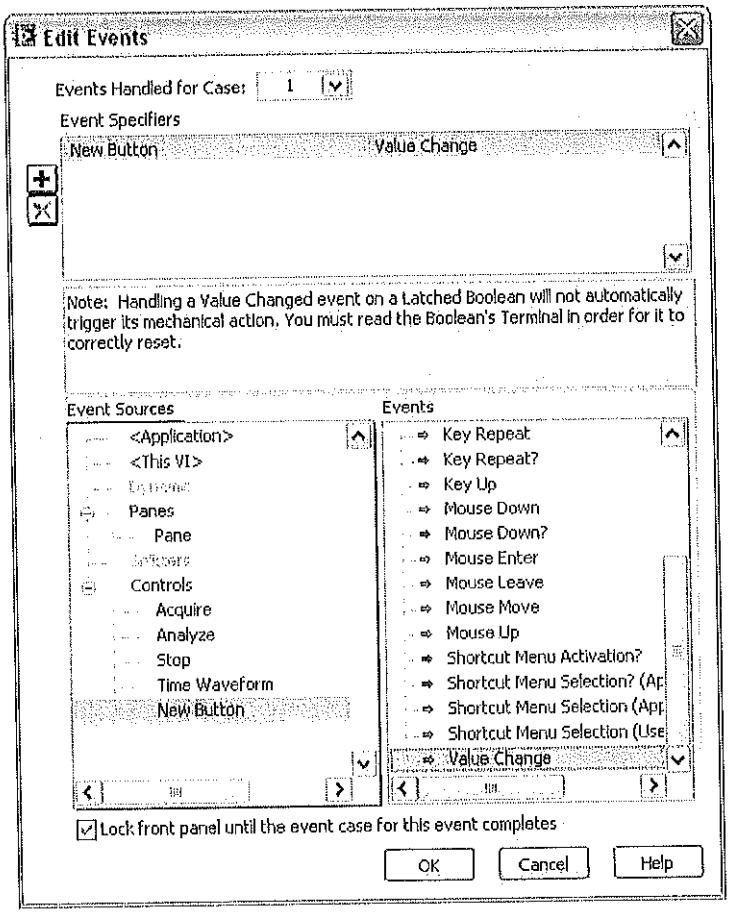


Figure 3-2. New Button Value Change Event

- ❑ Click **OK** to create the new Event structure case.
  - ❑ Place a One Button Dialog function in the New Button event case and wire a string constant to the **message** input. Set the string constant to `New Event Case`.
  - ❑ Wire the Time Waveform data through the case.
  - ❑ Wire a False constant to the Boolean tunnel.
  - ❑ Run the VI and click **New Button**. A dialog box should open and display the **`New Event Case`** message.

8. Modify the Event structure to add a Filter Event to discard the Panel Close event.
   - ❑ Right-click the Event structure and select **Add Event Case** from the shortcut menu to open the **Edit Events** dialog box.
   - ❑ Select **<This VI>** in the **Event Sources** section.
   - ❑ Select **Panel Close?** from the **Events** list and click the **OK** button.
   - ❑ Wire a True constant to the Discard? Event Filter Node in the Panel Close? case.
   - ❑ Wire the Time Waveform data through the case.
   - ❑ Wire a False constant to the Boolean tunnel.
9. Run the VI.
10. Attempt to close the VI by closing the front panel.
11. Click the **STOP** button to stop the VI.
12. Open the NI Example Finder.
13. Open the Old Event Handler VI example.
    - ❑ Navigate to **Building User Interfaces»Acquiring User Input» General** and double-click `Old Event Handler.vi`.
14. Enable execution highlighting on the block diagram.
15. Run the VI.
16. Observe the operation of the Old Event Handler VI and compare and contrast the operation with the New Event Handler VI.
17. Close all VIs without saving changes.

End of Exercise 3-1

B. Event-Based Design Patterns

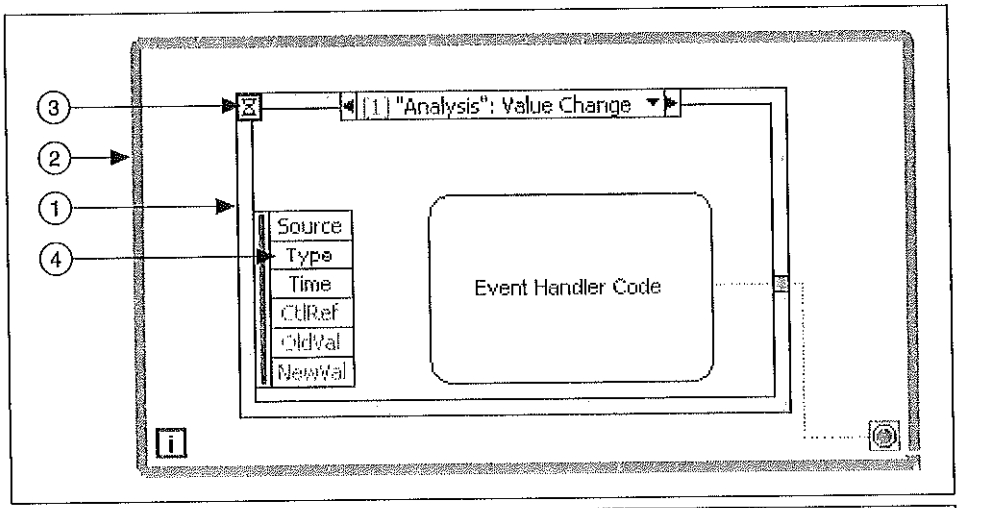
Event-based design patterns provide efficiency gains because they only respond when an event occurs. When LabVIEW executes the Event structure, the VI that contains the Event structure sleeps until a registered events occurs, or generates. When a registered event generates, the Event structure automatically wakes up and executes the appropriate subdiagram to handle the event.

User Interface Event Handler Design Pattern

The user interface event handler design pattern provides a powerful and efficient architecture for handling user interaction with LabVIEW. Use the user interface event handler for detecting when a user changes the value of a control, moves or clicks the mouse, or presses a key.

The standard user interface event handler template consists of an Event structure contained in a While Loop, as shown in Figure 3-3. Configure the Event structure to have one case for each category of event you want to detect. Each event case contains the handling code that executes immediately after an event occurs.

Because the event handler loop wakes up precisely when an event occurs and sleeps in between events, you do not have to poll or read control values repeatedly in order to detect when a user clicks a button. The user interface event handler allows you to minimize processor use without sacrificing interactivity.



1	Event Structure	3	Timeout Terminal
2	While Loop	4	Event Data Node

Figure 3-3. User Interface Event Handler Design Pattern

A common problem when using the user interface event handler is that it computes the While Loop termination before the Event structure executes. This can cause the While Loop to iterate one more time than you expected. To avoid this situation, compute the While Loop termination within all your event handling code.

The event handler code must execute quickly, generally within 200 ms. Anything slower can make it feel as if the user interface is locked up. Also, if the event handler code takes a long time to execute, the Event structure might lock. By default, the front panel locks while an event is handled. You can disable front panel locking for each event case to make the user interface more responsive. However, any new events that are generated while an event is being handled will not be handled immediately. So, the user interface will still seem unresponsive.

Any code that is in an event case cannot be shared with another Event structure. You must utilize good code design when using the Event structure. Modularize code that will be shared between multiple Event structure cases.

The Event structure includes a Timeout event, which allows you to control when the Timeout event executes. For example, if you set a Timeout of 200 ms, the Timeout event case executes every 200 ms in the absence of other events. You can use the Timeout event to perform critical timing in your code.

Producer/Consumer (Events) Design Pattern

One of the most versatile and flexible design patterns combines the producer/consumer and user interface event handler design patterns. A VI built using the producer/consumer (events) pattern responds to the user interface asynchronously, allowing the user interface to continuously respond to the user. The consumer loop of this pattern responds as events occur, similar to the consumer loop of the producer/consumer (data) design pattern.

The producer/consumer (events) design pattern uses the same implementation as the producer/consumer (data) design pattern except the producer loop uses an Event structure to respond to user interface events, as shown in Figure 3-4. The Event structure enables continuous response to user interaction.

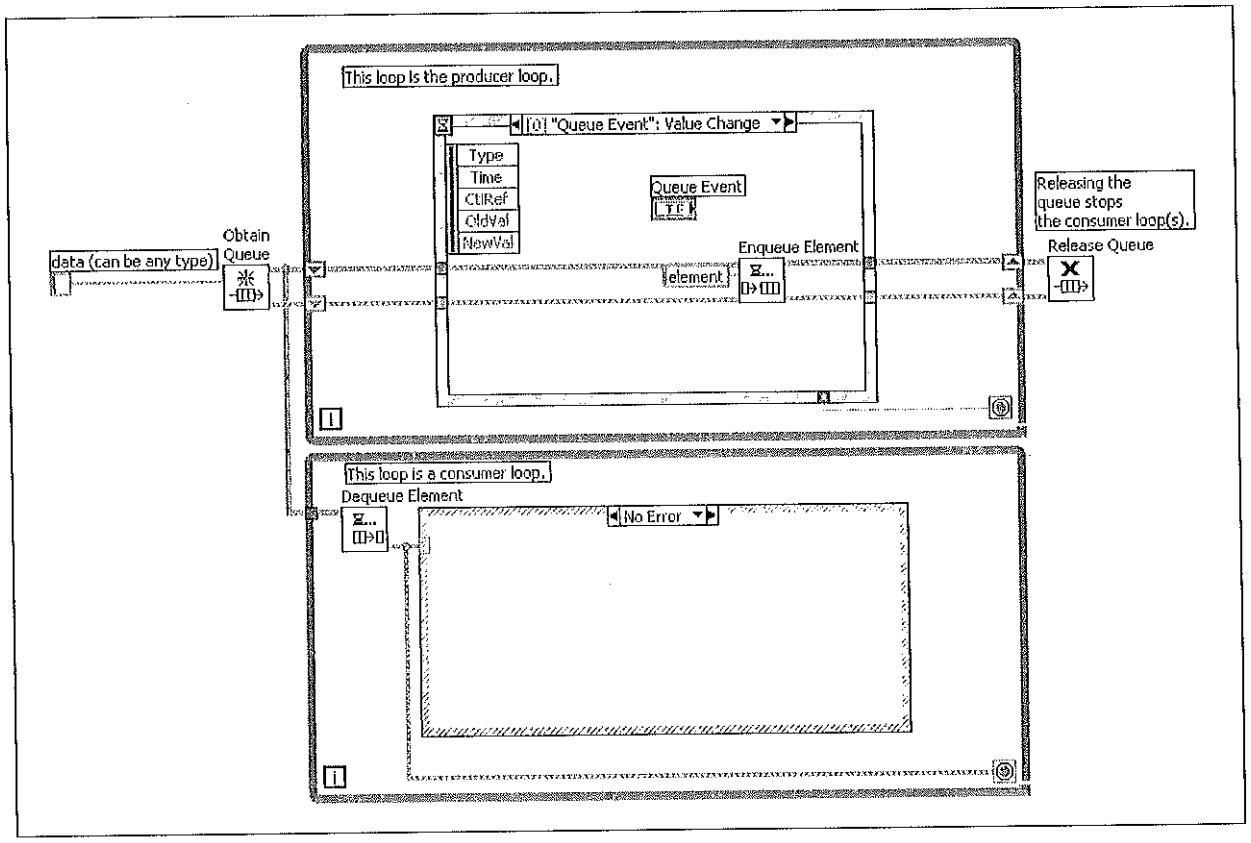


Figure 3-4. Producer/Consumer (Events) Design Pattern

Figure 3-4 shows how you can use Synchronization VIs and functions to add functionality to the design pattern

Queues have the ability to transfer any data type. The data type transferred in Figure 3-4is a string. A string is not the most efficient data type for passing data in design patterns. A more efficient data type for passing data in design patterns is a cluster consisting of an enumerated type control and a Variant.

Exercise 3-2 Concept: Experiment with Event-Based Design Patterns

Goal

Observe the functionality and design of the built-in event-based LabVIEW design patterns.

Description

Explore the event-based design pattern VI templates that ship with LabVIEW and observe how they operate. Use the event-based design pattern VI templates in LabVIEW as the basis for the underlying architecture of VIs you create.

1. Select **File»New** to open the **New** dialog box.
2. In the **Create New** section, expand the **VI»from Template» Frameworks»Design Patterns** tree.
3. Select the **User Interface Event Handler** design pattern from the tree and click the **OK** button to open the template.
4. Open the block diagram, turn execution highlighting on, and run the VI.

   Notice how the design pattern handles the following items:

   - ❑ State transitions
   - ❑ Data flow
   - ❑ VIs used to create design patterns

5. Close the design pattern. Do not save changes.
6. Repeat steps 1 through 5 to observe the operation of the **Producer/Consumer Design Pattern (Events)** design pattern.

End of Exercise 3-2

Self Review: Quiz

1. Event structures are normally used in While Loops.
   a. True
   b. False
2. Which of the following are user interface events?
   a. Clicking a button on the mouse
   b. Pressing a key on the keyboard
   c. While Loop
   d. Value change of a control
3. User interface events allow you to synchronize user actions on the front panel with block diagram execution.
   a. True
   b. False

Self Review: Quiz Answers

1. Event structures are normally used in While Loops.
   a. **True**
   b. False
2. Which of the following are user interface events?
   a. **Clicking a button on the mouse**
   b. **Pressing a key on the keyboard**
   c. While Loop
   d. **Value change of a control**
3. User interface events allow you to synchronize user actions on the front panel with block diagram execution.
   a. **True**
   b. False

Notes

4

Improving an Existing VI

A common problem when you inherit VIs from other developers is that features may have been added without attention to design, thus making it progressively more difficult to add features later in the life of the VI. This is known as software decay. One solution to software decay is to refactor the software. Refactoring is the process of redesigning software to make it more readable and maintainable so that the cost of change does not increase over time. Refactoring changes the internal structure of a VI to make it more readable and maintainable, without changing its observable behavior.

In this section, you will learn methods to refactor inherited code and experiment with typical issues that appear in inherited code.

Topics

A. Refactoring Inherited Code

B. Typical Refactoring Issues

A. Refactoring Inherited Code

Write large and/or long-term software applications with readability in mind because the cost of reading and modifying the software is likely to outweigh the cost of executing the software. It costs more for a developer to read and understand poorly designed code than it does to read code that was created to be readable. In general, more resources are allocated to reading and modifying software than to the initial implementation. Therefore VIs that are easy to read and modify are more valuable than those that are not.

Creating well-designed software facilitates rapid development and decreases possible decay. If a system starts to decay, you can spend large amounts of time tracking down regression failures, which is not productive. Changes also can take longer to implement because it is harder to understand the system if it is poorly designed.

Consider the inherited VI shown in Figure 4-1.

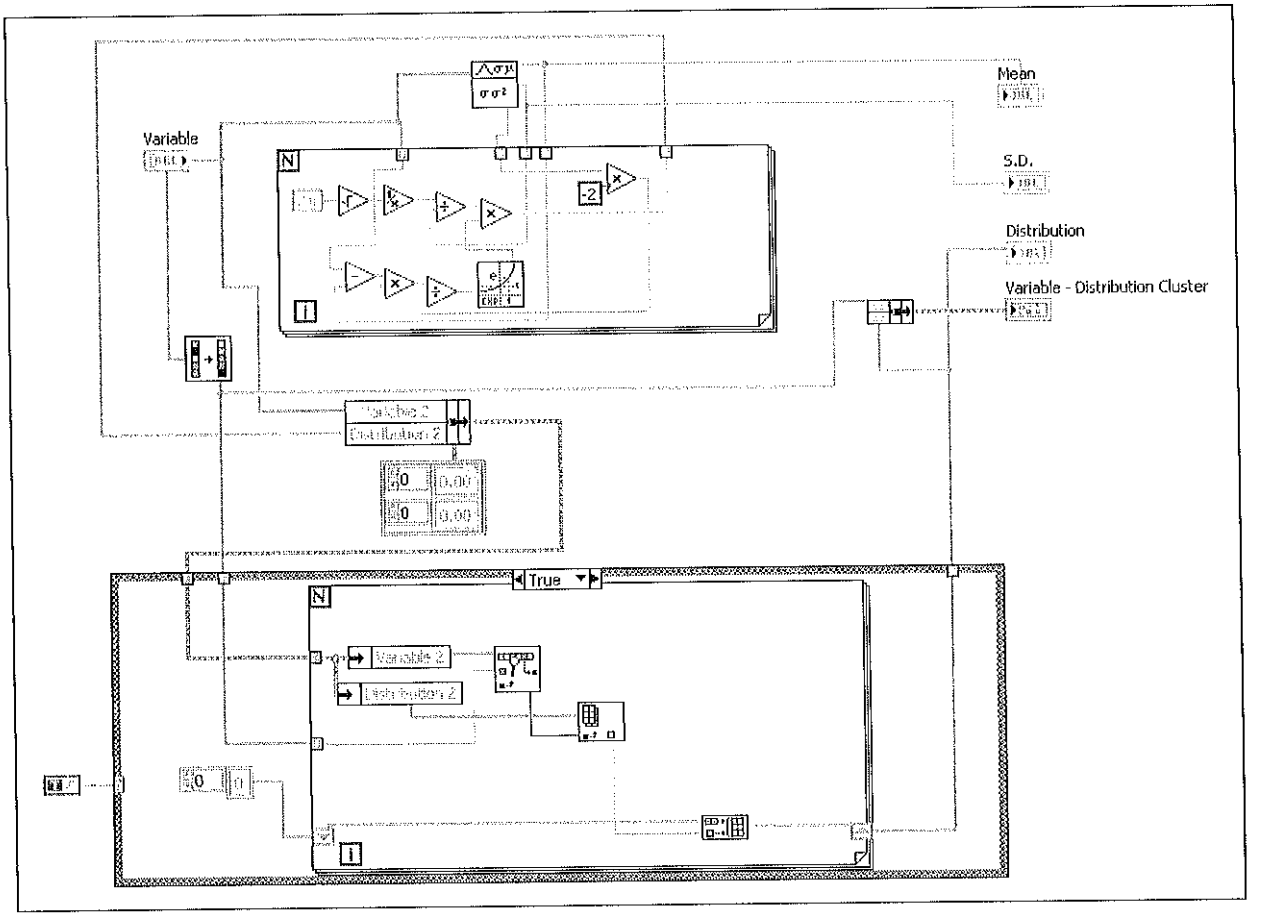


Figure 4-1. Inherited VI

You can refactor the code as shown in Figure 4-2.

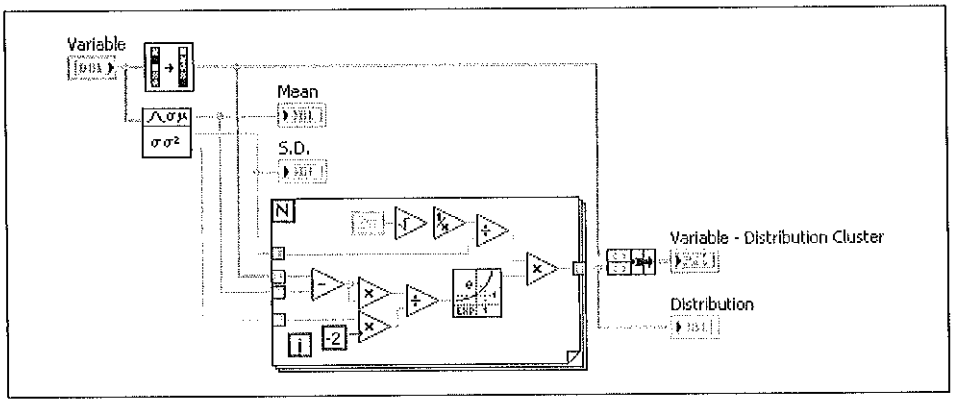


Figure 4-2. Refactored Inherited Code

The refactored code performs the same function as the inherited code, but the refactored code is more readable. The inherited code violates many of the block diagram guidelines you have learned.

When you make a VI easier to understand and maintain, you make it more valuable because it is easier to add features to or debug the VI. The refactoring process does not change observable behavior. Changing the way a VI interacts with clients (users or other VIs) introduces risks that are not present when you limit changes to those visible only to developers. The benefit of keeping the two kinds of changes separate is that you can better manage risks.

Refactoring versus Performance Optimization

Although you can make changes that optimize the performance of a VI, this is not the same as refactoring. Refactoring specifically changes the internal structure of a VI to make it easier to read, understand, and maintain.
A performance optimization is not refactoring because the goal of optimization is not to make the VI easier to understand and modify.
In fact, performance optimization can make VIs more difficult to read and understand, which might be an acceptable trade-off. Sometimes you must sacrifice readability for improved performance, however, readability usually takes priority over speed of performance.

When to Refactor

The right time to refactor is when you are adding a feature to a VI or debugging it. Although you might be tempted to rewrite the VI from scratch, there is value in a VI that works, even if the block diagram is not readable. Good candidates for complete rewrites are VIs that do not work or VIs that satisfy only a small portion of your needs. You also can rewrite simple VIs that you understand well. Consider what works well in an existing VI before you decide to refactor.

B. Typical Refactoring Issues

When you refactor a VI, manage the risk of introducing bugs by making small, incremental changes to the VI and testing the VI after each change. The flowchart shown in Figure 4-3 indicates the process for refactoring a VI.

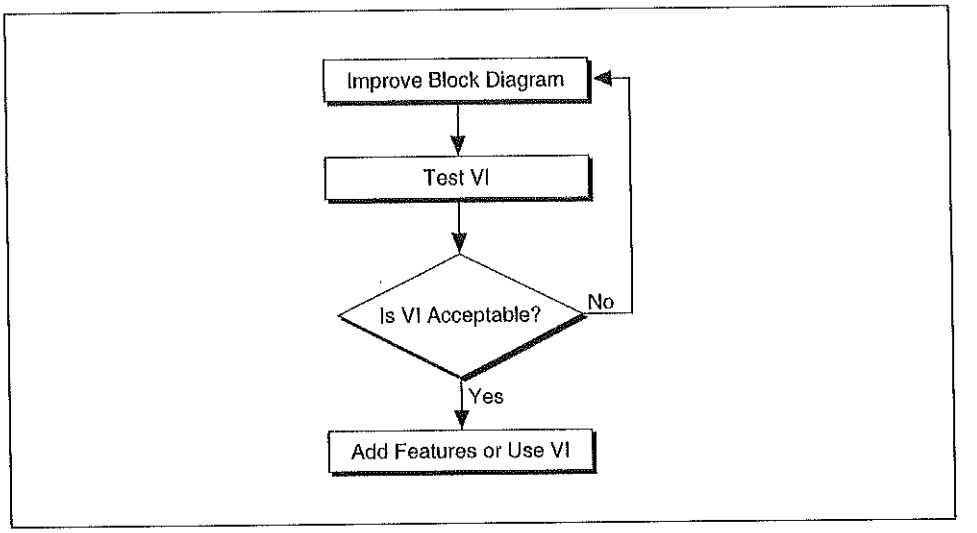


Figure 4-3. Refactoring Flowchart

When you refactor to improve the block diagram, make small cosmetic changes before tackling larger issues. For example, it is easier to find duplicated code if the block diagram is well organized and the terminals are well labeled.

There are several issues that can complicate working with an inherited VI. The following sections describe typical problems and the refactoring solutions you can use to make inherited VIs more readable.

Disorganized or Poorly Designed Block Diagram

Improve the readability of a disorganized VI by relocating objects within the block diagram. You also can create subVIs for sections of the VI that are disorganized. Place comments on areas of a VI that are disorganized to improve the readability of the VI.

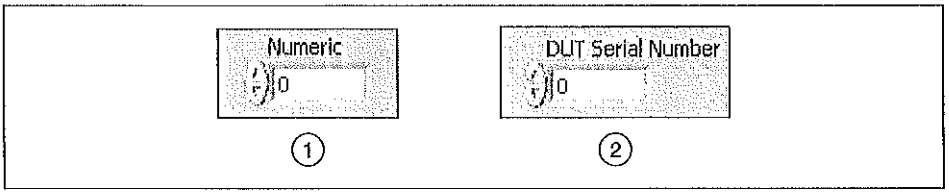
Overly Large Block Diagram

A VI that has a block diagram that is larger than the screen size is difficult to read. You should refactor the VI to make it smaller. The act of scrolling complicates reading a block diagram and understanding the code. Improve a large block diagram by moving objects around. Another technique to reduce the screen space a block diagram occupies is to create subVIs for

sections of code within the block diagram. If you cannot reduce the block diagram to fit on the screen, limit the scrolling to one direction.

Poorly Named Objects and Poorly Designed Icons

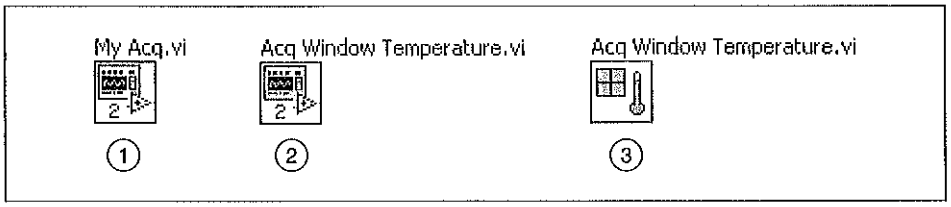
Inherited VIs often contain controls and indicators that do not have meaningful names. For example, the name of Control 1, shown in Figure 4-4, does not indicate its purpose. Control 2 is the same control, renamed to make the block diagram more readable and understandable.



1 Poorly Named Control
2 Meaningfully Named Control

Figure 4-4. Naming Controls

VI names and icons also are important for improving the readability of a VI. For example, the name `My Acqu.vi`, shown on the left in Figure 4-5, does not provide any information about the purpose of the VI. You can give the VI a more meaningful name by saving a copy of the VI with a new name and replacing all instances of the VI with the renamed VI. A simpler method is to open all callers of the VI you want to rename, then save the VI with a new name. When you use this method, LabVIEW automatically relinks all open callers of the VI to the new name. `Acq Window Temperature.vi` reflects a more meaningful name for the VI.



1 Poorly Named VI
2 Meaningfully Named VI
3 Meaningful VI Name and VI Icon

Figure 4-5. Poorly Named SubVI

The VI icon also should clarify the purpose of the VI. The default icons used for VI 1 and VI 2 in Figure 4-5 do not represent the purpose of the VI. You can improve the readability of the VI by providing a meaningful icon, as shown for VI 3.

By renaming controls and VIs and creating meaningful VI icons, you can improve the readability of an inherited VI.

Unnecessary Logic

When you read the block diagram in Figure 4-6, notice that it contains unnecessary logic. If a portion of the block diagram does not execute, delete it. Understanding code that executes is difficult, but trying to understand code that never executes is inefficient and complicates the block diagram.

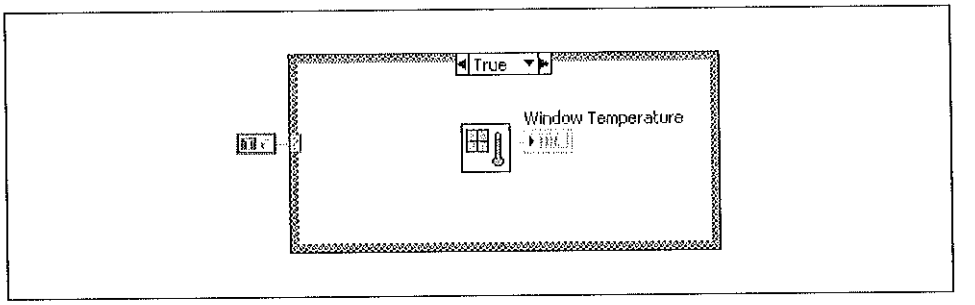


Figure 4-6. Unnecessary Logic

Duplicated Logic

If a VI contains duplicated logic, you always should refactor the VI by creating a subVI for the duplicated logic. This can improve the readability and testability of the VI.

Lack of Dataflow Programming

If there are Sequence structures and local variables on the block diagram, the VI probably does not use data flow to determine the programming flow.

You should replace most Sequence structures with the state machine design pattern. Delete local variables and wire the controls and indicators directly. The most acceptable use of local variables is to pass a control value to an indicator.

Complicated Algorithms

Complicated algorithms can make a VI difficult to read. Complicated algorithms can be more difficult to refactor because there is a higher probability that the changes introduce errors. When you refactor a complicated algorithm, make minor changes and test the code frequently. In some cases you can refactor a complicated algorithm by using built-in LabVIEW functions. For example, the VI in Figure 4-7 checks a user name and password against a database.

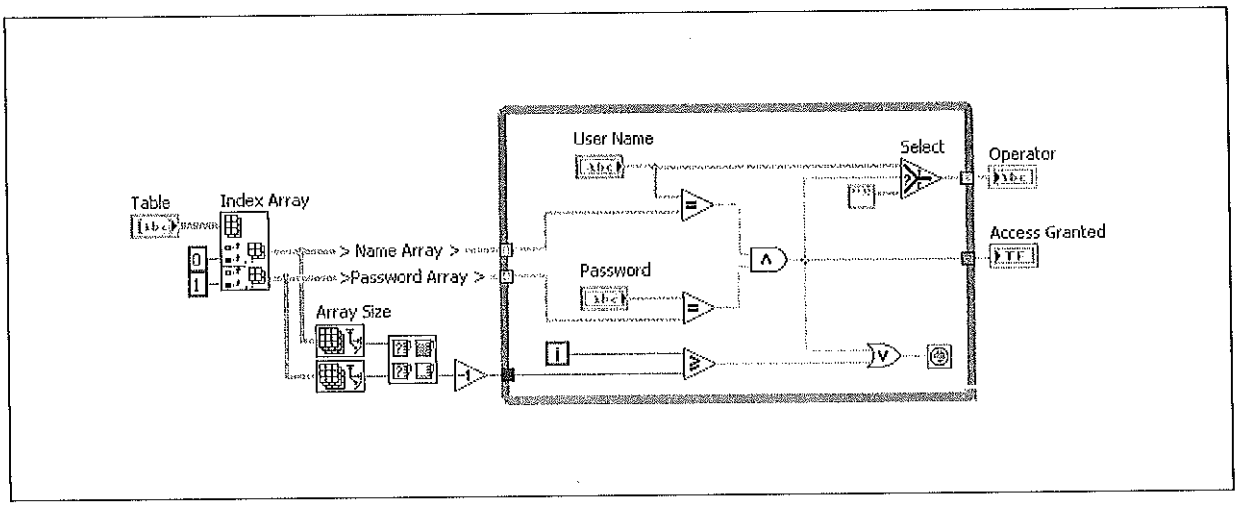


Figure 4-7. Complicated Algorithm VI

You could refactor this VI using the built-in functions for searching strings, as shown in Figure 4-8.

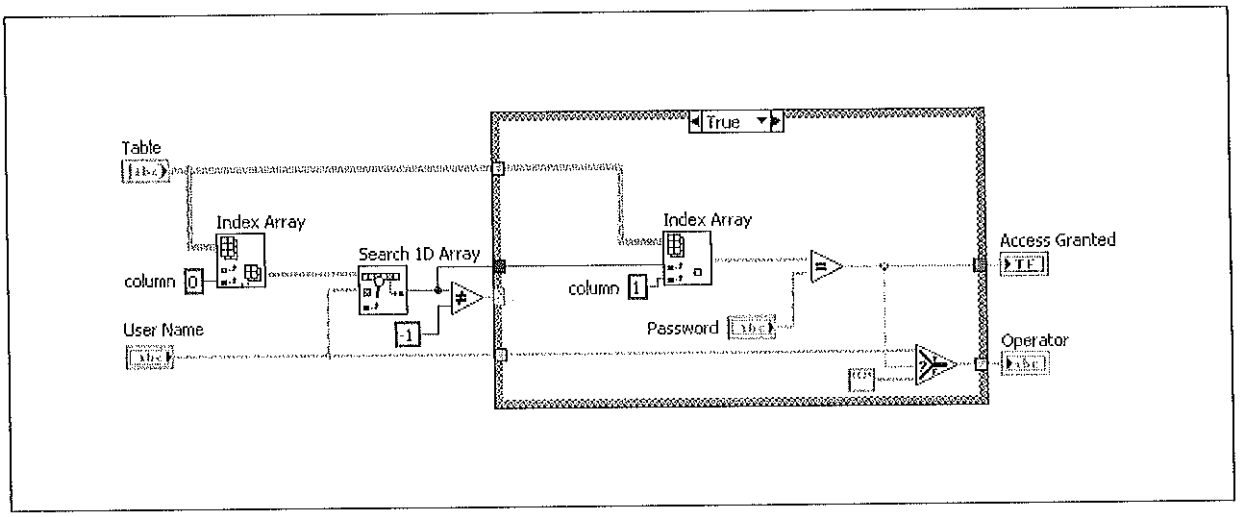


Figure 4-8. Refactored VI

Exercise 4-1 Concept: Typical Issues

Goal

Improve an existing VI that is poorly designed.

Description

You receive a VI that is used as a subVI in a larger project. You must improve the VI for readability and user friendliness.

Evaluate the VI

1. Open the `Determine Warning Bad One.vi` located in the `<Exercises>\LabVIEW Basics II\Determine Warnings` directory. Figure 4-9 shows the block diagram of this VI.

2. Use the following list to evaluate the VI. Place a checkmark for all issues that apply.

   - ❑ Disorganized block diagram
   - ❑ Overly large block diagram
   - ❑ Poorly named objects and poorly designed icons
   - ❑ Unnecessary logic
   - ❑ Duplicated logic
   - ❑ Lack of dataflow programming
   - ❑ Complicated algorithms

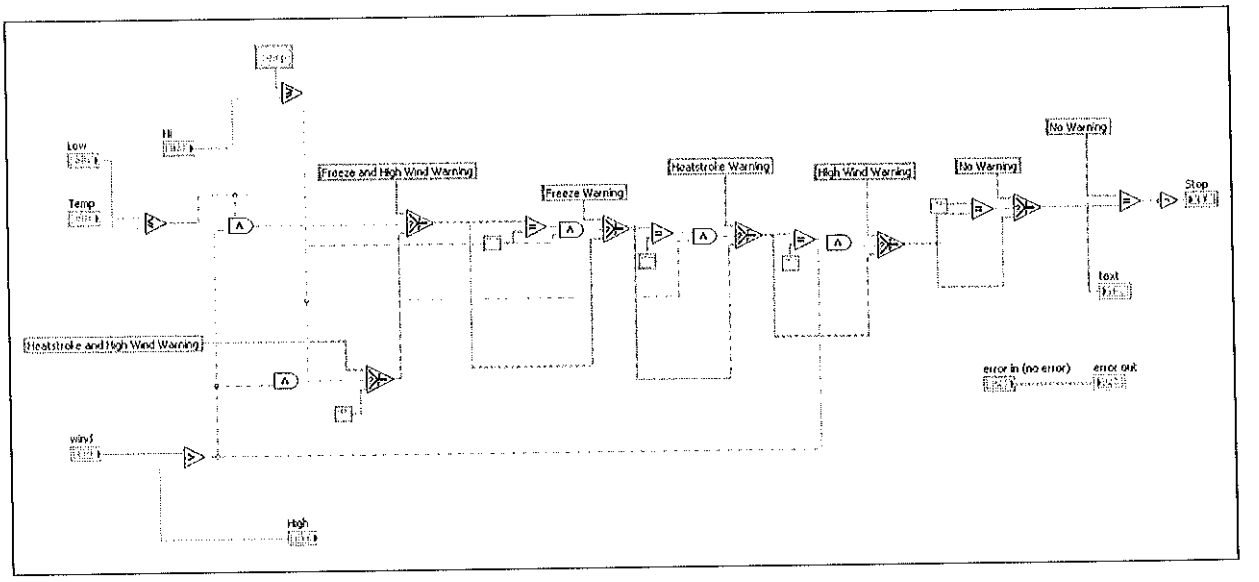


Figure 4-9. Poorly Designed Block Diagram

Improve the VI

Improve the VI in stages. Begin with the first checkmark: The block diagram is too disorganized.

1. Use the following tips to help you organize the block diagram:

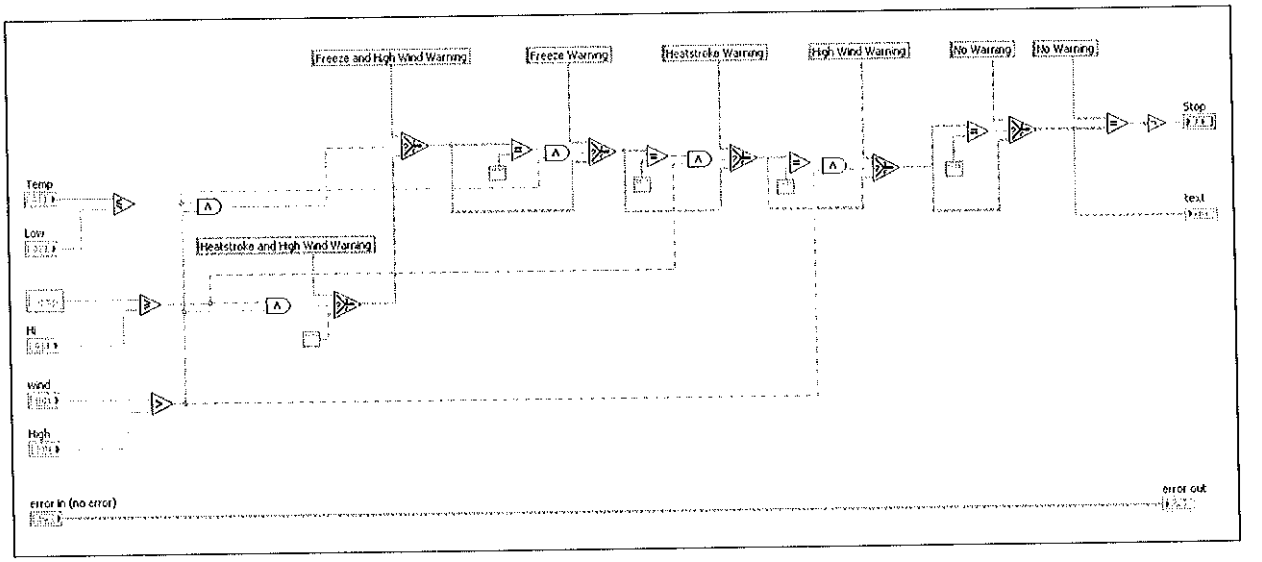


Figure 4-10. Reorganized Block Diagram

- ❑ Move all controls to the left of the block diagram.
- ❑ Move all indicators to the right of the block diagram.

- ❑ Use the **Align Objects** and **Distribute Objects** toolbar buttons to arrange the controls and indicators.
- ❑ Rearrange wires so that they do not overlap.
- ❑ Rearrange wires so that no input wires are running from right to left.
- ❑ Reduce the number of bends in wires.
- ❑ Do not allow wires to run under objects.

2. After the block diagram is better organized, rename controls and indicators using names that are more descriptive.
   - The purpose of this VI is to determine whether the current temperature and wind speed are at a level requiring a warning to generate. The VI also lights an LED if a warning occurs.
   - Suggested input names are `Current Temperature`, `Low Temp`, `High Temp`, `Current Wind Speed`, and `High Wind Speed`.
   - Suggested output names are `Warning Text` and `Warning?`.
3. Remove any unnecessary logic from the block diagram.

   Figure 4-11 shows an Equal? function followed by a Not function.

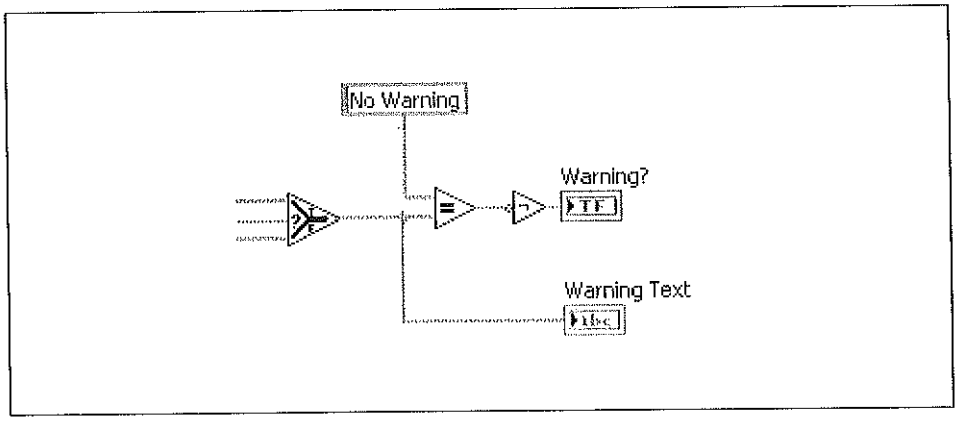


Figure 4-11. Unnecessary Logic

- ❑ You can replace this with a Not Equal? function, completing the same logic with fewer functions, as shown in Figure 4-12.

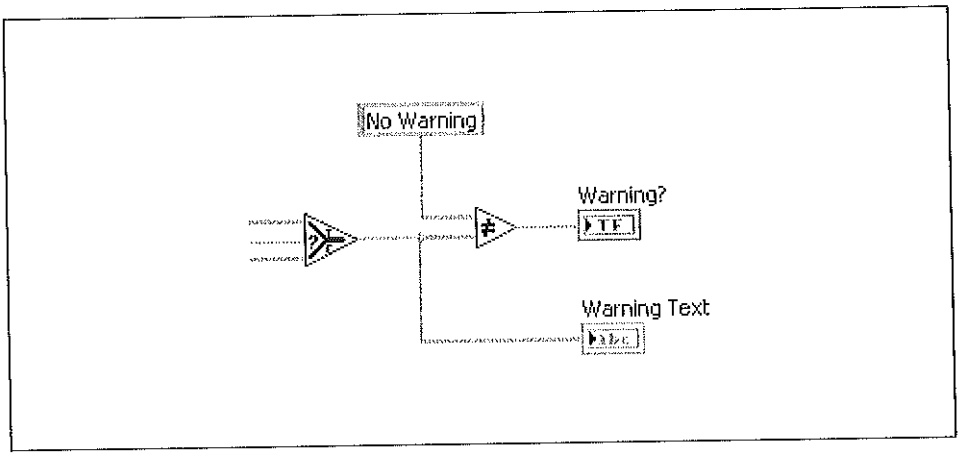


Figure 4-12. Unnecessary Logic Simplified

- ❑ You can reduce unnecessary logic even further by using the Boolean input of the Select function, as shown in Figure 4-13.

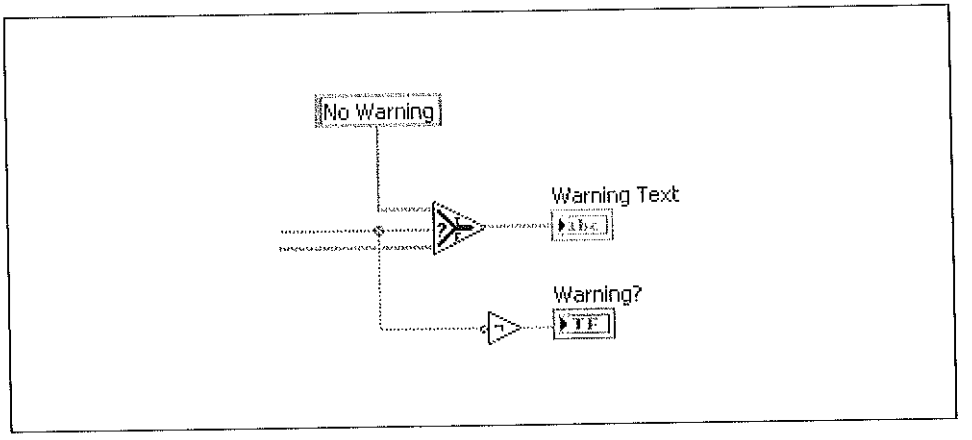


Figure 4-13. Unnecessary Logic Simplified Further

Refer to Figure 4-14 for assistance with wiring this duplicated function that occurs near the end of the VI.

- ❑ Delete the Equal? function.
- ❑ Delete the input wire to the Not function.
- ❑ Wire the input of the Not function to the input wire of the Select function.
- ❑ Test the edited VI to be sure the functionality has not changed.

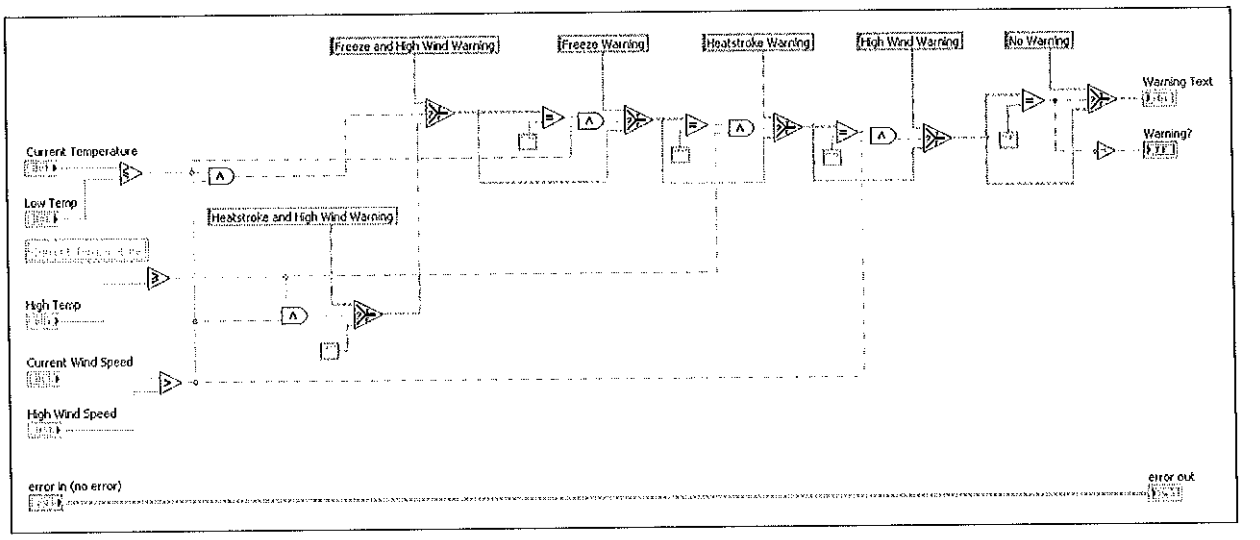


Figure 4-14. Well-Named Controls and Unnecessary Logic Removed

4. Save the VI as `Determine Warnings Good One.vi`.

Optional

1. Replace duplicated logic on the block diagram with subVIs. Figure 4-15 shows an example of the algorithm in the VI that is repeated. You can replace this algorithm with a subVI.

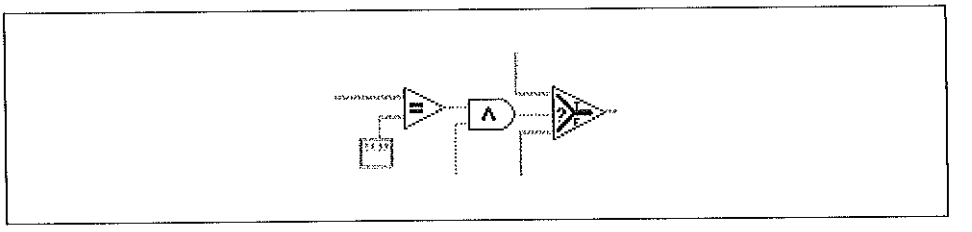


Figure 4-15. Repeated Algorithm

- ❑ Select the repeated algorithm by drawing a selection box around the objects.
- ❑ Select **Edit»Create SubVI**.
- ❑ Double-click the new subVI to open it.
- ❑ Edit the new subVI as necessary. Some things to consider: create an appropriate icon, recreate the connector pane, and rename the controls and indicators.
- ❑ Save the subVI.
- ❑ Close the subVI.

- ❑ Right-click the subVI icon on the block diagram and select **Relink to SubVI** from the shortcut menu.
- ❑ Delete the duplicated logic in other locations and replace with the new subVI.
- ❑ Test the edited VI.

2. Remove unnecessary local variables and wire to the appropriate control or indicator instead.

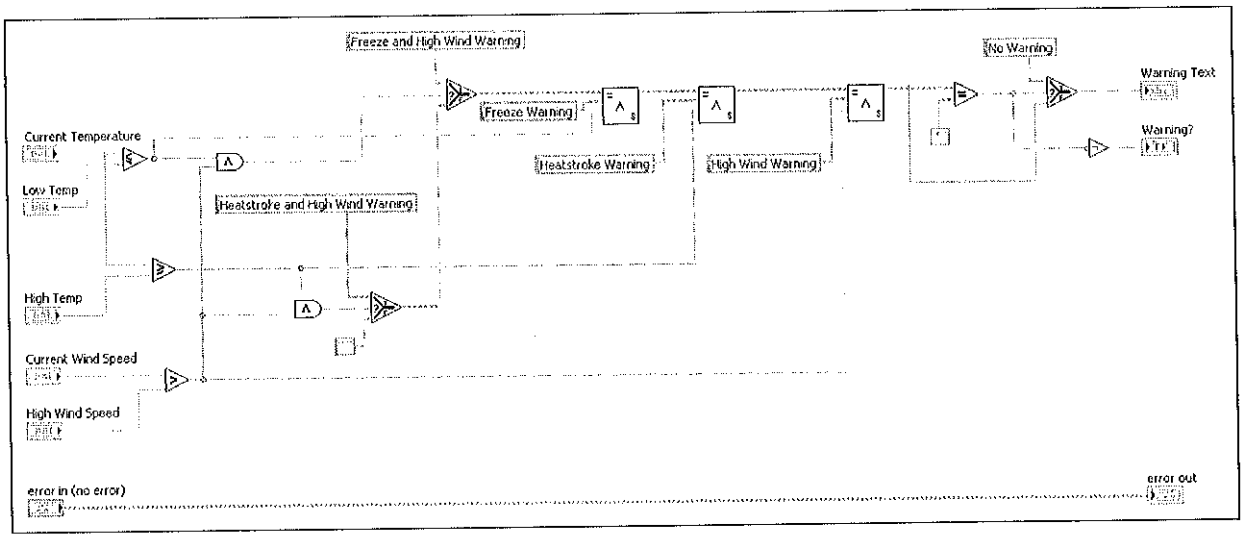


Figure 4-16. Duplicated Logic Placed in a SubVI and Local Variables Removed

3. Save the VI as `Determine Warnings Good One.vi`.

Challenge: Simplify Algorithm

If you have time remaining in this exercise, try to determine a way to simplify the algorithm and rewrite the code so that is easier to modify later.

An example solution is shown in Figure 4-17 using a state machine. The states contained are: Heatstroke, Freeze, High Wind, and Generate Warning. You can explore this solution in the `Determine Warnings State Machine.vi` located in the `<Exercises>\LabVIEW Basics I\Determine Warnings` directory. The course project also uses this solution.

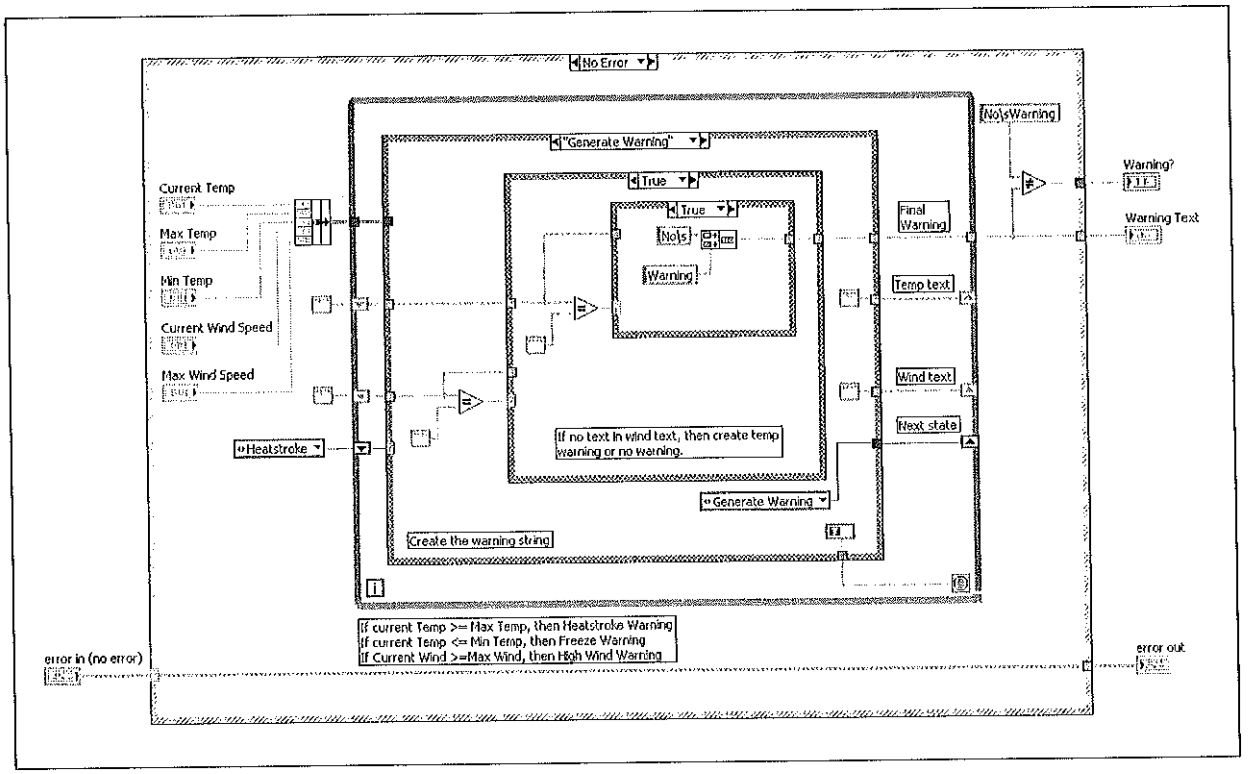


Figure 4-17. Simplified Algorithm that is Readable and Maintainable

End of Exercise 4-1

Job Aid

Use the following refactoring checklist to help determine if you should refactor a VI. If you answer yes to any of the items in the checklist, refer to the guidelines in the *When to Refactor* section of this lesson to refactor the VI.

- ❑ Disorganized block diagram.
- ❑ Overly large block diagram.
- ❑ Poorly named objects and poorly designed icons.
- ❑ Unnecessary logic.
- ❑ Duplicated logic.
- ❑ Lack of dataflow programming.
- ❑ Complicated algorithms.

Notes

5

Controlling the User Interface

When writing programs, often you must change the attributes of front panel objects programmatically. For example, you may want to make an object invisible until a certain point in the execution of the program. In LabVIEW, you can use VI Server to access the properties and methods of front panel objects. This lesson explains the VI Server, Property Nodes, control references, and Invoke Nodes.

Topics

A. VI Server Architecture

B. Property Nodes

C. Control References

D. Invoke Nodes

A. VI Server Architecture

The VI Server is an object-oriented, platform-independent technology that provides programmatic access to LabVIEW and LabVIEW applications. VI Server performs many functions; however, this lesson concentrates on using the VI Server to control front panel objects and edit the properties of a VI and LabVIEW.To understand how to use VI Server, it is useful to understand the terminology associated with it.

Object-Oriented Terminology

Object-oriented programming is based on objects. An *object* is a member of a class. A *class* defines what an object is able to do, what operations it can perform (methods), and what properties it has, such as color, size, and so on.

Objects can have methods and properties. *Methods* perform an operation, such as reinitializing the object to its default value. *Properties* are the attributes of an object. The properties of an object could be its size, color, visibility, and so on.

Control Classes

LabVIEW front panel objects inherit properties and methods from a class. When you create a Stop control, it is an object of the Boolean class and has properties and methods associated with that class, as shown in Figure 5-1.

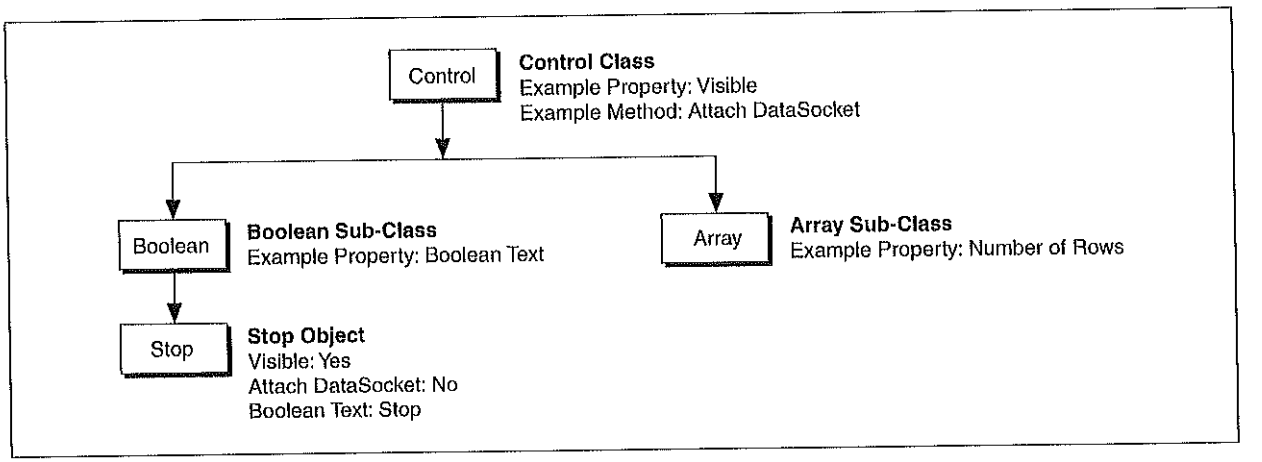


Figure 5-1. Boolean Class Example

VI Class

Controls are not the only objects in LabVIEW to belong to a class. A VI belongs to the VI Class and has its own properties and methods associated with it. For instance, you can use VI class methods to abort a VI, to adjust the position of the front panel window, and to get an image of the block

diagram. You can use VI class properties to change the title of a front panel window, to retrieve the size of the block diagram, and to hide the **Abort** button.

B. Property Nodes

Property Nodes access the properties of an object. In some applications, you might want to programmatically modify the appearance of front panel objects in response to certain inputs. For example, if a user enters an invalid password, you might want a red LED to start blinking. Another example is changing the color of a trace on a chart. When data points are above a certain value, you might want to show a red trace instead of a green one. Property Nodes allow you to make these modifications programmatically. You also can use Property Nodes to resize front panel objects, hide parts of the front panel, add cursors to graphs, and so on.

Property Nodes in LabVIEW are very powerful and have many uses. Refer to the *Property Node* topic of the *LabVIEW Help* for more information about Property Nodes.

Creating Property Nodes

When you create a Property Node from a front panel object by right-clicking the object, selecting **Create»Property Node**, and selecting a property from the shortcut menu, LabVIEW creates a Property Node on the block diagram that is implicitly linked to the front panel object.

If the object has a label, the Property Node has the same label. You can change the label after you create the node. You can create multiple Property Nodes for the same front panel object.

Using Property Nodes

When you create a Property Node, it initially has one terminal representing a property you can modify for the corresponding front panel object. Using this terminal on the Property Node, you can either set (write) the property or get (read) the current state of that property.

For example, if you create a Property Node for a digital numeric control using the Visible property, a small arrow appears on the right side of the Property Node terminal, indicating that you are reading that property value. You can change the action to write by right-clicking the terminal and selecting **Change To Write** from the shortcut menu. Wiring a False Boolean value to the Visible property terminal causes the numeric control to vanish from the front panel when the Property Node receives the data. Wiring a True Boolean value causes the control to reappear.

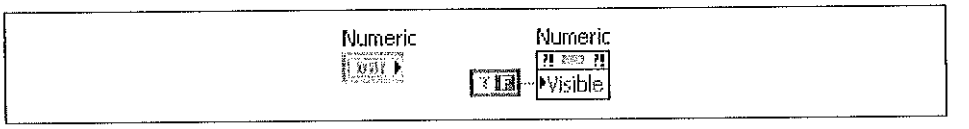


Figure 5-2. Using Property Nodes

To get property information, right-click the node and select **Change All to Read** from the shortcut menu. To set property information, right-click the node and select **Change All to Write** from the shortcut menu. If the small direction arrow on the Property Node is on the right, you are getting the property value. If the small direction arrow on a Property Node is on the left, you are setting the property value. If the Property Node in Figure 5-2 is set to Read, when it executes it outputs a True value if the control is visible or a False value if it is invisible.

Tip Some properties are read-only, such as the Label property, or write only, such as the Value (Signaling) property.

To add terminals to the node, right-click the white area of the node and select **Add Element** from the shortcut menu or use the Positioning tool to resize the node. Then, you can associate each Property Node terminal with a different property from its shortcut menu.

Tip Property Nodes execute each terminal in order from top to bottom.

Some properties use clusters. These clusters contain several properties that you can access using the cluster functions. Writing to these properties as a group requires the Bundle function and reading from these properties requires the Unbundle function. To access bundled properties, select **All Elements** from the shortcut menu. For example, you can access all the elements in the Position property by selecting **Properties»Position» All Elements** from the shortcut menu.

However, you also can access the elements of the cluster as individual properties, as shown in Figure 5-3.

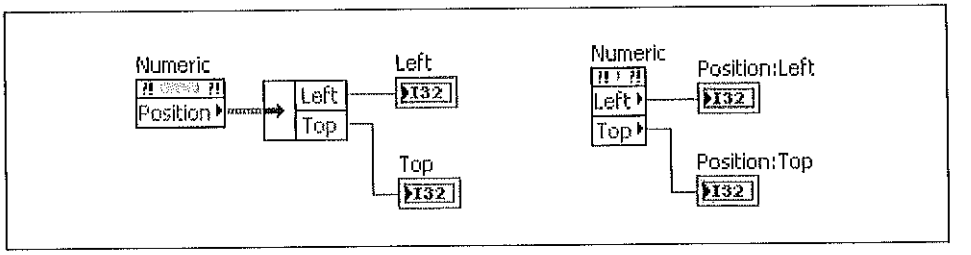


Figure 5-3. Properties Using Clusters

Exercise 5-1 Temperature Limit VI

Goal

Use Property Nodes to change the properties of front panel objects programmatically.

Scenario

Complete a VI that records temperature to a waveform chart. During execution, the VI performs the following tasks:

- Set the Δx value of the chart to the user-defined value.
- Clear the waveform chart so it initially contains no data.
- Change the color of a plot if the data exceeds a certain value.
- Blink an alarm indicator if the data exceeds a certain value.

Design

Add the following Property Nodes to the existing VI:

Type	Name	Property
Waveform Chart	`Temperature`	XScale.Multiplier
Waveform Chart	`Temperature`	History
Waveform Chart	`Temperature`	Active Plot 0»Plot.Color
Boolean Indicator (LED)	`Over Limit`	Blinking

Implementation

1. Open `Temperature Limit.vi` located in the `<Exercises>\LabVIEW Basics II\Temperature Limit` directory. The front panel is already created for you.

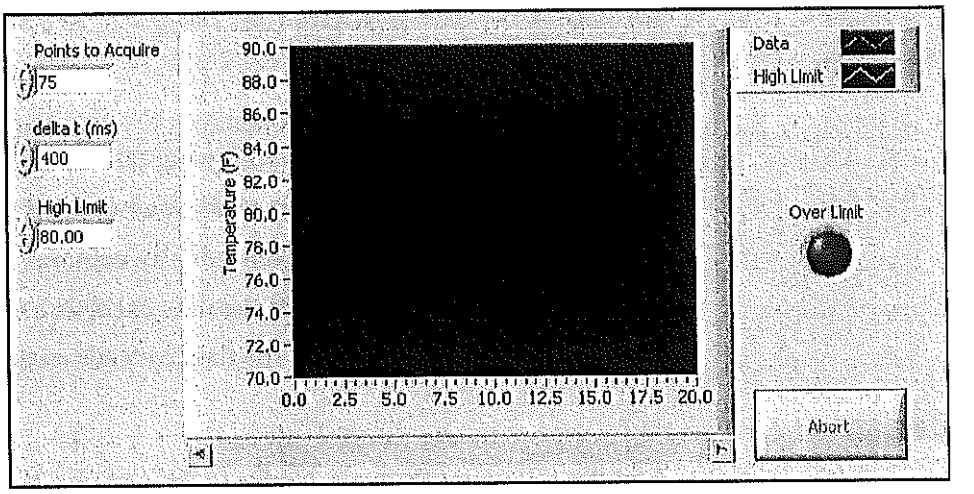


Figure 5-4. Temperature Limit Front Panel

2. Open the block diagram of the VI. A portion has been created for you. Figure 5-5 shows an example of the final block diagram.

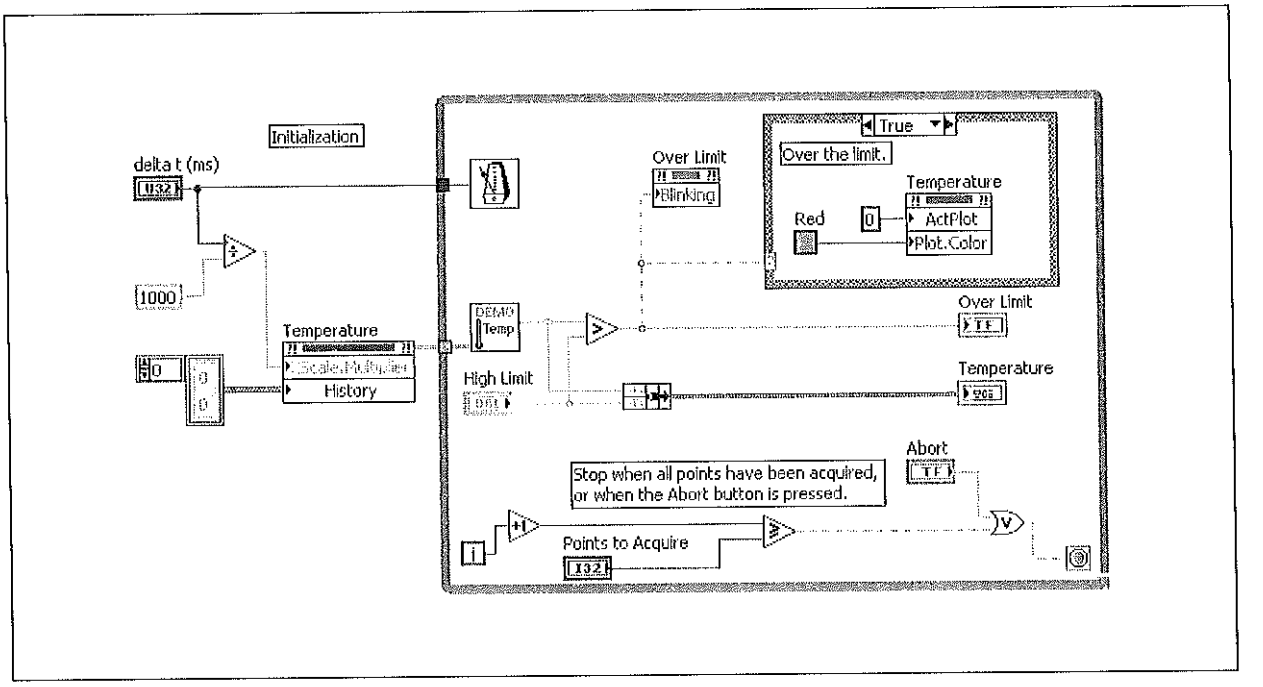


Figure 5-5. Temperature Limit Block Diagram

3. Modify the VI so that it sets the Δx value of the chart to the Δt (ms) value input by the user.

   - ❑ Right-click the Temperature indicator and select **Create» Property Node»X Scale»Offset and Multiplier»Multiplier** from the shortcut menu to create a Property Node.
   - ❑ Place the new Property Node to the left of the While Loop.
   - ❑ Right-click the Property Node and select **Change All To Write** from the shortcut menu.
   - ❑ Divide **delta t (ms)** by `1000` to determine the X-Scale Multiplier, as shown in Figure 5-5.

4. Modify the VI to clear old data from the Temperature chart before starting the temperature acquisition.



Tip To clear a waveform chart from the block diagram, send an empty array of data to the History Data property.

   - ❑ Resize the Property Node to two terminals.
   - ❑ Select the **History Data** property in the second terminal.
   - ❑ Verify that the History Data property is set to Write.
   - ❑ Right-click the History Data property and select **Create»Constant** from the shortcut menu.
   - ❑ Wire the Property Node as shown in Figure 5-5.

5. Modify the VI so that when the VI acquires data, it turns the **Data** trace red and the **Over Limit** LED blinks when the temperature exceeds the limit value.

   - ❑ Right-click the Temperature terminal and select **Create» Property Node»Active Plot** from the shortcut menu to create another Property Node.
   - ❑ Place the new Property Node in the True case of the Case structure.
   - ❑ Resize the node to two terminals.
   - ❑ Click the second terminal and select **Plot»Plot Color**.
   - ❑ Right-click the Property Node and select **Change All To Write** from the shortcut menu.

- ❑ Wire a numeric constant with a value of 0 to the Active Plot property to select the first plot on the Temperature chart.
- ❑ Wire the Red color box constant to the Plot Color property to set the plot color to red when the data rises above the High Limit.
- ❑ Create a copy of the Property Node by pressing <Ctrl> while selecting and dragging the Property Node.

Tip Do not use the clipboard (**Edit»Copy** or <Ctrl>-C) to create a copy of the Property Node. This creates a different type of Property Node that you learn about in the *Control References* section.

- ❑ Place the copy of the Property Node in the False case of the Case structure, as shown in Figure 5-6.
- ❑ Wire a numeric constant with a value of 0 to the Active Plot property to select the first plot on the Temperature chart.
- ❑ Connect the Green color box constant to the Plot Color property to set the plot color to green when the data is below the High Limit.

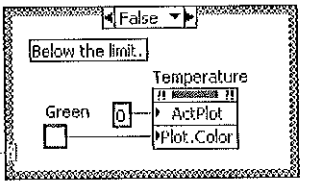


Figure 5-6. False Case in the Temperature Limit VI

6. Modify the VI so that when the VI acquires data, the **Over Limit** LED blinks when the temperature exceeds the limit value.
   - ❑ Right-click the **Over Limit** terminal and select **Create»Property Node»Blinking** from the shortcut menu.
   - ❑ Place the new Property Node inside the While Loop.
   - ❑ Right-click the Property Node and select **Change All To Write** from the shortcut menu.
   - ❑ Wire the Property Node as shown in Figure 5-5.
7. Save the VI.

Test

1. Run the VI to confirm that it behaves correctly.
2. Close the VI.

End of Exercise 5-1

C. Control References

A Property Node created from the front panel object or block diagram terminal is an implicitly linked Property Node. This means that the Property Node is linked to the front panel object. What if you must place your Property Nodes in a subVI? Then the objects are no longer located on the front panel of the VI that contains the Property Nodes. In this case, you need an explicitly linked Property Node. You create an explicitly linked Property Node by wiring a reference to a generic Property Node.

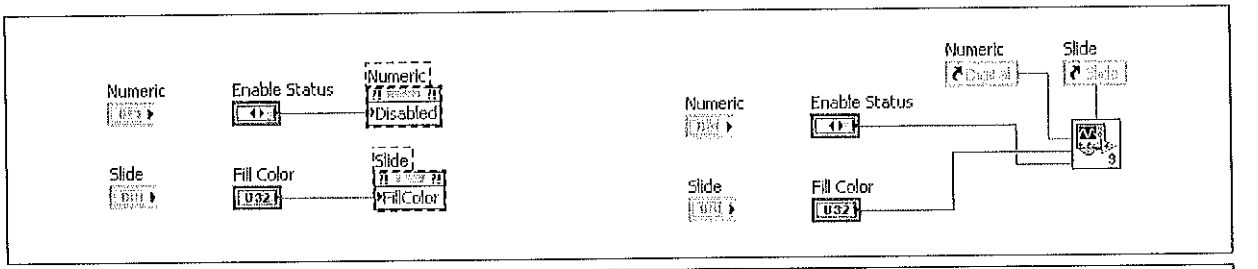
If you are building a VI that contains several Property Nodes or if you are accessing the same property for several different controls and indicators, you can place the Property Node in a subVI and use control references to access that node. A control reference is a reference to a specific front panel object.

This section shows one way to use control references. Refer to the *Controlling Front Panel Objects* topic of the *LabVIEW Help* for more information about control references.

Creating a SubVI with Property Nodes

As shown in Figure 5-7, the simplest way to create explicitly linked Property Nodes is to complete the following steps:

1. Create your VI.
2. Select the portion of the block diagram that is in the subVI, as shown in the first part of Figure 5-7.
3. Select **Edit»Create SubVI**. LabVIEW automatically creates the control references needed for the subVI.
4. Customize and save the subVI. As you can see in the second part of Figure 5-7, the subVI uses the default icon.



Objects Selected to Create SubVI | **Edit»Create SubVI** Used

Figure 5-7. Using **Edit»Create SubVI** to Create Control References

Figure 5-8 shows the subVI created. Notice that the front panel Control Refnum controls have been created and connected to a Property Node on the block diagram.

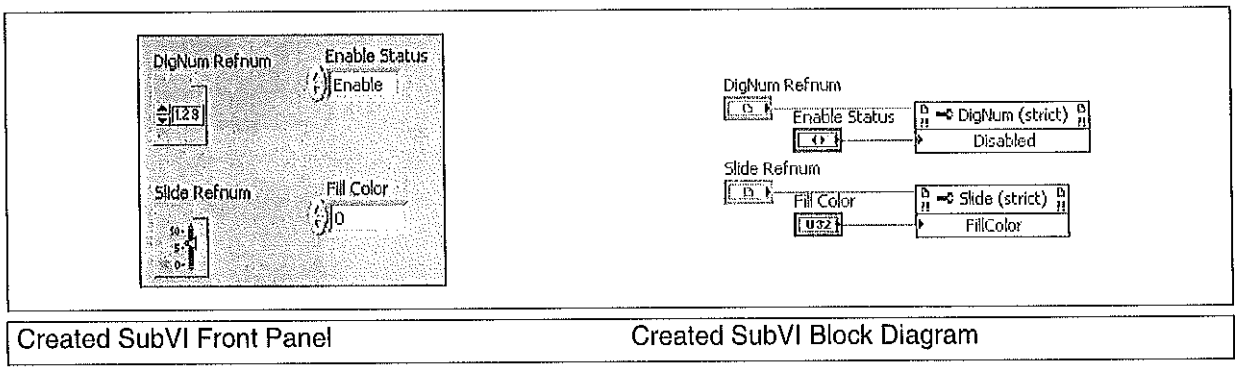


Figure 5-8. Sub VI Created Using **Edit»Create SubVI**

Note A red star on the Control Reference control indicates that the refnum is strictly typed. Refer to the *Strictly Typed and Weakly Typed Control Refnums* section of the *Controlling Front Panel Objects* topic of the *LabVIEW Help* for more information about weakly and strictly typed control references.

Creating Control References

To create a control reference for a front panel object, right-click the object or its block diagram terminal and select **Create»Reference** from the shortcut menu.

You can wire this control reference to a generic Property Node. You can pass the control reference to a subVI using a control refnum terminal.

Using Control References

Setting properties with a control reference is useful for setting the same property for multiple controls. Some properties apply to all classes of controls, such as the Disabled property. Some properties are only applicable to certain control classes, such as the Lock Boolean Text in Center property.

The following example shows how to construct a VI that uses a control reference on the subVI to set the Enable/Disable state of a control on the main VI front panel.

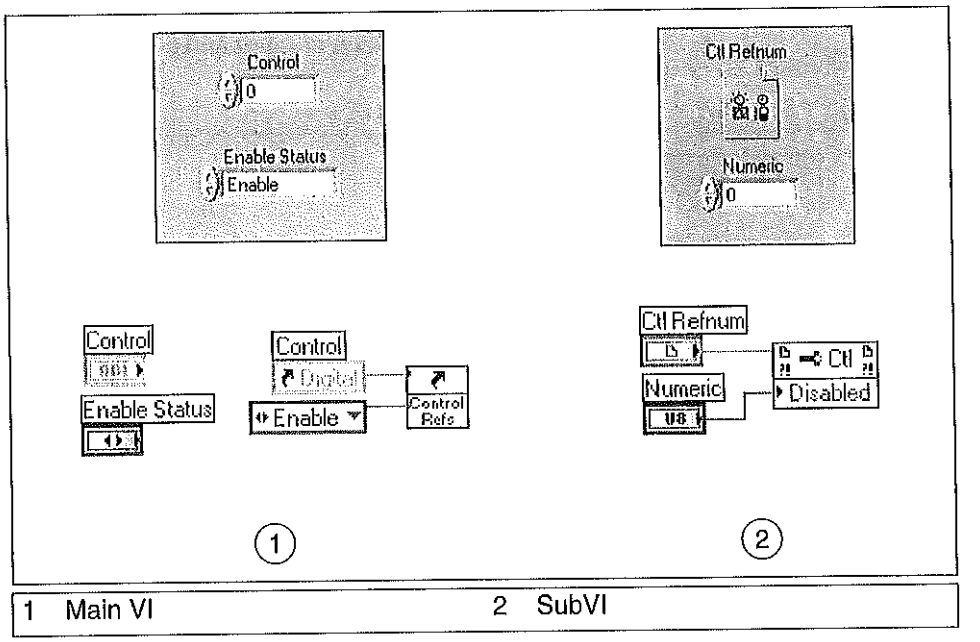


Figure 5-9. Control References

The main VI sends a reference for the digital numeric control to the subVI along with a value of zero, one, or two from the enumerated control. The subVI receives the reference by means of the **Ctl Refnum** on its front panel window. Then, the reference is passed to the Property Node. Because the Property Node now links to the numeric control in the main VI, the Property Node can change the properties of that control. In this case, the Property Node manipulates the Enabled/Disabled state.

Notice the appearance of the Property Node in the block diagram. You cannot select a property in a generic Property Node until the class is chosen. The class is chosen by wiring a reference to the Property Node. This is an example of an explicitly linked Property Node. It is not linked to a control until the VI is running and a reference is passed to the Property Node. The advantage of this type of Property Node is its generic nature. Because it has no explicit link to any one control, it may be reused for many different controls. This generic Property Node is available on the **Functions** palette.

Selecting the Control Type

When you add a Control Refnum to the front panel of a subVI, you next need to specify the VI Server Class of the control. This specifies the type of control references that the subVI will accept. In the previous example, Control was selected as the VI Server Class type, as shown in Figure 5-9. This allows the VI to accept a reference to any type of front panel control.

However, you can specify a more specific class for the refnum to make the subVI more restrictive. For example, you can select Digital as the class, and the subVI only can accept references to numeric controls of the class

Digital. Selecting a more generic class for a control refnum allows it to accept a wider range of objects, but limits the available properties to those that apply to all objects which the Property Node can accept.

To select a specific control class, right-click the control and select **Select VI Server Class»Generic»GObject»Control** from the shortcut menu. Then, select the specific control class.

Exercise 5-2 Set Plot Names

Goal

Use control references to create a subVI that modifies graph or chart properties.

Scenario

Create a subVI that allows you to assign a list of plot names to a chart or graph. The subVI should resize the plot legend as necessary to display all of the plots.

Design

Inputs and Outputs

Type	Name	Default Value
Control Reference to a GraphChart object	Graph Reference	N/A
1-D Array of Strings Control	Plot Names	Empty Array
Error Cluster Control	Error In	No Error
Error Cluster Indicator	Error Out	No Error

Control References

The only class that contains both the Waveform Chart and the Waveform Graph is the GraphChart class. To write a subVI that can accept references to both charts and graphs you must use a weakly typed control reference of the GraphChart class. However, this class also contains other charts and graphs, such as the XY Graph. This subVI generates an error if the user wires any type of graph other than a Waveform Chart or a Waveform Graph. You can determine if the user has wired the correct type by using the ClassName property to control a Case structure. If the correct class is wired, use the To More Specific Class function to get a reference to the appropriate subclass. After you have a reference to a WaveformChart or a WaveformGraph you can set the properties to modify plot names.

Properties

Graphs and charts do not have a single property to set all of the plot names. Instead you must use a combination of properties to set each plot name. In this exercise, use the following properties:

ClassName—This property returns a string indicating the control class of the object that the property is called on. You can access this property for any control.

LegAutosize—This property controls whether the Graph Legend automatically resizes to accommodate the plot names within it. Before modifying the plot names you should set this property to False. Otherwise, the legend may resize in such a way that it is separated from the graph or covers the graph or other controls.

LegNumRows—This property controls the number of rows on the plot legend. When adding your legend to the front panel, remember to leave room for the legend to expand when you set this property. The legend expands downwards.

ActPlot—Properties affecting a plot act on one plot at a time. This property controls the active plot. Any time a plot property is set or read it applies to the active plot. The plots are numbered sequentially as they are created, starting with zero.

Plot.Name—This property sets the name of the active plot.

Implementation

1. Open a blank VI.
2. Save the VI as `Set Plot Names.vi` in the `<Exercises>\LabVIEW Basics II\Set Plot Names` directory.
3. Create the front panel window.



- ❑ Add a control refnum to the front panel window.
- ❑ Name the control refnum `Graph Reference`.
- ❑ Right-click Graph Reference and choose **Select VI Server Class»Generic»GObject»Control»GraphChart»GraphChart** from the shortcut menu.



- ❑ Add an array to the front panel window.
- ❑ Name the array `Plot Names`.

- ❑ Add a string control to the **Plot Names** array.
- ❑ Add an error in cluster.
- ❑ Add an error out cluster.
- ❑ Arrange the controls as shown in Figure 5-10.

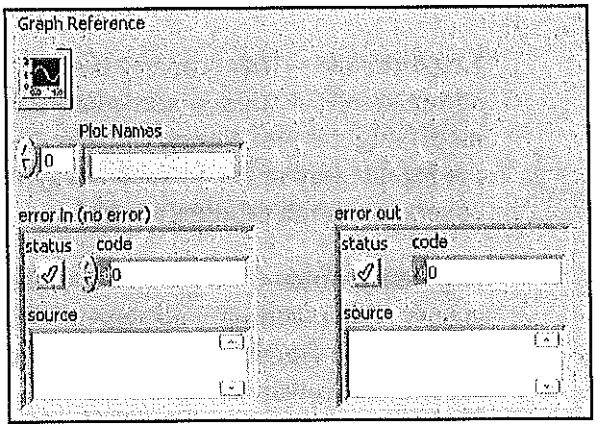


Figure 5-10. Set Plot Names Front Panel

Tip Because the front panel of this subVI is not displayed to the user, you do not have to put as much effort into making it visually appealing. You should always organize your front panels logically. However, you should not spend too much time on panels that the user does not see.

4. Switch to the block diagram.
5. Identify the class of the control reference and generate an error if it has an invalid class.

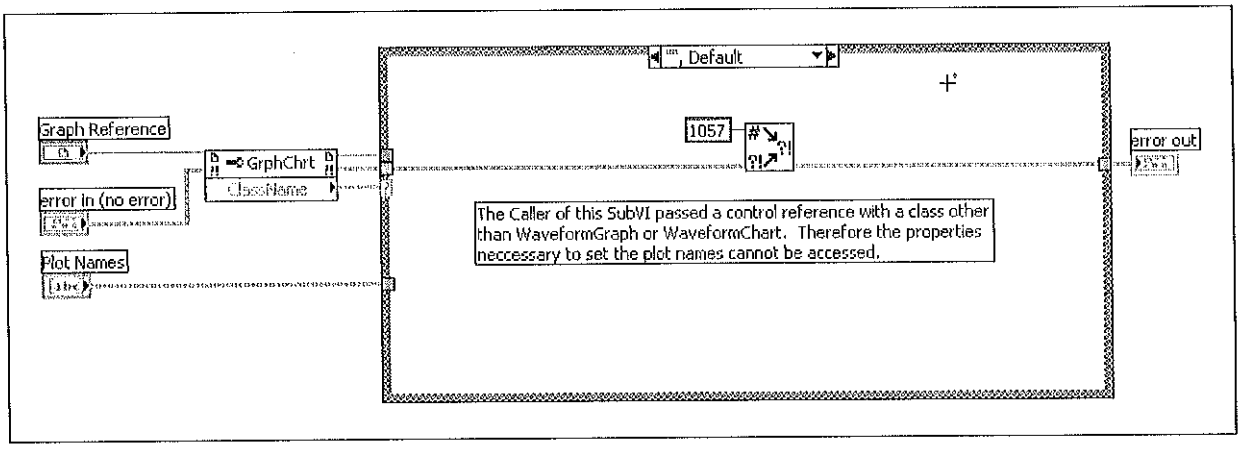


Figure 5-11. Default Case



- ❑ Add a Property Node from the **Application Control** palette to the block diagram. Most of the functions you use in this section come from the Application Control palette.
- ❑ Wire **Graph Reference** to the **reference** input of the Property Node.
- ❑ Click the Property Node terminal and select the **Class Name property**.



- ❑ Add a Case structure to the block diagram, as shown in Figure 5-11.
- ❑ Wire the **ClassName** output of the Property Node to the case selector of the Case structure.
- ❑ Switch to the False case of the Case structure.
- ❑ Delete the `False` text in the case name so that the case name resembles Figure 5-11.

  The Default case of the Case structure is selected if the class of the control reference does not match one of the other cases. In this case, if the default case executes, then the control reference passed to this subVI is not a WaveformGraph or a WaveformChart. Remember for a Case structure, the case selector label is case sensitive.



- ❑ Add an Error Cluster From Error Code VI to the Case structure.
- ❑ Right-click the **error code** input of the Error Cluster From Error Code VI and select **Create»Constant** from the shortcut menu.
- ❑ Enter `1057` in the constant.

  Error code `1057` corresponds to the message **`Object cannot be typecasted to the specified type`**. This is the appropriate error to generate if the caller of the subVI passes a control reference of the wrong class.
- ❑ Wire the block diagram as shown in Figure 5-11.

6. Handle the WaveformGraph references.

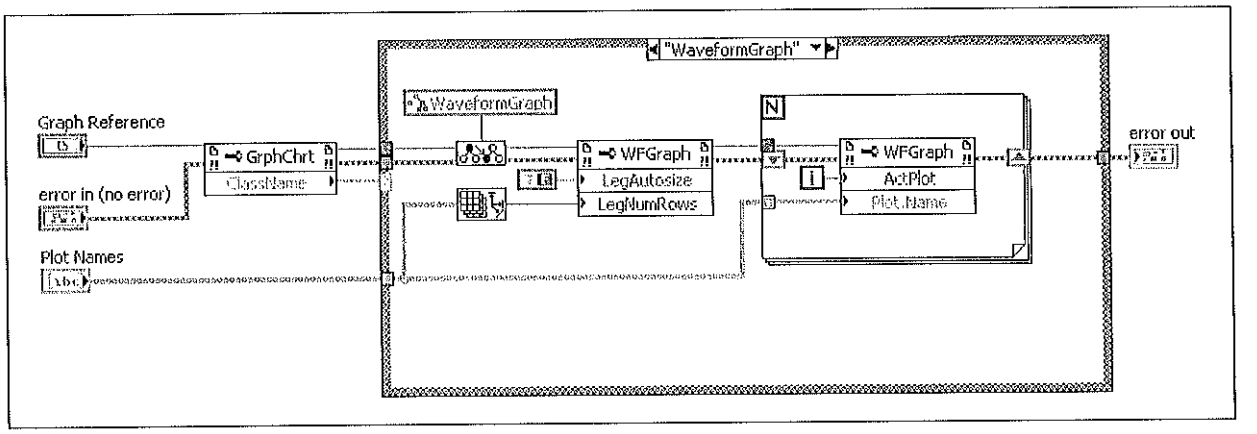


Figure 5-12. WaveformGraph Case

- ❑ Switch to the True case of the Case structure.
- ❑ Change the `True` text in the case name to `WaveformGraph` so that the case name resembles Figure 5-12.

Caution The text entered the case selector label must *exactly* match the input string, including spaces and case. For this example, enter `WaveformGraph`.

- ❑ Add a To More Specific Class function to the Case structure
- ❑ Right-click the **target class** input of the To More Specific Class function and select **Create»Constant** from the shortcut menu.
- ❑ Click the constant you created in the previous step and select the **Generic»GObject»Control»GraphChart»WaveformGraph» WaveformGraph** class.
- ❑ Add a Property Node to the Case structure
- ❑ Wire the **specific class reference** output of the To More Specific Class function to the **reference** input of the Property Node.
- ❑ Click the Property Node terminal and select **Legend»Autosize**.
- ❑ Expand the Property Node to display two properties.
- ❑ Click the second property in the Property Node and select **Legend» Number of Rows**.
- ❑ Right-click the Property Node and select **Change All To Write** from the shortcut menu.

- ❑ Right-click the **LegAutosize** property and select **Create»Constant** from the shortcut menu. Verify that the value of the constant is False.



- ❑ Add an Array Size function to the Case structure.



- ❑ Add a For Loop to the Case structure.



- ❑ Add a Property Node to the For Loop.

- ❑ Wire the dup **reference out** output of the first Property Node through the border of the For Loop to the **reference** input of the second Property Node.

- ❑ Click the Property Node terminal and select **Active Plot** from the list.

- ❑ Expand the Property Node to display two properties.

- ❑ Click the second property in the Property Node and select **Plot» Plot Name** from the list.

- ❑ Right-click the Property Node and select **Change All To Write** from the shortcut menu.

- ❑ Wire the block diagram as shown in Figure 5-12. Use shift registers when wiring the error wires through the For Loop and enable Auto Indexing for the Plot Names terminal.

7. Handle WaveformChart references.

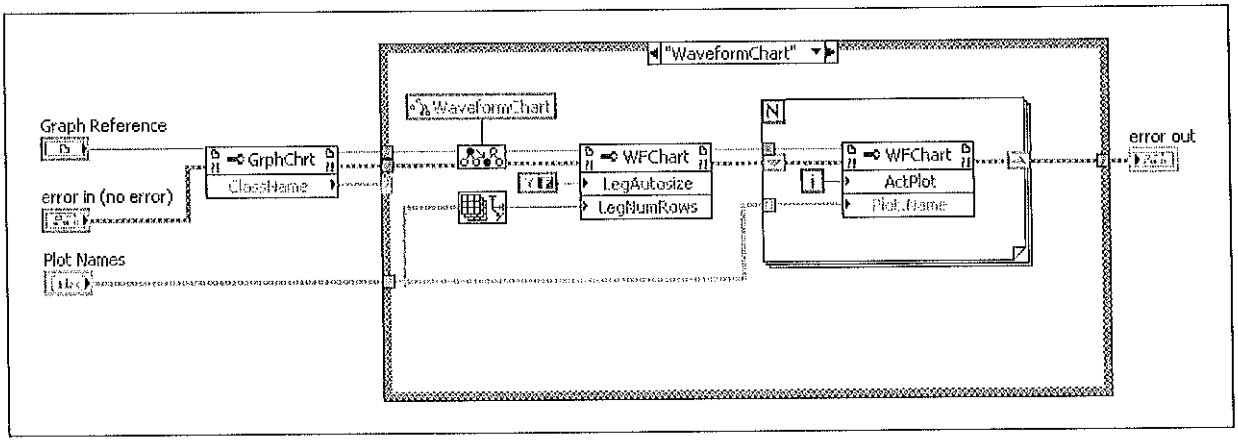


Figure 5-13. WaveformChart Case

- ❑ Right-click the border of the Case structure and select **Duplicate Case** from the shortcut menu.

- ❑ Enter `WaveformChart` in the case name.

Caution The text entered the case selector label must *exactly* match the input string, including spaces and case. For this example, enter `WaveformChart`.

- ❑ Click the WaveformGraph reference constant and select **Generic»GOjbect»Control»GraphChart»WaveformChart**.

Note When you change the class of a control reference, all Property Nodes and Invoke Nodes using the reference become invalid because the properties refer to a class that does not match the reference. Notice that all the property names change to black when you change the class reference and that the run arrow is broken. Leave the broken wires alone, because the wires reconnect as you reselect the properties.

- ❑ Click each of the four properties and select the correct property again. The four properties are **Legend»Autosize**, **Legend» Number of Rows**, **Active Plot**, and **Plot»Plot Name**. The resulting block diagram appears as shown in Figure 5-13.

8. Create the icon and connector pane for the subVI. Figure 5-14 shows an example icon and connector pane.

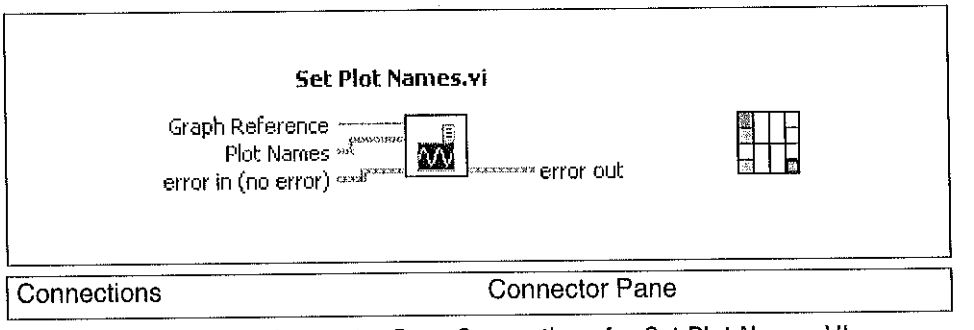


Figure 5-14. Connector Pane Connections for Set Plot Names VI

- ❑ Switch to the front panel of the VI.



- ❑ Right-click the VI icon and select **Show Connector** from the shortcut menu.
- ❑ Right-click the connector pane and select **Patterns** from the shortcut menu to choose a pattern.
- ❑ Wire the connector pane.
- ❑ Right-click the connector pane and select **Show Icon** from the shortcut menu.
- ❑ Right-click the icon and select **Edit Icon** from the shortcut menu.
- ❑ Use the tools in the Icon Editor to create an icon.

If you prefer to use a pre-built icon, select **Edit»Import Picture to Clipboard** and navigate to `<Exercises>\LabVIEW Basics II\Set Plot Names\Set Plot Names Icon.bmp`. Select **Edit» Paste**.

- ❑ Close the Icon Editor when you are finished.

9. Save the VI.

Testing

1. Test the VI using a Waveform Graph.

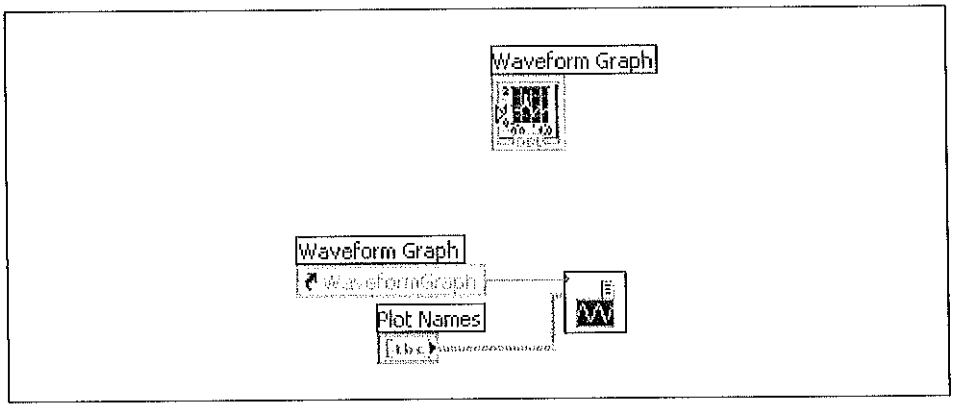


Figure 5-15. Set Plot Names Test

- ❑ Create a blank VI.



- ❑ Add a waveform graph to the front panel window.
- ❑ Open the block diagram.
- ❑ Right-click the Waveform Graph indicator and select **Create» Reference** from the shortcut menu.
- ❑ Add the Set Plot Names VI to the block diagram of the new VI.



Tip If the Set Plot Names VI is open, you can drag the icon from upper right corner of its front panel to the block diagram of the new VI.

- ❑ Wire the WaveformGraph reference to the **Graph Reference** input of the Set Plot Names VI.
- ❑ Right-click the **Plot Names** input of the Set Plot Names VI and select **Create»Control** from the shortcut menu. The block diagram should resemble Figure 5-15.
- ❑ Switch to the front panel window of the new VI.

- ❑ Enter `One` and `Two` as items in the **Plot Names** array.
- ❑ Move the **Plot Legend** to the right of the graph so that you can expand the legend.
- ❑ Run the VI. **One** and **Two** appear in the legend.

2. Test the VI using a Waveform Chart.
   - ❑ Right-click the waveform graph and select **Replace» Graph»Waveform Chart** from the shortcut menu.
   - ❑ Add `Three` as another item in the **Plot Names** array.
   - ❑ Run the VI. **Three** appears in the legend of the chart.
3. Test the VI with an XY Graph.
   - ❑ Right-click the waveform chart and select **Replace» Graph»XY Graph** from the shortcut menu.
   - ❑ Add `Four` as another item in the **Plot Names** array.
   - ❑ Run the VI. A typecasting error occurs.
4. Close the VI. You do not need to save the VI used for testing the Set Plot Names VI.

End of Exercise 5-2

D. Invoke Nodes

Invoke Nodes access the methods of an object.

Use the Invoke Node to perform actions, or methods, on an application or VI. Unlike the Property Node, a single Invoke Node executes only a single method on an application or VI. Select a method by using the Operating tool to click the method terminal or by right-clicking the white area of the node and selecting **Methods** from the shortcut menu. You also can create an implicitly linked Invoke Node by right-clicking the front panel object, selecting **Create»Invoke Node**, and selecting a method from the shortcut menu.

The name of the method is always the first terminal in the list of parameters in the Invoke Node. If the method returns a value, the method terminal displays the return value. Otherwise, the method terminal has no value.

The Invoke Node lists the parameters from top to bottom with the name of the method at the top and the optional parameters, which are dimmed, at the bottom.

Example Methods

An example of a method common to all controls is the Reinitialize to Default method. Use this method to reinitialize a control to its default value at some point in your VI. The VI class has a similar method called Reinitialize All to Default.

Figure 5-16 is an example of a method associated with the Waveform Graph class. This method exports the waveform graph image to the clipboard or to a file.

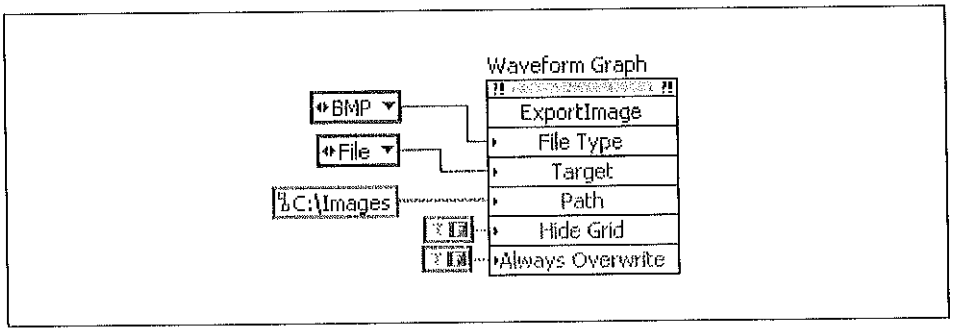


Figure 5-16. Invoke Node for the Export Image Method

Exercise 5-3 Front Panel Properties VI

Goal

Affect the attributes of a VI by using Property Nodes and Invoke Nodes.

Scenario

You can set the appearance properties of a VI statically by using the VI properties page. However, robust user interfaces often must modify the appearance of a front panel while the program runs.

You must create a VI that can perform the following tasks on demand:

- Show or hide its title bar
- Show or hide its menu bar
- Become transparent so that objects behind the VI can be seen
- Move to the center of the screen

Design

Inputs and Outputs

Type	Name	Default Value
Vertical Toggle Switch	`Show Menu Bar?`	True (Yes)
Vertical Toggle Switch	`Show Title Bar?`	True (Yes)
Vertical Toggle Switch	`Make VI Transparent?`	True (Yes)
OK Button	`Center`	False
Stop Button	`Stop`	False



Tip Use the vertical toggle switches because their default mechanical action is switch when pressed. Use the OK button because its default action is latch when released.

Properties

Use the following properties and methods on the VI class:

ShowMenuBar—When this property is true, the menu bar of the VI is visible.

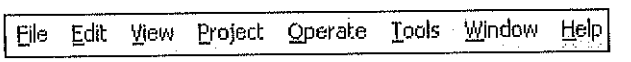


Figure 5-17. VI Menu Bar

TitleBarVisible—When this property is true, the title bar of the VI is visible.



Figure 5-18. VI Title Bar

RunVITransparently—When this property is TRUE, the transparency of the VI can vary. The default value of this property is FALSE, so you must write a TRUE value to this property before varying the transparency of the VI.

Transparency—This property varies the transparency of the VI. The property accepts any value between 0 and 100. A value of 0 makes the VI completely opaque (normal behavior), and a value of 100 makes the VI completely transparent (invisible). For this exercise, you set the value to 50 when the **Make VI Transparent?** button is clicked.

Methods

Unlike properties, a method has an effect every time you call it. Therefore, you should call methods only when you want to perform an action. For example, if you call the Fp.Center method during each iteration of a loop, the VI is continually centered, thereby preventing the user from moving it. You can use a Case structure to control calling the method in a given iteration of a loop. Use the following method on the VI class:

Center—Each time this method is called, the VI moves to the center of the screen.



Tip Use the **Context Help** window to view descriptions of each property and method.

VI Structure

The VI polls the front panel controls every 50 milliseconds and sets the value of the properties based on the current value of the controls. A Case structure controls the execution of the Center method.

Implementation

In the following steps, you create the front panel window for the VI. An example of the front panel window is shown in Figure 5-19.

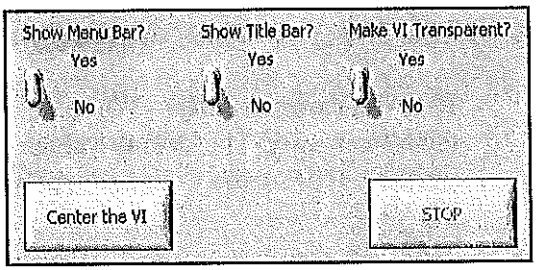


Figure 5-19. Front Panel Properties VI Front Panel Window

1. Open a blank VI.
2. Save the VI as `Front Panel Properties.vi` in the `<Exercises>\LabVIEW Basics II\Front Panel Properties` directory.
3. Create the **Show Menu Bar?** vertical toggle switch.
   - ❑ Add a vertical toggle switch to the front panel window.
   - ❑ Name the switch `Show Menu Bar?`.
   - ❑ Create free labels for the `Yes` and `No` states of the switch.
4. Create the **Show Title Bar?** switch.
   - ❑ Make a copy of the Show Menu Bar? switch.
   - ❑ Rename the switch `Show Title Bar?`.
   - ❑ Copy the free labels for the Yes and No states from the Show Menu Bar? switch.
5. Create the **Make VI Transparent?** switch.
   - ❑ Make a copy of the Show Menu Bar? switch.
   - ❑ Rename the switch `Make VI Transparent?`.
   - ❑ Copy the free labels for the Yes and No states from the Show Menu Bar? switch.
6. Create the **Center** button.
   - ❑ Add an OK button to the front panel window.

- ❑ Name the button `Center`.
- ❑ Change the Boolean text on the button to `Center the VI`.
- ❑ Right-click the button and select **Visible Items»Label** from the shortcut menu to hide the label.

7. Create the **Stop** button.



- ❑ Add a stop button to the front panel window.
- ❑ Right-click the button and select **Visible Items»Label** from the shortcut menu to hide the label.

8. Select **Edit»Make Current Values Default.**

9. Arrange and organize objects on the front panel window. Use the **Align**, **Distribute**, and **Resize** buttons on the toolbar.

In the following steps, create the block diagram for the VI. An example of the block diagram is shown in Figure 5-20.

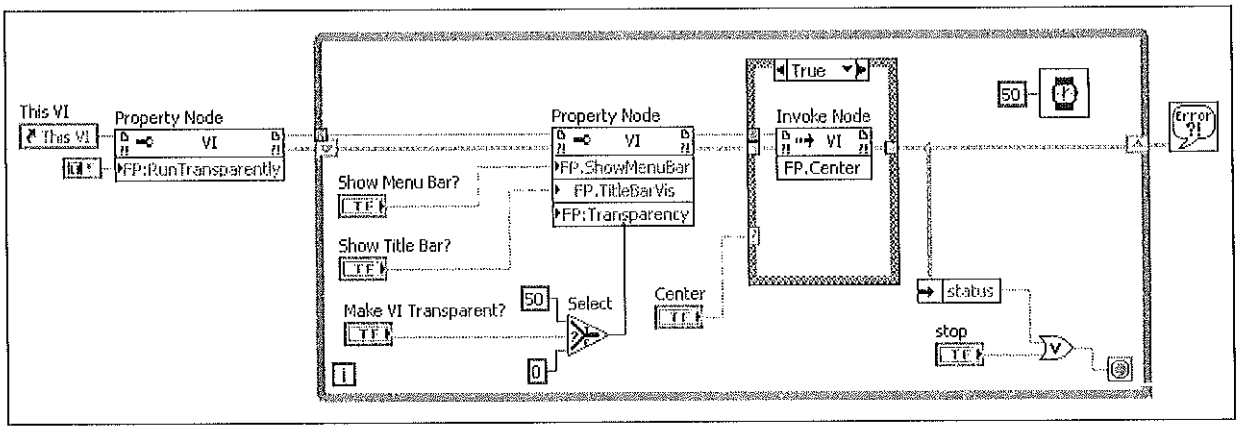


Figure 5-20. Front Panel Properties Block Diagram

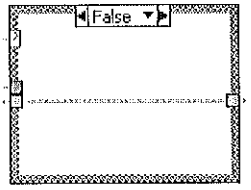


Figure 5-21. False Case for Center Method

10. Add a While Loop around the terminals.

11. Create a reference to the VI.



- ❑ Add a VI Server Reference to the block diagram to the left of the While Loop.
- ❑ Set the VI Server Reference to **This VI** if it is not already.



Note Using the This VI reference you can access all the methods and properties of the current VI without having to explicitly open and close a reference.

12. Create a Property Node for the RunTransparently property.
    - ❑ Right-click the **This VI** reference and select **Create»Property for VI Class»Front Panel Window»Run VI Transparently** from the shortcut menu to create a Property Node.
    - ❑ Move the Property Node to the right of the This VI reference, outside of the While Loop.
    - ❑ Right-click the Property Node and select **Change All to Write** from the shortcut menu.
    - ❑ Right-click the **FP.RunTransparently** property and select **Create» Constant** from the shortcut menu.
    - ❑ Change the value of the constant to True.
13. Create a Property Node for the ShowMenuBar, TitleBarVIs, and Transparency properties.
    - ❑ Right-click the This VI reference and select **Create»Property for VI Class»Front Panel Window»Show Menu Bar** from the shortcut menu to create another Property Node.
    - ❑ Expand the Property Node to show three elements.
    - ❑ Click the second item in the Property Node and select **Front Panel Window»Title Bar Visible**.
    - ❑ Click the third item in the Property Node and select **Front Panel Window»Transparency**.
    - ❑ Right-click the Property Node and select **Change All to Write** from the shortcut menu.
    - ❑ Move the Property Node inside the While Loop.



- ❑ Add a Select function inside the While Loop.

- ❑ Add two numeric constants with values 0 and 50 to the left of the Select function.
- ❑ Wire the 0 numeric constant to the **f** input of the Select function.
- ❑ Wire the 50 numeric constant to the **t** input of the Select function.
- ❑ Wire the Boolean controls to the appropriate properties, as shown in Figure 5-20.

14. Create an Invoke Node for the Center method.
    - ❑ Right-click the **This VI** reference and select **Create»Method for VI Class»Front Panel»Center** from the shortcut menu to create an Invoke Node.



15. Add a Case structure around the FP.Center Invoke Node.

16. Add a 50 ms wait to the loop.



    - ❑ Add a Wait (ms) function to the While Loop.
    - ❑ Right-click the **milliseconds to wait** input and select **Create» Constant** from the shortcut menu.
    - ❑ Enter 50 in the constant.

17. Set the While Loop to stop when the user clicks the Stop button or when an error occurs.



    - ❑ Add an Unbundle By Name function to the While Loop.



    - ❑ Add an Or function to the While Loop.

18. Wire the diagram as shown in Figure 5-20 and Figure 5-21. Make sure to replace the error cluster tunnel with a shift register.

19. Display any errors that may occur to the user.



    - ❑ Add a Simple Error Handler VI to the right of the While Loop.
    - ❑ Wire the Simple Error Handler VI to the error cluster output shift register from the While Loop.

20. Save the VI.

Testing

1. Switch to the front panel window of the VI.
2. Run the VI.
3. Try each of the buttons and observe the results.

End of Exercise 5-3

Self-Review: Quiz

1. For each of the following items, determine whether they operate on a VI class or a Control class.
   - Format and Precision
   - Blinking
   - Reinitialize to Default Value
   - Show Tool Bar



2. You have a ChartGraph control refnum, shown at left, in a subVI. Which of the following control references could you wire to the control refnum terminal of the subVI? (multiple answers)
   a. Control reference of an XY graph
   b. Control reference of a numeric array
   c. Control reference of a waveform chart
   d. Control reference of a Boolean control

Self-Review: Quiz Answers

1. For each of the following items, determine whether they operate on a VI class or a Control class.
   - Format and Precision: **Control**
   - Blinking: **Control**
   - Reinitialize to Default Value: **Control**
   - Show Tool Bar: **VI**



2. You have a GraphChart control refnum, shown at left, in a subVI. Which control references could you wire to the control refnum terminal of the subVI?
   a. **Control reference of an XY graph**
   b. Control reference of a numeric array
   c. **Control reference of a waveform chart**
   d. Control reference of a Boolean control

Notes

6

Advanced File I/O Techniques

Frequently, the decision to separate the production of data and the consumption of data into separate processes occurs because you must write the data to a file as it is acquired. This lesson explains ASCII, Binary, and Technical Data Management Streaming (TDMS) file formats and when each is a good choice for your application.

Topics

A. File Formats

B. Binary Files

C. TDMS Files

A. File Formats

At their lowest level, all files written to your computer's hard drive are a series of binary bits. However, many formats for organizing and representing data in a file are available. In LabVIEW, three of the most common techniques for storing data are the ASCII file format, direct binary storage, and the TDMS file format. Each of these formats has advantages and some formats work better for storing certain data types than others.

When to Use Text (ASCII) Files

Use text format files for your data to make it available to other users or applications if disk space and file I/O speed are not crucial, if you do not need to perform random access reads or writes, and if numeric precision is not important.

Text files are the easiest format to use and to share. Almost any computer can read from or write to a text file. A variety of text-based programs can read text-based files.

Store data in text files when you want to access it from another application, such as a word processing or spreadsheet application. To store data in text format, use the **String** functions to convert all data to text strings. Text files can contain information of different data types.

Text files typically take up more memory than binary and datalog files if the data is not originally in text form, such as graph or chart data, because the ASCII representation of data usually is larger than the data itself. For example, you can store the number –123.4567 in 4 bytes as a single-precision, floating-point number. However, its ASCII representation takes 9 bytes, one for each character.

In addition, it is difficult to randomly access numeric data in text files. Although each character in a string takes up exactly 1 byte of space, the space required to express a number as text typically is not fixed. To find the ninth number in a text file, LabVIEW must first read and convert the preceding eight numbers.

You might lose precision if you store numeric data in text files. Computers store numeric data as binary data, and typically you write numeric data to a text file in decimal notation. Loss of precision is not an issue with binary files.

When to Use Binary Files

Storing binary data, such as an integer, uses a fixed number of bytes on disk. For example, storing any number from 0 to 4 billion in binary format, such as 1, 1,000, or 1,000,000, takes up 4 bytes for each number.

Use binary files to save numeric data and to access specific numbers from a file or randomly access numbers from a file. Binary files are machine readable only, unlike text files, which are human readable. Binary files are the most compact and fastest format for storing data. You can use multiple data types in binary files, but it is uncommon.

Binary files are more efficient because they use less disk space and because you do not need to convert data to and from a text representation when you store and retrieve data. A binary file can represent 256 values in 1 byte of disk space. Often, binary files contain a byte-for-byte image of the data as it was stored in memory, except for cases like extended and complex numeric values. When the file contains a byte-for-byte image of the data as it was stored in memory, reading the file is faster because conversion is not necessary.

Datalog Files

A specific type of binary file, known as a datalog file, is the easiest method for logging cluster data to file. Datalog files store arrays of clusters in a binary representation. Datalog files provide efficient storage and random access, however, the storage format for datalog files is complex, and therefore they are difficult to access in any environment except LabVIEW. Furthermore, in order to access the contents of a datalog file, you must know the contents of the cluster type stored in the file. If you lose the definition of the cluster, the file becomes very difficult to decode. For this reason, datalog files are not recommended for sharing data with others or for storing data in large organizations where you could lose or misplace the cluster definition.

When to Use TDMS Files

To reduce the need to design and maintain your own data file format, National Instruments has created a flexible data model called Technical Data Management Streaming, which is natively accessible through LabVIEW, LabWindows™/CVI™, and DIAdem, and is portable to other applications such as Microsoft Excel. The TDMS data model offers several unique benefits such as the ability to scale your project requirements and easily attach descriptive information to your measurements while streaming your data to disk.

The TDMS file format consists of two files—a `.tdms` file and a `.tdms_index` file. The `.tdms` file is a binary file that contains data and stores properties about that data. The `.tdms_index` file is a binary index file that speeds up access while reading and provides consolidated information on all the attributes and pointers in the TDMS file. All the information in the `.tdms_index` file is also contained in the `.tdms` file. For this reason, the `.tdms_index` file can be automatically regenerated from the `.tdms` file. Therefore, when you distribute TDMS files, you only need to distribute the `.tdms` file. The internal structure of the TDMS file

format is publicly documented, so it is possible to create third-party programs to write and read TDMS files. In addition, there is a TDM Excel Add-in Tool you can install to load `.tdms` files into Microsoft Excel.

Use TDMS files to store test or measurement data, especially when the data consists of one or more arrays. TDMS files are most useful when storing arrays of simple data types such as numbers, strings, or Boolean data. TDMS files cannot store arrays of clusters directly. If your data is stored in arrays of clusters, use another file format, such as binary, or break the cluster up into channels and use the structure of the TDMS file to organize them logically.

Use TDMS files to create a structure for your data. Data within a file is organized into channels. You can also organize channels into channel groups. A file can contain multiple channel groups. Well-grouped data simplifies viewing and analysis and can reduce the time required to search for a particular piece of data.

Use TDMS files when you want to store additional information about your data. For example, you might want to record the following information:

- Type of tests or measurements
- Operator or tester name
- Serial numbers
- Unit Under Test (UUT) numbers for the device tested
- Time of the test
- Conditions under which the test or measurement was conducted

B. Binary Files

Although all file I/O methods eventually create binary files, you can directly interact with a binary file by using the Binary File functions. The following list describes the common functions that interact with binary files.



Open/Create/Replace File—Opens a reference to a new or existing file for binary files as it does for ASCII Files.



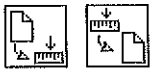
Write to Binary File—Writes binary data to a file. The function works much like the Write to Text File function, but can accept most data types.



Read from Binary File—Reads binary data starting at its current file position. You must specify to the function the data type to read. Use this function to access a single data element or wire a value to the count input. This causes the function to return an array of the specified data type.



Get File Size—Returns the size of the file in bytes. Use this function in combination with the Read from Binary File function when you want to read all of a binary file. Remember that if you are reading data elements that are larger than a byte you must adjust the count to read.



Get/Set File Position—These functions get and set the location in the file where reads and writes occur. Use these functions for random file access.



Close File—Closes an open reference to a file.

Figure 6-1 shows an example that writes an array of doubles to a binary file. Refer to the *Arrays* section of this lesson for more information about the **Prepend array or string size?** option.

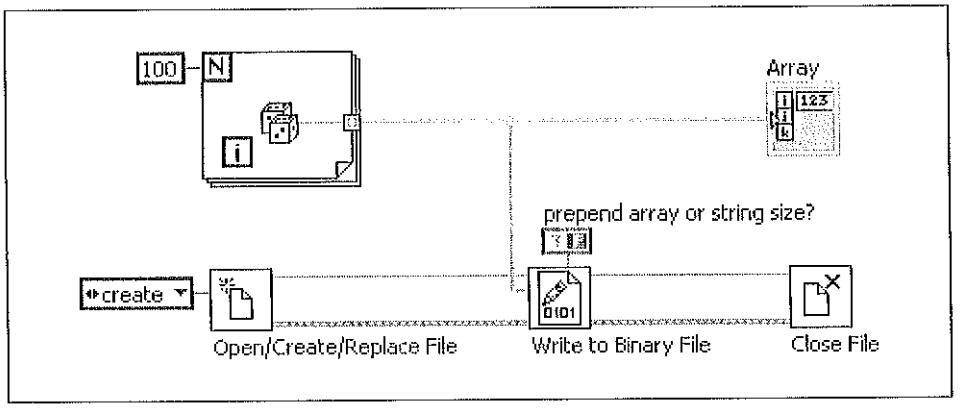


Figure 6-1. Writing a Binary File

Binary Representation

Each LabVIEW data type is represented in a specified way when written to a binary file. This section discusses the representation of each type and important issues when dealing with the binary representation of each type.



Tip A bit is a single binary value. Represented by a 1 or a 0, each bit is either on or off. A byte is a series of 8 bits.

Boolean Values

LabVIEW represents Boolean values as 8-bit values in a binary file. A value of all zeroes represents False. Any other value represents True. This divides files into byte-sized chunks and simplifies reading and processing files. To efficiently store Boolean values, convert a series of Boolean values into an integer using the Boolean Array To Number function. Figure 6-2 shows two methods for writing six Boolean values to a binary file.

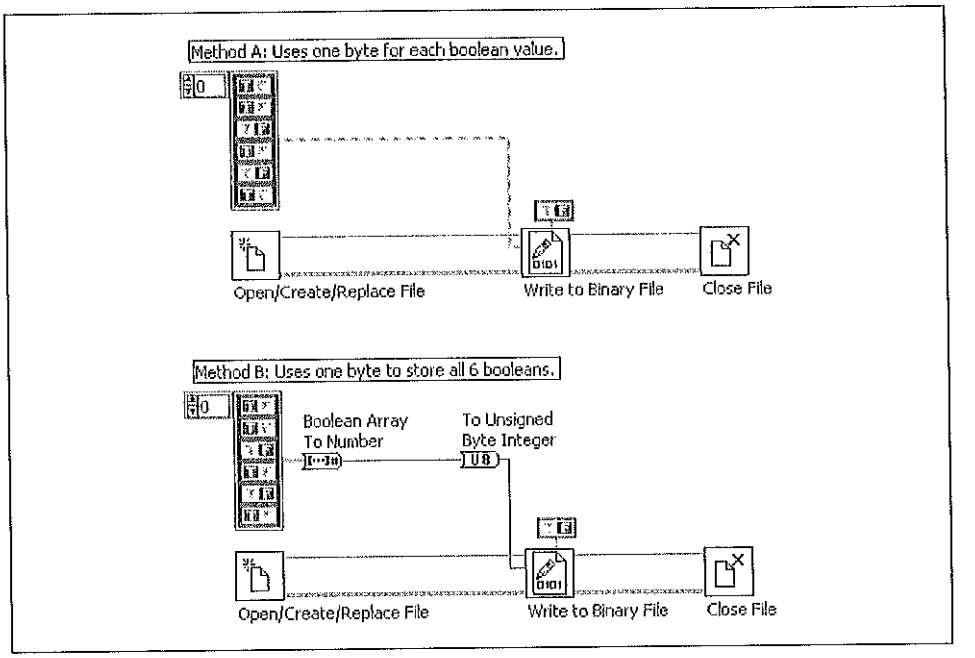


Figure 6-2. Writing Boolean Values to a Binary File

Table 6-1 displays a binary representation of the file contents resulting from running the programs in Figure 6-2. Notice that Method B is a more efficient storage method.

Table 6-1. Results of Figure 6-2

Method A	00000001 00000001 00000000 00000001 00000000 00000001
Method B	00101011

8-bit Integers

Unsigned 8-bit integers (U8) directly correspond to bytes written to the file. When you must write values of various types to a binary file, convert each type into an array of U8s using the Boolean Array To Number, String to Byte Array, Split Number, and Type Cast functions. Then, you can concatenate the various arrays of U8s and write the resulting array to a file. This process is unnecessary when you write a binary file that contains only one type of data.

Table 6-2. U8 Representation

Binary Value	U8 Value
00000000	0
00000001	1

Table 6-2. U8 Representation (Continued)

Binary Value	U8 Value
00000010	2
11111111	255

Other Integers

Multi-byte integers are broken into separate bytes and are stored in files in either little-endian or big-endian byte order. Using the Write to Binary File VI, you can choose whether you store your data in little-endian or big-endian format.

Little-endian byte order stores the least significant byte first, and the most significant byte last. Macintosh computers traditionally used little-endian order and often internally represents data in LabVIEW.

Big-endian order stores the most significant byte first, and the least significant byte last. Most Windows programs use big-endian when storing data to files.

Table 6-3. Integer Representations

U32 Value	Little-endian Value	Big-endian Value
0	00000000 00000000 00000000 00000000	00000000 00000000 00000000 00000000
1	00000001 00000000 00000000 00000000	00000000 00000000 00000000 00000001
255	11111111 00000000 00000000 00000000	00000000 00000000 00000000 11111111
65535	11111111 11111111 00000000 00000000	00000000 00000000 11111111 11111111
4,294,967,295	11111111 11111111 11111111 11111111	11111111 11111111 11111111 11111111

Floating-Point Numbers

Floating point numbers are stored as described by the IEEE 754 Standard for Binary Floating-Point Arithmetic. Single-precision numerics use 32-bits each and double-precision numerics use 64-bits each. The length of extended-precision numerics depends on the operating system.

Strings

Strings are stored as a series of unsigned 8-bit integers, each of which is a value in the ASCII Character Code Equivalents Table. This means that there is no difference between writing strings with the Binary File functions and writing them with the Text File functions.

Arrays

Arrays are represented as a sequential list of each of their elements. The actual representation of each element depends on the element type. When you store an array to a file you have the option of preceding the array with a header. A header contains a 4-byte integer representing the size of each dimension. Therefore, a 2D array with a header contains two integers, followed by the data for the array. Figure 6-3 shows an example of writing a 2D array of 8-bit integers to a file with a header. The **prepend array or string size?** input of the Write to Binary File function enables the header. Notice that the default value of this terminal is True. Therefore, headers are added to all binary files by default.

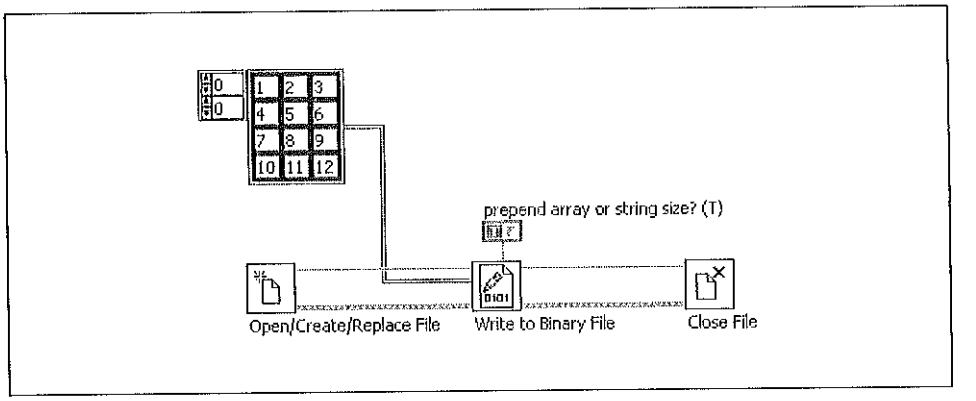


Figure 6-3. Writing a 2D Array of Unsigned Integers to a File with a Header

Table 6-4 shows the layout of the file that the code in Figure 6-3 generates. Notice that the headers are represented as 32-bit integers even though the data is 8-bit integers.

Table 6-4. Example Array Representation In Binary File

4	3	1	2	3	4	5	6	7	8	9	10	11	12

Clusters

Datalog files best represent clusters in binary files. Refer to the *Datalog Files* section for more information.

Sequential vs. Random Access

When reading a binary file, there are two methods of accessing data. The first is to read each item in order, starting at the beginning of a file. This is called sequential access and works similar to reading an ASCII file. The second is to access data at an arbitrary point within the file for random access. For example, if you know that a binary file contains a 1D array of 32-bit integers that was written with a header and you want to access the tenth item in the array, you could calculate the offset in bytes of that element in the file and then read only that element. In this example, the element has an offset of 4 (the header) + 10 (the array index) × 4 (the number of bytes in an I32) = 44.

Sequential Access

To sequentially access all the data in a file, you can call the Get File Size function and use the result to calculate the number of items in the file, based on the size of each item and the layout of the file. You can then wire the number of items to the count terminal of the Read Binary function.

Figure 6-4 shows an example of this method.

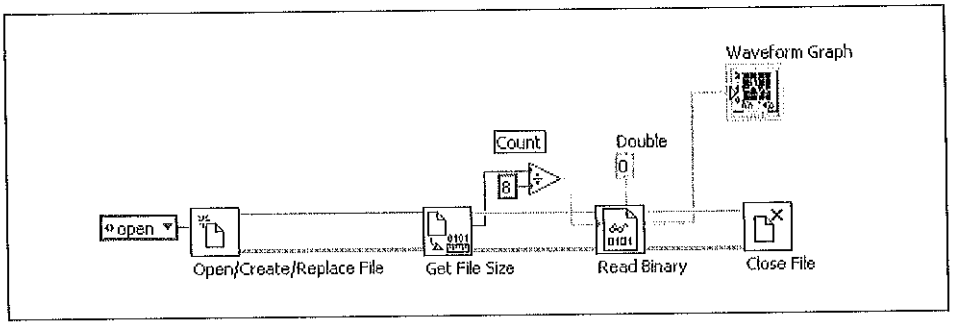


Figure 6-4. Sequentially Reading an Entire File

Alternately, you can sequentially access the file one item at a time by repeatedly calling the Read Binary function with the default count of 1. Each read operation updates the position within the file so that you read a new item each time read is called. When using this technique to access data you can check for the **End of File** error after calling the Read Binary function or calculate the number of reads necessary to reach the end of the file by using the Get File Size function.

Random Access

To randomly access a binary file, use the Set File Position function to set the read offset to the point in the file you want to begin reading. Notice that the offset is in bytes. Therefore, you must calculate the offset based on the layout of the file. In Figure 6-5, the VI returns the array item with the index specified, assuming that the file was written as a binary array of

double-precision numerics with no header, like the one written by the example in Figure 6-1.

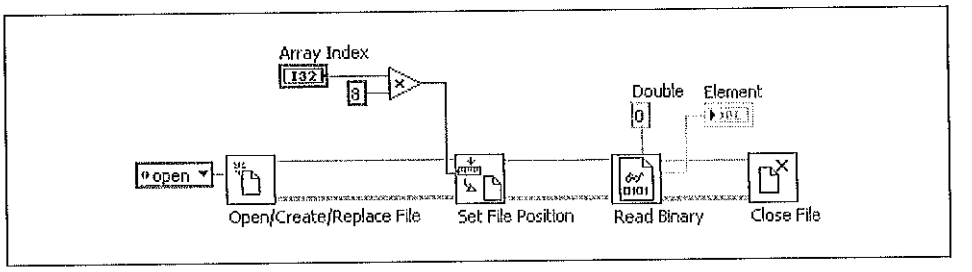


Figure 6-5. Randomly Accessing a Binary File

Datalog Files

Datalog files are designed for storing a list of records to a file. Each record is represented by a cluster, and can contain multiple pieces of data with any data type. Datalog files are binary files, however, they use a different API than other binary files. The Datalog functions allow you to read and write arrays of clusters to and from datalog files.

When you open a datalog file for either reading or writing, you must specify the record type used by the file. To do this, wire a cluster of the appropriate type to the Open/Create/Replace Datalog function. After the file is open, you program datalog files like any other binary file. Random access is available, although offsets are specified in records instead of bytes.

Figure 6-6 shows an example of writing a datalog file. Notice that the cluster bundles the data and opens the datalog file.

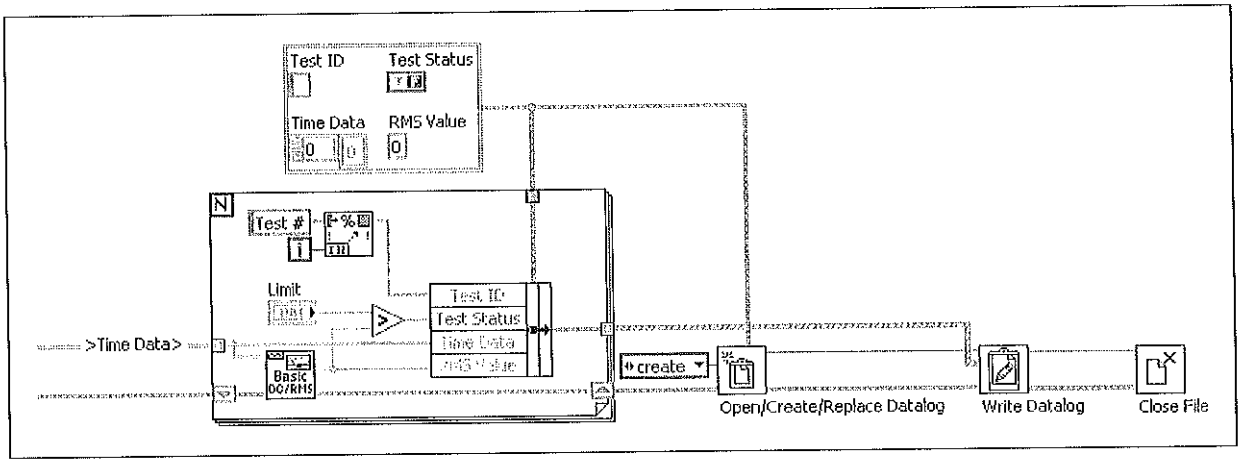


Figure 6-6. Writing a Datalog File

Figure 6-7 shows an example of randomly accessing a datalog file. Notice that the Record Definition cluster matches the cluster used to write the file. If the **record type** wired to the Open/Create/Replace Datalog function does not match the records in the file being opened, an error occurs.

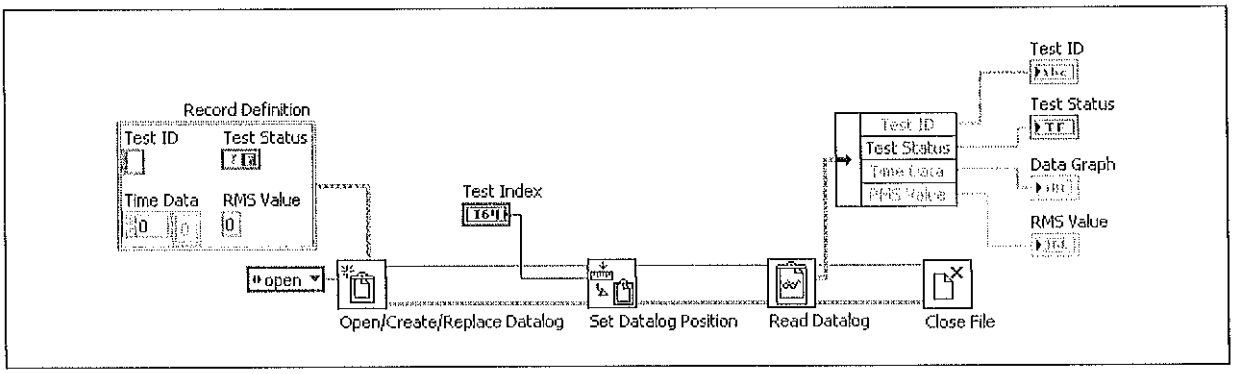


Figure 6-7. Reading a Datalog File

Instead of using random access, you can read an entire datalog file by wiring the output of the Get Number of Records function to the **count** input of the Read Datalog function.

Exercise 6-1 Bitmap File Writer VI

Goal

Use Binary File I/O to write to a file using a specified format.

Scenario

Write a file storage routine to store image files for an existing LabVIEW-based Drawing Pad. The drawing pad VI returns a drawing as a 2D array of red, green, and blue (RGB) values. Save the data as a 24-bit bitmap file.

Design

You can use binary file I/O to write or read data in any file format, assuming that you have a specification that tells you the file layout for that format. The following section describes the format for a 24-bit Windows bitmap file.

24-bit Bitmap File Layout

Bitmaps (.bmp) files are a format for storing image data. Bitmaps come in multiple varieties, with differences such as the number of bits used to represent a pixel and the level of image compression used. The easiest type of bitmap file to understand and create is a 24-bit uncompressed bitmap file. Table 6-5 describes the format of a 24-bit uncompressed bitmap file.

Table 6-5. 24-Bit Bitmap File Layout

Section Name	Size (bytes)	Notes
BITMAPFILEHEADER	14	Data is provided. Write data to the file.
BITMAPINFOHEADER	40	Data is provided. Write data to the file.
Image Data	3 × Number of Pixels	Stored in BGR order. Image is inverted.

BITMAPFILEHEADER—Contains information about the file such as the file type and file size. A subVI calculates the data for this segment. The subVI returns an array of U8 numerics that you must write to the file.

BITMAPINFOHEADER—Contains information about the image such as the height, width, compression level, and number of bits per pixel. A subVI calculates the data for this segment. The subVI returns an array of U8 numerics that you must write to the file.

Image Data—For a 24-bit image, three bytes represent each pixel in the image. The three bytes represent the RGB values for the pixel and are stored in reverse order, BGR. The pixel array you are given is a 2D array of clusters. Each cluster has an RGB value in it.

The rows in the Image Data are also stored from bottom to top, and the first pixel stored is the lower left corner of the image. For this exercise, the pixel array you are given is already vertically inverted so you can write the pixels in the order they are given.

Note The data for each row of the file must have a number of bytes that is divisible by four. You can pad each row with zeroes to bring the number of bytes to a multiple of four. For this exercise, all of the pictures returned from the drawing pad have a width that is a multiple of four.

Inputs and Outputs

The main VI for this program contains no inputs or outputs. Dialog boxes control all user interaction. The Drawing Pad VI displays a dialog box that allows the user to draw a picture. When the user clicks **Save**, the application prompts the user to enter a save location by using a File Dialog VI.

Program Flow

1. Call the Drawing Pad VI to create a picture.

Note The Drawing Pad VI returns the picture as a 2D array of clusters, each containing RGB values. The 2D array has a width that is a multiple of four and is inverted in preparation for writing to file.

2. Display a file dialog box to the user to select a location and filename and open the selected file for writing.
3. Call the BITMAPFILEHEADER VI and pass the dimensions of the pixel array to it.
4. Write the 1D array of unsigned integers returned by the BITMAPFILEHEADER VI to the open file.

Note Disable the **prepend array or string size** option when you call the Write to Binary File VI, otherwise LabVIEW inserts the array size at the beginning of the data, which invalidates the file layout.

5. Call the BITMAPINFOHEADER VI and pass the dimensions of the pixel array to it.
6. Write the 1D array of unsigned integers returned by the BITMAPINFOHEADER VI to the open file.

7. Process each pixel in the array by using a pair of For Loops to remove the values from the cluster in BGR order and build them into an array.

Note Use a 3D array to store the processed pixel data, because it allows you to use For Loop auto-indexing and simplify the program. The number of dimensions in the array is not important, because the File I/O functions automatically reformat the array to write to the file.

8. Write the processed pixel array to the open file.
9. Close the file and handle any errors.

Implementation

1. Display the drawing pad.
   - ❑ Open a blank VI.
   - ❑ Save the VI as `Bitmap File Writer.vi` in the `<Exercises>\LabVIEW Basics II\Bitmap File Writer` directory.
   - ❑ Open the block diagram.
   - ❑ Add the Drawing Pad VI, located in the `<Exercises>\LabVIEW Basics II\Bitmap File Writer` directory, to the block diagram.
   - ❑ Right-click the Drawing Pad VI and select **SubVI Node Setup** from the shortcut menu.
   - ❑ Check the **Show Front Panel when called** and **Close afterwards if originally closed** boxes.
   - ❑ Click **OK** to exit the SubVI Node Setup dialog box.

Note The subVI node setup lets you specify how to call a subVI. Checking the **Show Front Panel when called** box instructs the VI to show its front panel, even if the VI properties would otherwise prevent it from doing so.

   - ❑ Run the VI and observe the drawing pad. Click the **Save** button to exit. Currently, the program ends when you click **Save**, because you have not yet implemented the file I/O.

2. Open a new binary file.



- ❑ Add a File Dialog Express VI to the block diagram.
- ❑ Click **OK** to exit the Configure File Dialog dialog box. The default values allow the user to select a single new or existing file.
- ❑ Configure the File Dialog Express VI to show the **selected path**, **error out**, **error in**, **prompt**, **pattern label**, and **pattern (all files)** terminals by expanding the node and then clicking each terminal to select an input or output.
- ❑ Right-click the **prompt** input of the File Dialog Express VI and select **Create»Constant** from the shortcut menu.
- ❑ Enter `Select File to Save` in the string constant.
- ❑ Right-click the **pattern label** input of the File Dialog Express VI and select **Create»Constant** from the shortcut menu.
- ❑ Enter `Bitmap Files` in the string constant.
- ❑ Right-click the **pattern (all files)** input of the File Dialog Express VI and select **Create»Constant** from the shortcut menu.
- ❑ Enter `*.bmp` in the string constant.



- ❑ Add an Open/Create/Replace File function to the block diagram.
- ❑ Right-click the **operation** input of the Open/Create/Replace File function and select **Create»Constant** from the shortcut menu.
- ❑ Select replace or create as the value of the constant.
- ❑ Wire the block diagram as shown in Figure 6-8.

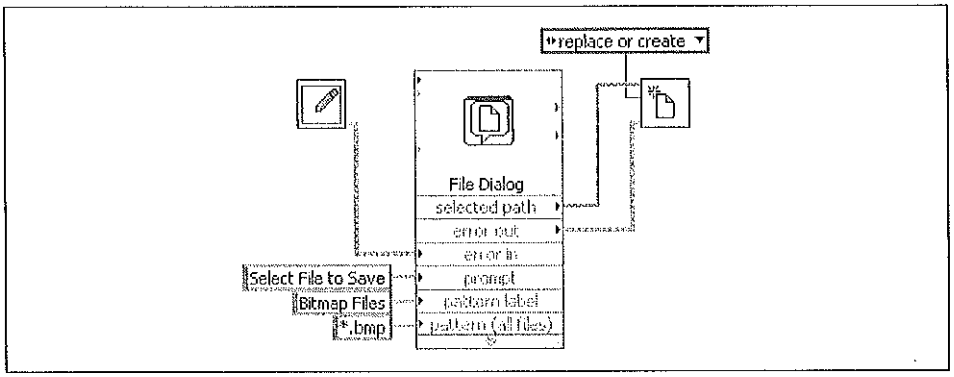


Figure 6-8. Open Binary File

3. Create bitmap headers.

   ❑ Add the BITMAPFILEHEADER VI located in the `<Exercises>\LabVIEW Basics II\Bitmap File Writer` directory to the block diagram.

   ❑ Add the BITMAPINFOHEADER VI located in the `<Exercises>\LabVIEW Basics II\Bitmap File Writer` directory to the block diagram.



   ❑ Add an Array Size function to the block diagram.



   ❑ Add two Write to Binary File functions to the block diagram.

   ❑ Right-click the **prepend array or string size** terminal of each Write to Binary File VI and select **Create»Constant** from the shortcut menu.

   ❑ Set the constant values to False.

   ❑ Wire the block diagram as shown in Figure 6-9.

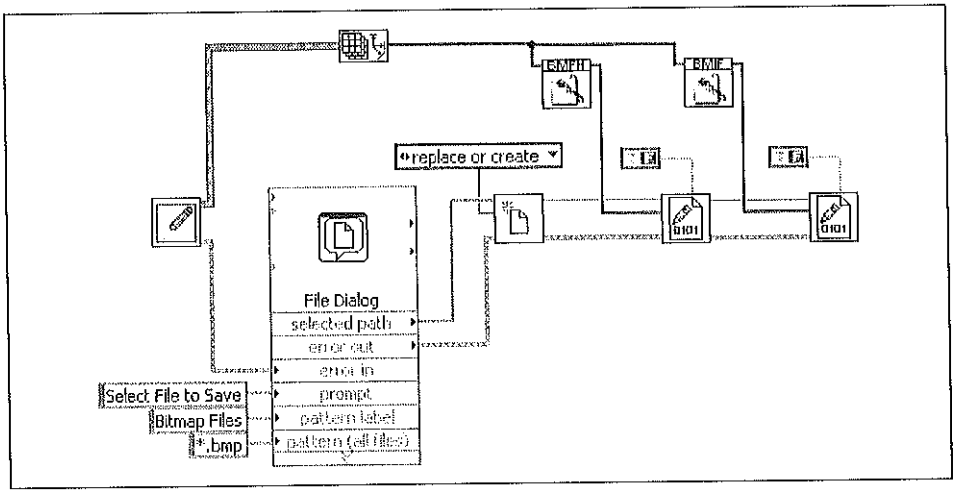


Figure 6-9. Write Bitmap Headers

4. Write image data.



   ❑ Add a For Loop to the block diagram.

   ❑ Add a second For Loop inside the first For Loop.



   ❑ Add an Unbundle by Name function to the For Loops.



   ❑ Add a Build Array function to the For Loops.

   ❑ Wire the Image Data array through the For Loop borders to the Unbundle By Name function.

- ❑ Expand the Unbundle by Name function so that three elements are shown.
- ❑ Choose Blue, Green, and Red, in order, for the elements.

Note The bitmap file definition specifies that pixels must be stored in blue, green, red order. Storing the pixels in another order causes the colors in your image to be incorrect.

- ❑ Add a Write to Binary File function to the block diagram.
- ❑ Right-click the **prepend array or string size** input of the Write to Binary File function and select **Create»Constant** from the shortcut menu.
- ❑ Set the constant value to False.

5. Close the file and handle the errors.

- ❑ Add a Close File function to the block diagram.
- ❑ Add a Simple Error Handler VI to the block diagram.
- ❑ Wire the block diagram as shown in Figure 6-10.

6. Save the VI.

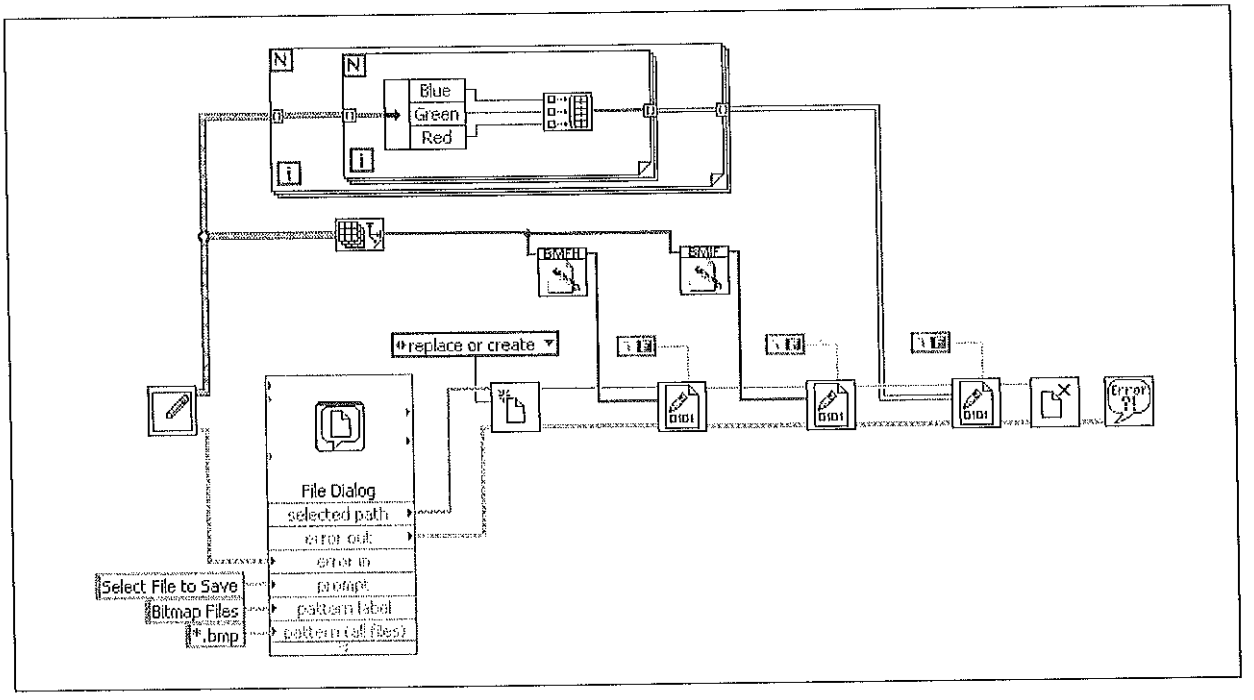


Figure 6-10. Complete Block Diagram

Test

1. Run the VI.
   - ❑ Switch to the VI front panel window.
   - ❑ Run the VI.
   - ❑ Draw a picture in the drawing pad.
   - ❑ Click **Save**.
   - ❑ Select `<Exercises>\LabVIEW Basics II\Bitmap File Writer\My Image.bmp` as the file to save.
2. Open the image in an image viewer.
   - ❑ Open the `<Exercises>\LabVIEW Basics II\Bitmap File Writer` directory in Windows Explorer.
   - ❑ Double-click the image to open it in your default image viewer. Verify that the image displayed is the picture you created.
3. Close the VI.

End of Exercise 6-1

C. TDMS Files

Creating TDMS Files

In LabVIEW, you can create TDMS Files in two ways. Use the Write to Measurement File Express VI and Read from Measurement File Express VI or the TDM Streaming API.

With the Express VIs you can quickly save and retrieve data from the TDM format. Figure 6-11 shows the configuration dialog box for the Write to Measurement File Express VI. Notice that you can choose to create a LVM or TDMS file type. However, these Express VIs give you little control over your data grouping and properties and do not allow you to use some of the features that make TDMS files useful, such as defining channel names and channel group names.

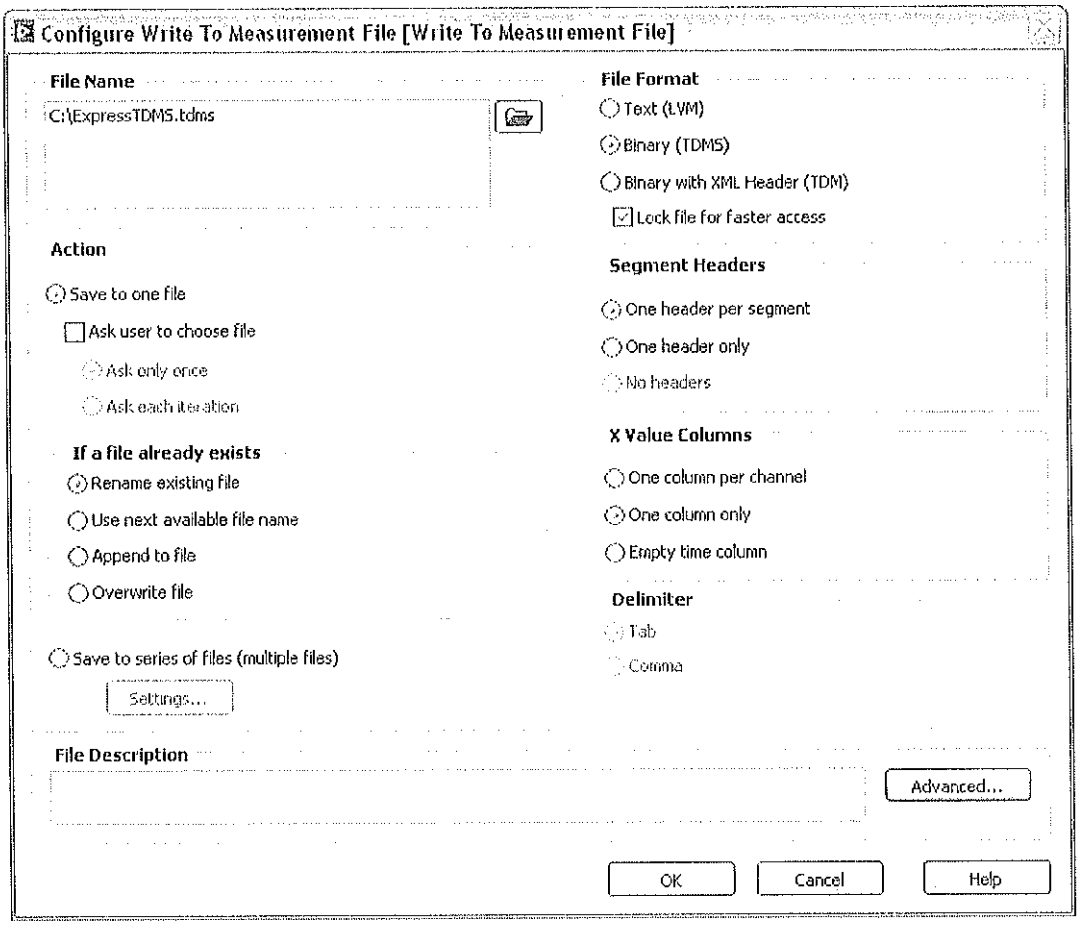


Figure 6-11. Creating a TDMS with Write to Measurement File Express VI

To gain access to the full capabilities of TDMS files, use the TDM Streaming functions. Use the TDM Streaming functions to attach descriptive information to your data and quickly save and retrieve data.

Some of the commonly used TDM Streaming functions are described in the *TDMS API* section of this lesson.

Data Hierarchy

Use TDMS files to organize your data in channels and in channel groups.

A channel stores measurement signals or raw data in a TDMS file. The signal is an array of measurement data. Each channel also can have properties that describe the data. The data stored in the signal is stored as binary data on disk to conserve disk space and efficiency.

A channel group is a segment of a TDMS file that contains properties to store information as well as one or more channels. You can use channel groups to organize your data and to store information that applies to multiple channels.

TDMS files each contain as many channel group and channel objects as you want. Each of the objects in a file has properties associated with it, which creates three levels of properties you can use to store data. For example, test conditions are stored at the file level. UUT information is stored at the channel or channel group level. Storing plenty of information about your tests or measurements can make analysis easier.

TDMS API

The following describes some of the most commonly used TDM Streaming VI and functions.



- **TDMS Open**—Opens a reference to a TDMS file for reading or writing.



- **TDMS Write**—Streams data to the specified TDMS file. It also allows you to create channels and channel groups within your file.



- **TDMS Read**—Reads the specified TDMS file and returns data from the specified channel and/or channel group.



- **TDMS Set Properties**—Sets the properties of the specified TDMS file, channel group, or channel.



- **TDMS Get Properties**—Returns the properties of the specified TDMS file, channel group, or channel.



- **TDMS Close**—Closes a reference to a TDMS File. Notice that you only must close the file reference, any references that you acquire to channels and channel groups close automatically when you close the file reference.



- **TDMS File Viewer**—Opens the specified TDMS file and presents the file data in the TDMS File Viewer dialog box.



- **TDMS List Contents**—Provides a list of group and channel names contained within the specified TDMS file.



- **TDMS Defragment**—Defragments the file data in the specified TDMS data. Use this function to clean up your TDMS data when it becomes cluttered and to increase performance.



- **TDMS Flush**—Flushes the system memory of all TDMS data to maintain data security.

TDMS Programming

Writing a TDMS File

Figure 6-12 shows the simplest form of writing measurement data with the TDMS API. This example writes data to the channel Main Channel in the channel group Main Group.

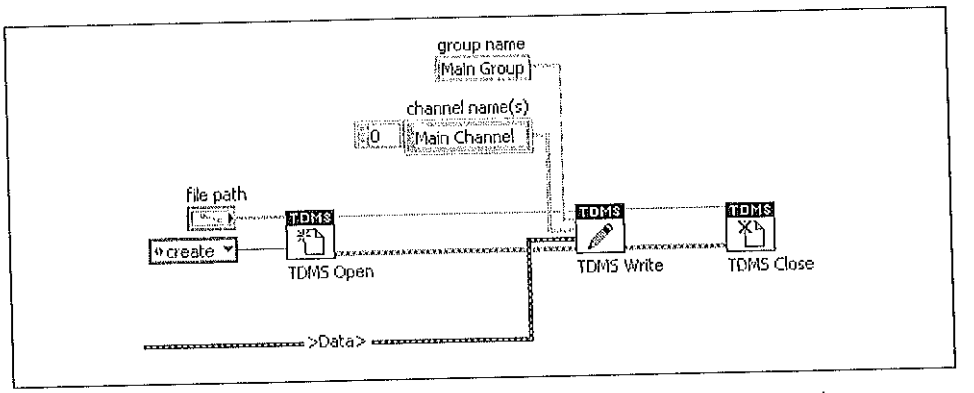


Figure 6-12. Write Data to a TDMS File at the Channel Level

Reading a TDMS File

Figure 6-13 shows the simplest form of reading data using the TDMS API. This example reads all the data in the channel Main Channel from channel group Main Group and displays it in the Channel Data waveform graph. Next, the example reads data from all the channels in the channel group Main Group and displays it in the Group Data waveform graph.

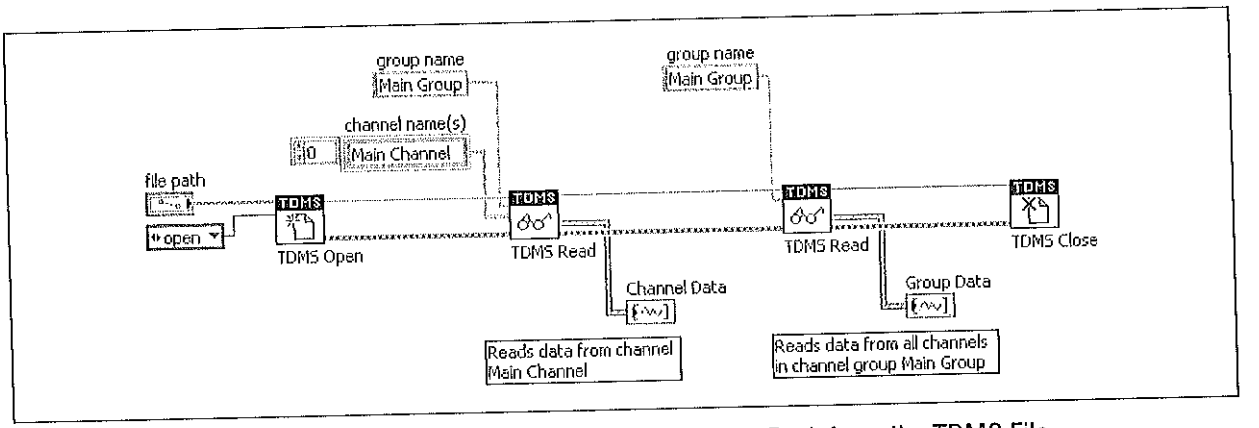


Figure 6-13. Read Data Back from the TDMS File

TDMS Write Custom Properties

The TDMS data model automatically creates certain properties when some data types are written. However, in many cases you may want to create a property specific to your particular needs, such as UUT, serial number, and test temperature. This task can be accomplished using the TDMS Set Properties function with which you can write properties at the file, group, or channel level.

The file level of a property determines which input terminals need to be wired. For example, to write a group property, only the group name input must be wired. To write a channel property, both the group name and channel name inputs must be wired. But, to write a file property, the group name and channel names inputs should be left unwired. Figure 6-14 illustrates these examples.

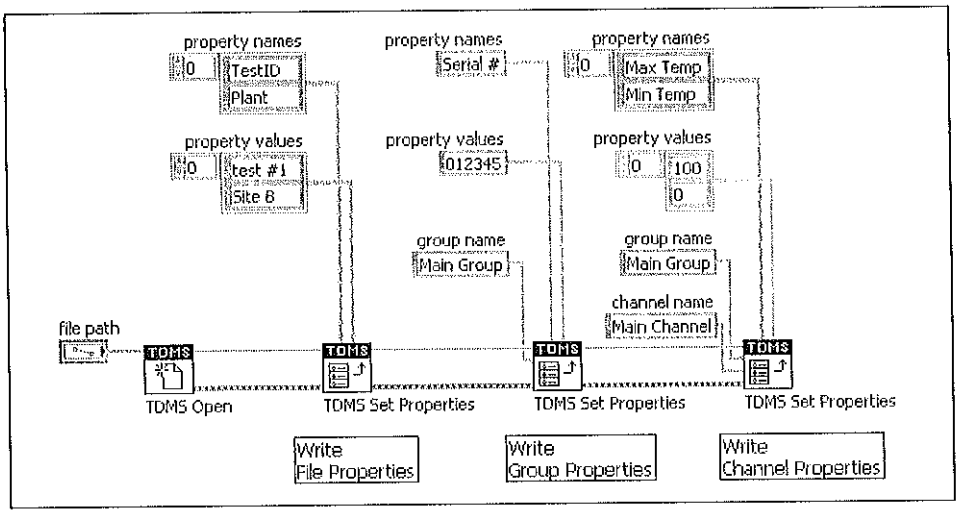


Figure 6-14. Write Custom Properties at Three Different Levels before Writing Data to the File

With the TDMS Set Properties function, you can specify an individual property by itself or specify many properties by using arrays. Figure 6-14 shows two properties specified at the file level (TestID and Plant). You could expand this to specify many more properties by increasing the size of the array. Arrays are not necessary if only a single property, such as a serial number, is written.

Property values can also be different data types. In Figure 6-14, string properties are written at the file and group level. But at the channel level, two numeric properties are written to specify the minimum and maximum temperature.

TDMS Read Custom Properties

When a TDMS file has been written, the properties can be read back into LabVIEW using TDMS Get Properties function. Properties are returned only for the level specified by the wiring of the group name and channel name inputs. This process is similar to writing the properties, as it is shown in Figure 6-15.

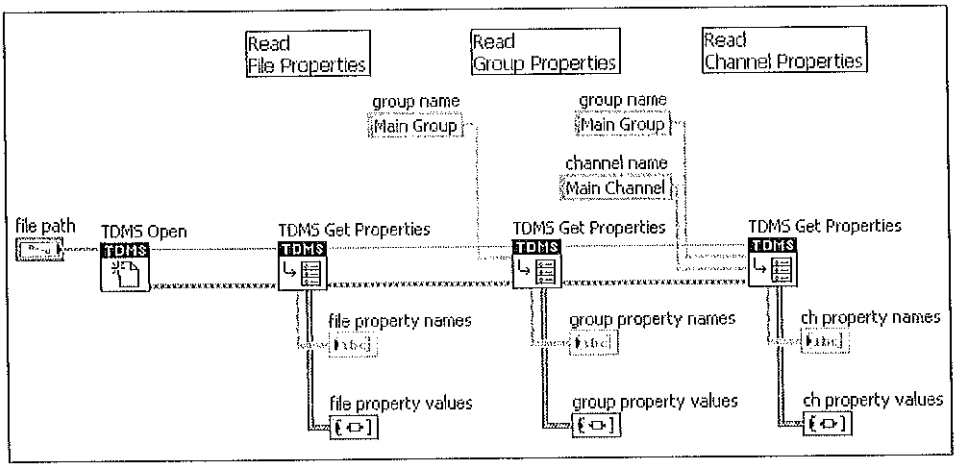


Figure 6-15. Read TDMS Properties from Three Different Levels

In this configuration, the property values are returned as an array of Variant data because the data could be a string, double, Boolean, or another data type. The data can be displayed on the front panel as a Variant or it can be converted in LabVIEW to the appropriate data type. If a property name and its data type are known, they can be wired as inputs to TDMS Get Properties function and read directly with the correct data type.

TDMS File Viewer

Use the TDMS File Viewer VI when developing a TDMS application to automatically see everything that has been written to a TDMS file by making a single VI call. The TDMS File Viewer function is flexible and can read complex TDMS files. The TDMS File Viewer VI is included with the TDMS API, so it can be easily placed in a program. Place the TDMS File Viewer VI after the file is closed to use it. The TDMS File Viewer launches another window in which you can view the data and properties inside the TDMS file.

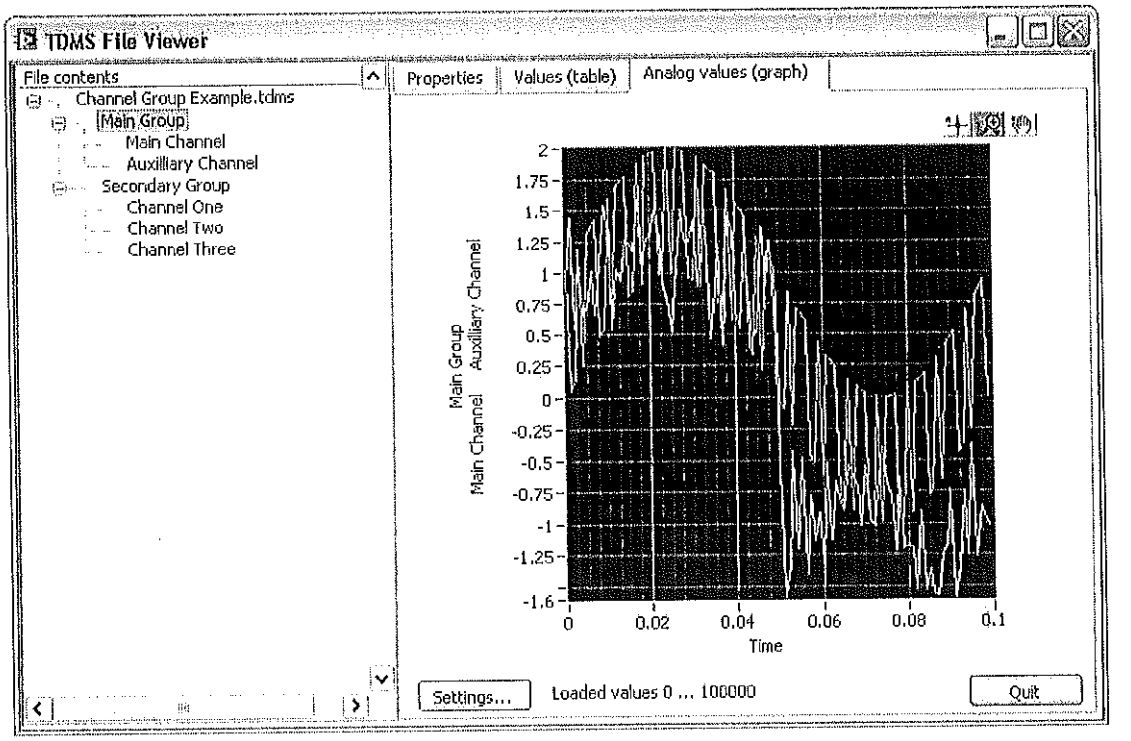


Figure 6-16. TDMS File Viewer

Grouping TDMS Data

Carefully consider the best way to group your data because the data grouping can have a significant impact on the execution speed and implementation complexity of writes and reads. Consider the original format of your data and how you want to process or view the data when choosing a grouping scheme.

One technique is to group data by the type of data. For example, you might put numeric data in one channel group and string data in another, or you might put time domain data in one group and frequency domain data in another. This makes it easy to compare the channels in a group, but can make it difficult to find two channels that are related to each other. Figure 6-17 shows an example of grouping by the type of data. In this example, the temperature data is placed in one group and the wind data is placed in another. Each group contains multiple channels of data. Notice that when grouping by data type you typically have a fixed number of groups, two in this case, and a dynamically determined number of channels. Exercise 6-2 is also an example of grouping by data type.

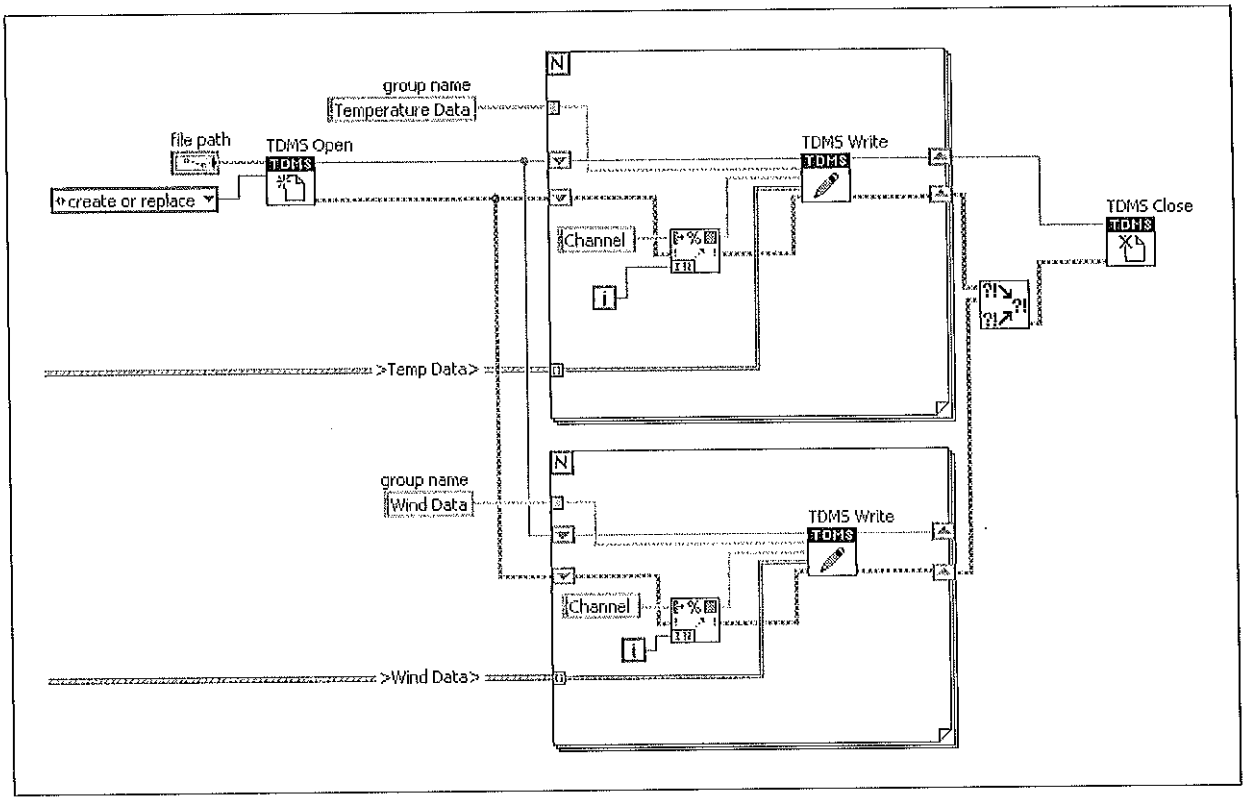


Figure 6-17. Grouping Data by Type

Another technique is to group related data. For example, you might put all the data that applies to a single UUT in one group. Grouping related data allows you to easily locate all the related data about a particular subject, but makes it harder to compare individual pieces of data among subjects. Relational grouping helps convert cluster-based storage to a TDMS format. You can store all the information from a given cluster in a channel group, with arrays in the cluster representing channels within the group, and scalar items in the cluster representing properties of the channel group. Figure 6-18 shows an example of relational grouping.

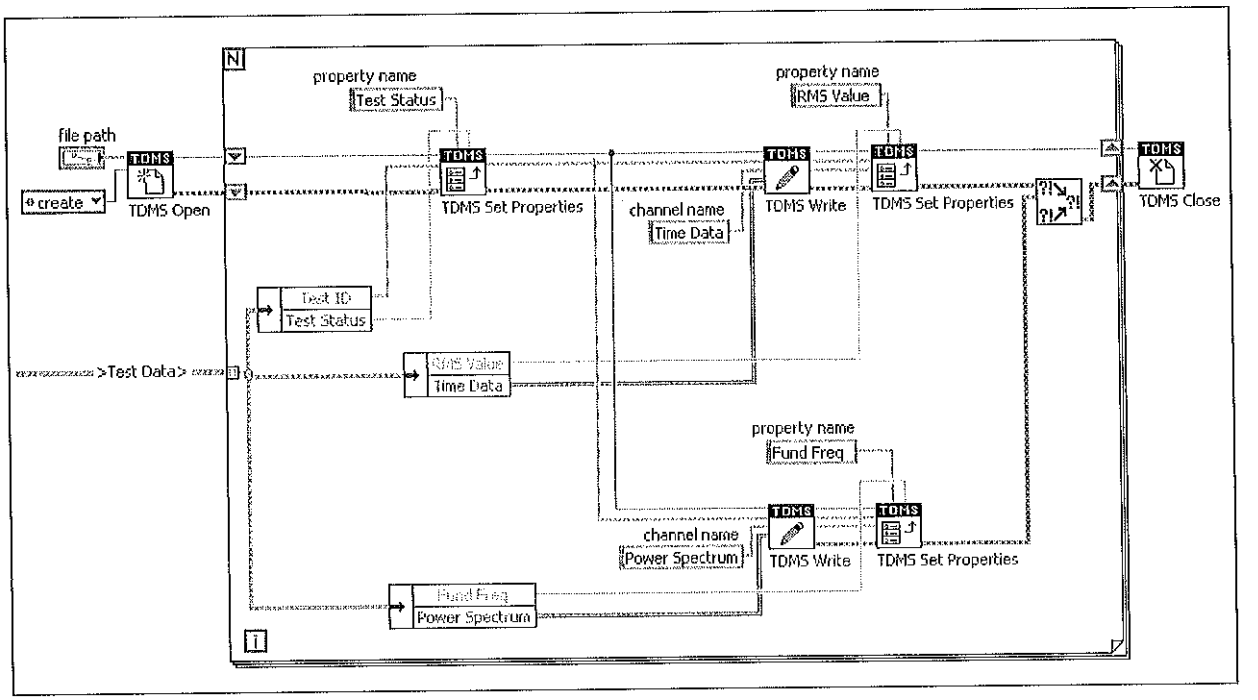


Figure 6-18. Grouping Related Data

Notice that the input data is an array of clusters, each of which contains multiple pieces of information about a test. Each test is stored as a separate channel group. Information that applies to the entire test, such as the Test ID and Test Status, is stored as properties of the channel group. Arrays of data, such as the time data and power spectrum, are stored in channels, and information which relates to the arrays of data, such as the RMS Value and Fundamental Frequency, are stored as properties of the channels. Relational data typically uses a fixed number of channels in a group, but the number of groups is dynamic.

Exercise 6-2 TDMS Reader VI

Goal

Log data to a TDMS file and read the same TDMS file to access information about a specific channel.

Scenario

You are given a TDMS Logger VI that generates measurement data for UUTs. The UUT measurement data consists of a time domain waveform and the power spectrum of a waveform.

Run the TDMS Logger VI that accepts an arbitrary number of UUTs, identified by serial numbers. The TDMS Logger VI retrieves the measurement data from the Generate Data VI, and logs the UUT data and additional properties to a TDMS file.

The TDMS file contains the author, timestamp, and two channel groups—Time Data and Power Spectrum Data. Each group contains a channel for each UUT. The serial number of the UUT names each channel and contains the matching signal data.

Saving data to a file serves no purpose unless you also can access the data. Create a reader VI to access data from the TDMS file you generated. The reader should return either time data or power spectrum data for a particular UUT serial number.

Design

TDMS File Reference Information

- File Level Information
  - Time Stamp—contains the current time.
  - Author—contains the test operator name, acquired through a front panel control.
  - The file contains two channel groups, one for time data and one for the power spectrum data.
- Channel Group Level Information
  - Name—contains `Time Data` or `Power Spectrum Data`. This identifies the channel group.
  - Each channel group should contain a channel for each UUT.
- Channel Level Information
  - Name—contains the UUT Serial Number, which associates the numeric data with a particular unit.

- Signal—contains an array of floating-point numeric data.
- Several other properties, such as the signal minimum and maximum will automatically be calculated and added to the file.

TDMS Reader Inputs and Outputs

Table 6-6. TDMS Reader VI Inputs and Outputs

Type	Name	Properties
File Path Control	`TDMS File Path`	Default Value = `C:\Exercises\LabVIEW Basics II\TDMS Logger\Test Data.TDMS`
String Control	`Serial Number`	—
Combo Box	`Data Set`	Item 1 = "Time Data" Item 2 = "Power Spectrum"
Waveform Graph Indicator	`Channel Data`	—
String Indicator	`Test Operator`	—
Time Stamp Indicator	`Time of Test`	—

Begin by opening the TDMS and reading the author and time stamp file properties. Then read the time data or power spectrum data for the specified UUT and display the data on the Channel Data waveform graph.

Implementation

1. Open `TDMS Logger.vi` in the `<Exercises>\LabVIEW Basics II\TDMS Logger` directory. This VI is pre-built for you as shown in Figure 6-19.

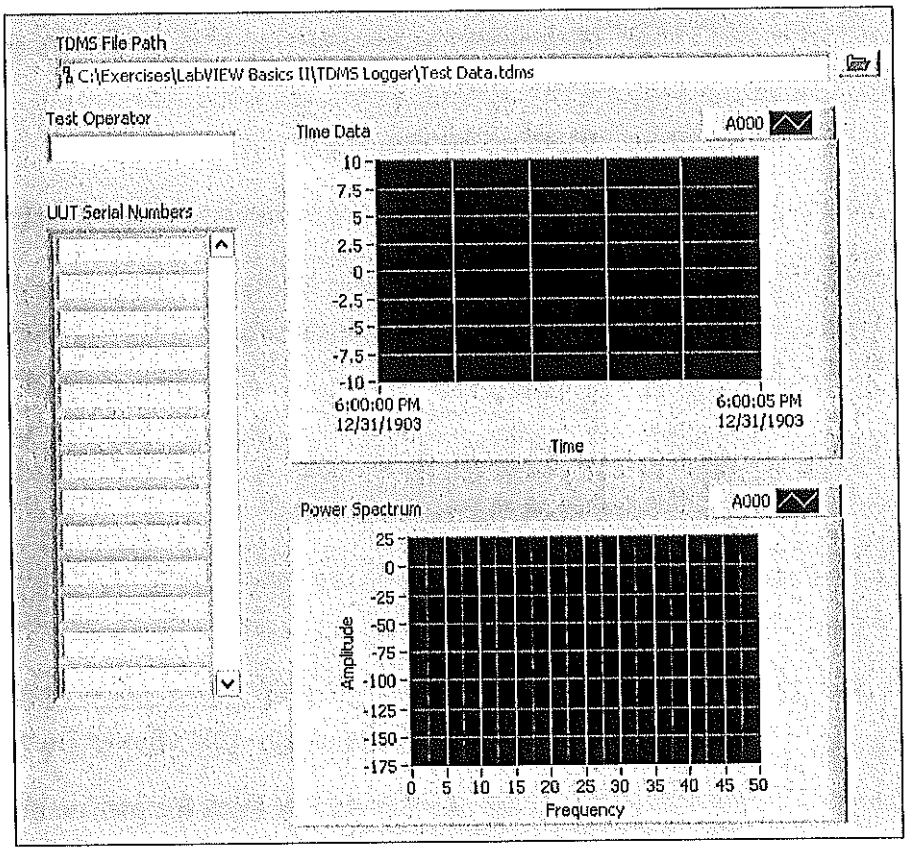


Figure 6-19. TDMS Logger Front Panel

2. Run the TDMS Logger VI.

   - ❑ Verify that the default value of the TDMS File Path control is `<Exercises>\LabVIEW Basics II\TDMS Logger\Test Data.tdms`.
   - ❑ Enter your name in the **Test Operator** field.
   - ❑ Enter `A001`, `A002`, and `A003` in the **UUT Serial Numbers** control.
   - ❑ Run and test the TDMS Logger VI. View the logged data in the TDMS File Viewer window. The graphs on the front panel of the TDMS Logger VI should also display a plot for each serial number you enter.
   - ❑ Click **Quit** to close the TDMS File Viewer window.

- ❑ Examine the block diagram.
- ❑ Close the TDMS Logger VI. Do not save any changes.

3. Create a blank VI.
4. Save the VI as `<Exercises>\LabVIEW Basics II\TDMS Reader\TDMS Reader.vi`.
5. Build the VI front panel.

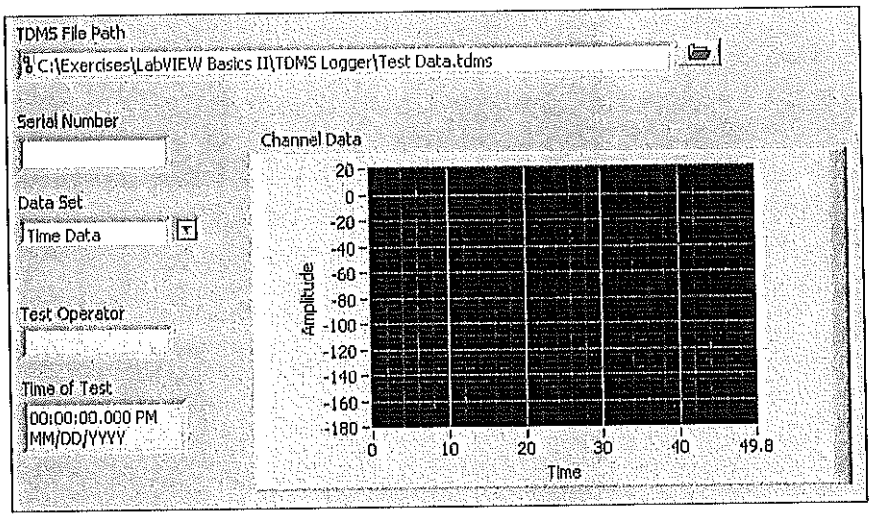


Figure 6-20. TDMS Reader Front Panel

- ❑ Create the **TDMS File Path** file path control with a default value of `<Exercises>\LabVIEW Basics II\TDMS Logger\Test Data.tdms`.
- ❑ Create the **Serial Number** string control.
- ❑ Create the **Time of Test** time stamp indicator.
- ❑ Create the **Test Operator** string control.
- ❑ Create the **Channel Data** waveform graph.
- ❑ Place a combo box control on the front panel. Label the combo box `Data Set`.
- ❑ Right-click the **Data Set** control and select **Edit Items** from the shortcut menu.
- ❑ Click **Insert**.
- ❑ Enter `Power Spectrum` in the **Items** list.

- ❑ Click **Insert**.
- ❑ Enter `Time Data` in the **Items** list.
- ❑ Remove the checkmark from the **Allow undefined values at run time** box.
- ❑ Click **OK**.
- ❑ Select **Time Data** from the drop-down menu of the **Data Set** control.
- ❑ Right-click the **Data Set** control and select **Data Operations»Make Current Value Default** from the shortcut menu.
- ❑ Arrange the front panel as shown in Figure 6-20.

6. Open the TDMS file.



- ❑ Add a TDMS Open function to the block diagram.
- ❑ Right-click the **operation** input of the TDMS Open function and select **Create»Constant** from the shortcut menu.
- ❑ Select **open (read-only)** as the value of the constant.



Note Opening a file with the open (read-only) option increases the speed of reads in the file. Also, it does not lock the file so that other programs can use it at the same time.

- ❑ Wire the TDMS File Path control to the **file path** input of the TDMS Open function.

7. Read the Author file property.



- ❑ Add a TDMS Get Properties function to the block diagram.
- ❑ Right-click the property name input of the TDMS Get Properties function and select **Create»Constant** from the shortcut menu.
- ❑ Enter `Author` in the string constant.



- ❑ Wire an empty string constant to the **data type** input of the TDMS Get Properties function.
- ❑ Wire the **property value** output of the TDMS Get Properties function to the **Test Operator** indicator.

8. Read the Time Stamp file property.



- ❑ Add a TDMS Get Properties function to the block diagram.
- ❑ Right-click the **property name** input of the TDMS Get Properties function and select **Create»Constant** from the shortcut menu.
- ❑ Enter `Time Stamp` in the string constant.



- ❑ Wire a time stamp constant to the **data type** input of the TDMS Get Properties function.
- ❑ Wire the **property value** output of the TDMS Get Properties function to the **Time of Test** indicator.

9. Read the time data or power spectrum data for the specified UUT and display the data on the Channel Data waveform graph.



- ❑ Add a TDMS Read function to the block diagram.
- ❑ Wire the **Data Set** control to the **group name in** input of the TDMS Read function.
- ❑ Wire the **Serial Number** control to the **channel name(s) in** input of the TDMS Read function.
- ❑ Wire the **data** output of the TDMS Read function to the **Channel Data** indicator.

10. Close the file and handle errors.



- ❑ Add a TDMS Close function to the block diagram.
- ❑ Add a Simple Error Handler VI to the block diagram.
- ❑ Wire the block diagram as shown in Figure 6-21.

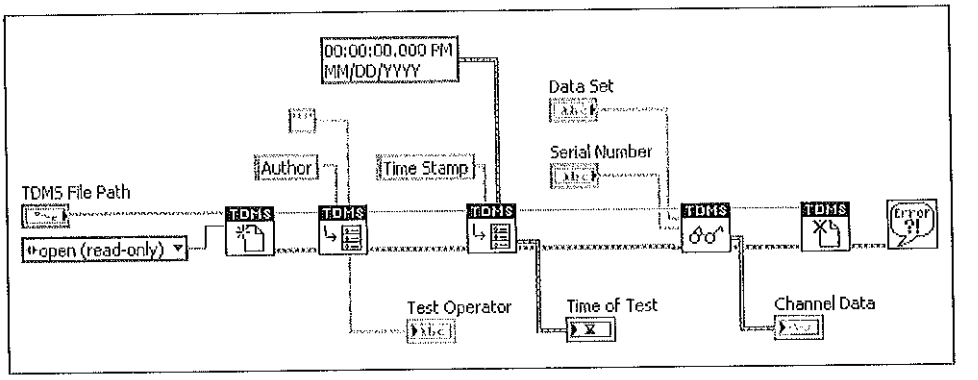


Figure 6-21. TDMS Reader Block Diagram

11. Save the VI.

12. Set the attributes and time stamp properties of the waveform graph.

    - ❑ On the front panel, right-click the **Channel Data** graph and select **Ignore Attributes**.



Note You must ignore the attributes of the waveform, otherwise the waveform name attribute would overwrite the labels you set. This option is available only after you wire a waveform to the graph.

- ❑ Right-click the Channel Data graph and deselect **Ignore Time Stamp** from the shortcut menu to plot the waveform data against its corresponding time stamp.
- ❑ Right-click the **Channel Data** graph and deselect **X Scale»Loose Fit** from the shortcut menu to fit the X scale to the time stamp of the displayed waveform.

Test

1. Read and display the time domain data.
   - ❑ On the VI front panel, ensure that the default TDMS File Path matches Table 6-6 and the Data Set is set to **Time Data**.
   - ❑ Enter `A001` in the **Serial Number** control.



Note `A001` was one of the serial numbers you entered when you ran the TDMS Logger VI.

   - ❑ Run the VI. A sine wave should display in the Channel Data graph.
   - ❑ Change the Serial Number to `A002`.
   - ❑ Run the VI. A different sine wave should display.
2. Read and display the power spectrum data.
   - ❑ Change the Data Set control to Power Spectrum.
   - ❑ Run the VI. Power spectrum data should display in the Channel Data graph.
3. Close the VI.

End of Exercise 6-2

Self-Review: Quiz

1. You must store the results of tests to a file. In the future, you must efficiently search for the tests which meet specific criteria. Which file storage format makes it easiest to query the data?
   a. Tab-delimited ASCII
   b. Custom binary format
   c. TDMS
   d. Datalog
2. You must write a program which saves Portable Network Graphics (PNG) image files. Which file storage VIs should you use?
   a. Storage file VIs
   b. Binary file VIs
   c. ASCII file VIs
   d. Datalog file VIs
3. You must store data that other engineers must analyze with Microsoft Excel. Which file storage format should you use?
   a. Tab-delimited ASCII
   b. Custom binary format
   c. TDMS
   d. Datalog
4. Which of the following is a little-endian representation of an unsigned 32-bit integer (U32) with a value of 10?
   a. 00001010 00000000 00000000 00000000
   b. 00000000 00000000 00000000 00001010
   c. 00001010
   d. 01010000 00000000 00000000 00000000
5. You can use the Binary File VIs to read ASCII files.
   a. True
   b. False
6. TDMS files store properties only at the channel or channel group level.
   a. True
   b. False

Self-Review: Quiz Answers

1. You must store the results of tests to a file. In the future, you need to efficiently search for the tests which meet specific criteria. Which file storage format makes it easiest to query the data?
   a. Tab-delimited ASCII
   b. Custom binary format
   c. **TDMS**
   d. Datalog
2. You must write a program which saves Portable Network Graphics (PNG) image files. Which file storage VIs should you use?
   a. Storage file VIs
   b. **Binary file VIs**
   c. ASCII file VIs
   d. Datalog file VIs
3. You need to store data which other engineers must analyze with Microsoft Excel. Which file storage format should you use?
   a. **Tab-delimited ASCII**
   b. Custom binary format
   c. TDMS
   d. Datalog
4. Which of the following is a little endian representation of an unsigned 32-bit integer (U32) with a value of 10?
   a. **00001010 00000000 00000000 00000000**
   b. 00000000 00000000 00000000 00001010
   c. 00001010
   d. 01010000 00000000 00000000 00000000
5. You can use the Binary File VIs to read ASCII files.
   a. **True**
   b. False
6. TDMS files store properties only at the channel or channel group level.
   a. True
   b. **False**

Notes

7

Creating and Distributing Applications

This lesson describes the process of creating a stand-alone application and installer for your LabVIEW projects.

Topics

A. LabVIEW Features for Project Development

B. Preparing the Application

C. Building the Application and Installer

A. LabVIEW Features for Project Development

LabVIEW provides several features you can use to manage your projects more efficiently.

VI History

One of the most useful LabVIEW tools for team-oriented development is the History window. Use the History window in each VI to display the development history of the VI, including revision numbers. The revision number starts at zero and increments every time you save the VI. Record and track the changes you make to the VI in the History window as you make them. Select **Edit»VI Revision History** to display the History window. You also can print the revision history.

Select **File»VI Properties** and click the **Revision History** button to set options for the current VI. Select **Tools»Options** to set options for all new VIs.

VI Hierarchy

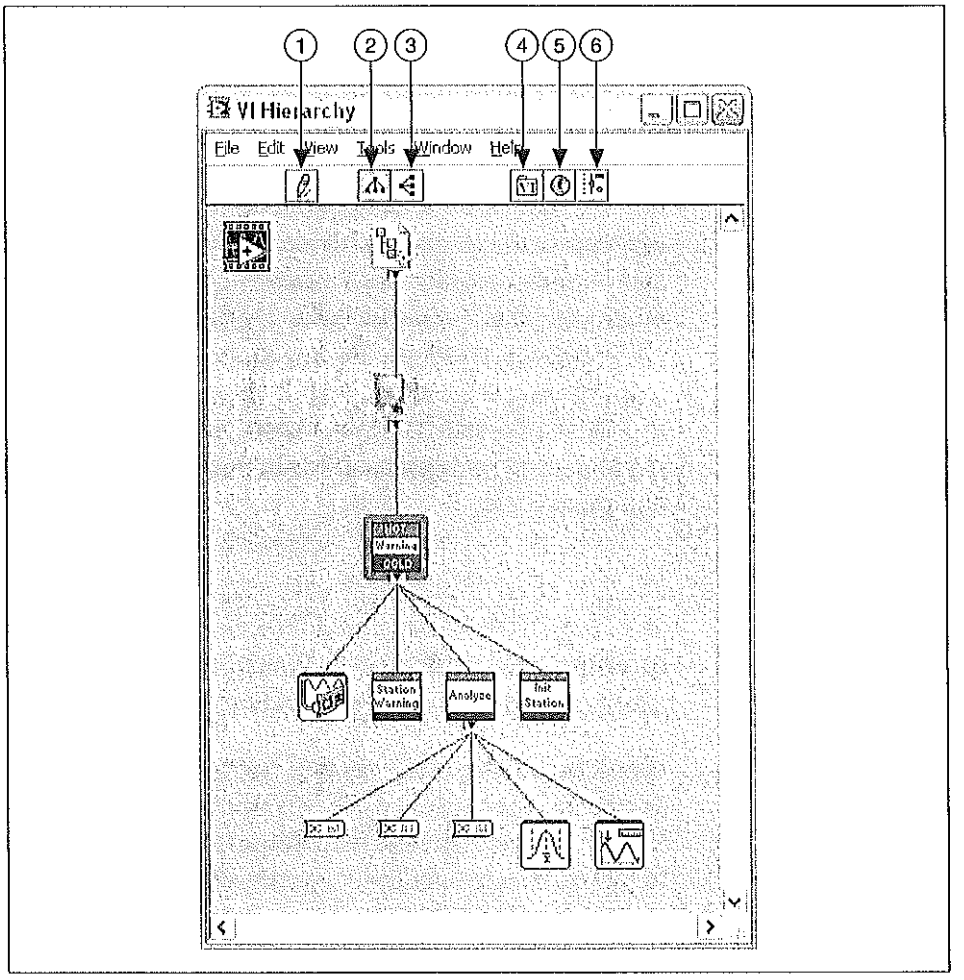
Saving memory is an important advantage of separating your main application into subVIs. Using subVIs makes the high-level block diagram easy to read, debug, understand, and maintain.

Therefore, try to keep the block diagram for your top-level VI under 500 KB in size. In general, keep your subVIs a smaller size. To check the size of a VI, select **File»VI Properties** and select **Memory Usage** from the **Category** pull-down menu. Typically, you should break a VI into several subVIs if the block diagram for your VI is too large to fit entirely on the screen.

The VI Hierarchy window displays a graphical representation of all open LabVIEW projects and targets, as well as the calling hierarchy for all VIs in memory, including type definitions and global variables. Select **View»VI Hierarchy** to display the VI Hierarchy window. Use this window to view the subVIs and other nodes that make up the VIs in memory and to search the VI hierarchy.

Use the toolbar at the top of the VI Hierarchy window to show or hide various categories of objects used in the hierarchy, such as global variables or VIs shipped with LabVIEW, as well as whether the hierarchy expands horizontally or vertically. A VI that contains subVIs has an arrow button on its bottom border. Click this arrow button to show or hide subVIs. A red arrow button appears when all subVIs are hidden. A black arrow button appears when all subVIs are displayed.

The VI Hierarchy window shown in Figure 7-1 contains the hierarchy of the Weather Station project built in this course. The VIs from the LabVIEW `vi.lib` directory are not shown. Right-click a blank area of the window and select **Show All VIs** from the shortcut menu to show the entire hierarchy.



1	Redo Layout	4	Include VI Lib
2	Vertical Layout	5	Include Globals
3	Horizontal Layout	6	Include Type Definitions

Figure 7-1. VI Hierarchy Window

As you move the cursor over objects in the VI Hierarchy window, LabVIEW displays the name of each VI in a tip strip. You can use the Positioning tool to drag a VI from the VI Hierarchy window to the block diagram to use the VI as a subVI in another VI. You also can select and copy a node or several nodes to the clipboard and paste them on other block diagrams.
Double-click a VI in the VI Hierarchy window to display the front panel of that VI. The VI Hierarchy window is a dynamic view of what is in memory. If you rearrange items in the hierarchy, LabVIEW does not save

the new position of items. When you close the VI Hierarchy window, the original positions regenerate.

You also can locate a VI in the hierarchy by typing the name of the item you want to find anywhere in the window. As you enter the text, the search string appears, displaying the text as you type. LabVIEW highlights one item at a time whose name begins with the search string. You also can find a node in the hierarchy by selecting **Edit»Find**.

Use the VI Hierarchy window as a development tool when planning or implementing your project. For example, after developing a flowchart of the VIs required for an application, you can create, from the bottom of the hierarchy up, each of these VIs so that they have all necessary inputs and outputs on their front panels and the subVIs called on their block diagrams. This builds the basic application hierarchy that now appears in the VI Hierarchy window. Then, you can start developing each subVI, such as color-coding their icons, which also is colored in the VI Hierarchy window to reflect their status. For example, white icons can represent untouched VIs, red icons can represent subVIs in development, and blue icons can represent completed VIs.

Comparing VIs

The LabVIEW Professional Development System includes a utility to determine the differences between two VIs loaded into the memory. Select **Tools»Compare»Compare VIs** to display the Compare VIs dialog box.

From this dialog box, you can select the VIs you want to compare, as well as the characteristics of the VIs to check. When you compare the VIs, both VIs display a Differences window that lists all differences between the two VIs. In this window, you can select various differences and details that you can circle for clarity.

Exercise 7-1 Concept: LabVIEW Project Management Tools

Goal

Examine some of the built-in LabVIEW features for project management.

Description

You can use the LabVIEW tools to determine the layout and architecture of the application. This is important when preparing to modify an application. Also, you can simplify documenting a developed application by including the VI revision history, and the VI hierarchy.

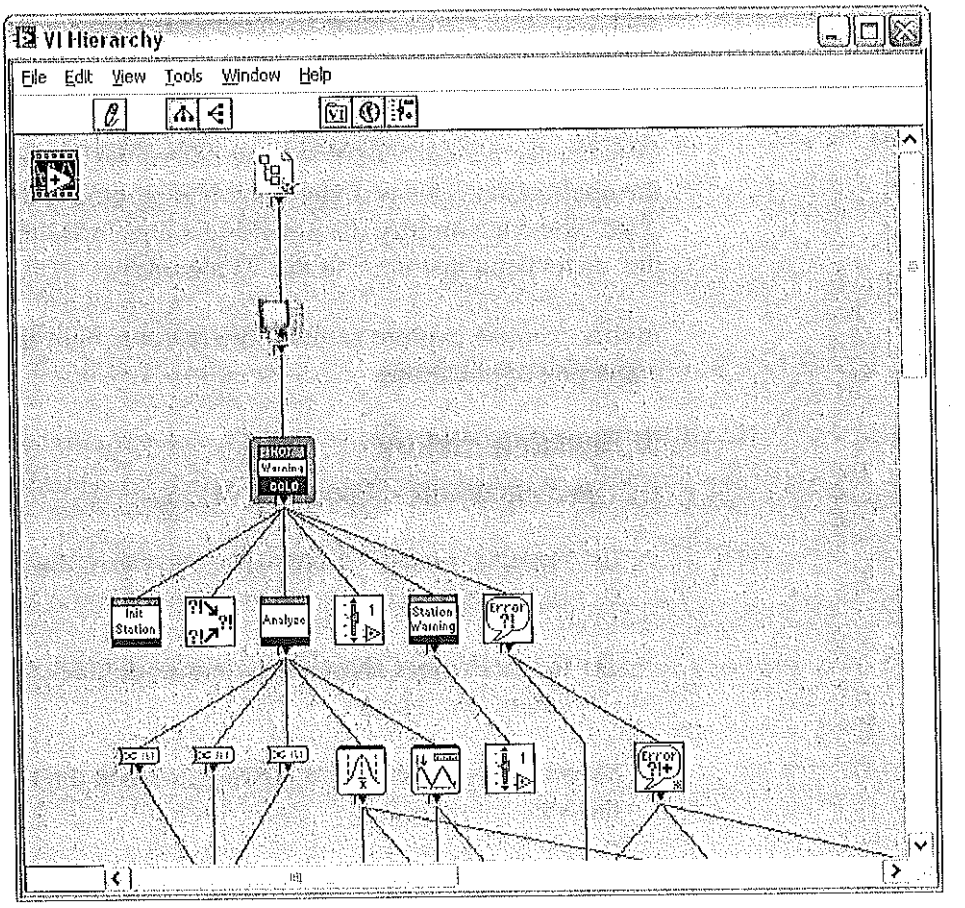
In this exercise explore some of the features built into LabVIEW for managing applications.

VI Revision History

1. Open the Weather Station UI VI.
   - ❑ Open `Weather Station.lvproj` in the `<Exercises>\LabVIEW Basics II\Course Project` directory.
   - ❑ In the **Project Explorer** window, double-click `Weather Station UI.vi`.
2. Select **Edit»VI Revision History** to open the History window for the VI.
3. Click the **Reset** button to clear the current history. Click **Yes** to confirm the deletion of the history and reset the revision number.
4. In the **Comment** text box of the History window, enter `Initial Application Created` and click the **Add** button. Your comment appears in the **History** text box, along with a date and time stamp. Close the History window.

VI Hierarchy

1. Select **View»VI Hierarchy**. The application hierarchy appears.



2. Experiment with expanding and collapsing the hierarchy. Notice that as you click the small black and red arrows in the hierarchy, they expand or collapse branches of the hierarchy. You might see some icons with a red arrow, indicating that they call one or more subVIs.

3. Examine the operation of the buttons on the toolbar. Notice how you can arrange the hierarchy using the **Layout** buttons or by dragging the icons. You also can include various application components using the **Include** buttons. Use **Redo Layout** to redraw the window layout to minimize line crossing and maximize symmetry.

4. Double-click any subVI icon in the hierarchy to display the appropriate subVI. Close the subVI you selected and close the VI Hierarchy window.

5. Close the VIs. Do not save any changes.

End of Exercise 7-1

B. Preparing the Application

A stand-alone application allows the user to run your VIs without installing the LabVIEW development system. Installers distribute the stand-alone application. Installers can include the LabVIEW Run-Time Engine, which is necessary for running stand-alone applications. However, you can also download the LabVIEW Run-Time Engine at `ni.com`.

To create a professional, stand-alone application with VIs, you must consider several programming issues.

External Code

Know what external code your applications uses. For example, do you call any system or custom DLLs or shared libraries? Are you going to process command line arguments? These are advanced examples that are beyond the scope of this course, but you must consider them for the application. Refer to the *Using External Code in LabVIEW* topic in the *LabVIEW Help*.

Path Names

Consider the path names you use in the VI. Assume you read data from a file during the application, and the path to the file is hard-coded on the block diagram. Once an application is built, the file is embedded in the executable, changing the path of the file. Being aware of these issues will help you to build more robust applications in the future.

Quit LabVIEW

In a stand-alone application, the top-level VI must quit LabVIEW or close the front panel when it is finished executing. To completely quit and close the top-level VI, you must call the Quit LabVIEW function on the block diagram of the top-level VI.

Providing Online Help in Your LabVIEW Applications

As you put the finishing touches on your application, you should provide online help to the user. Create descriptions for VIs and their objects, such as controls and indicators, to describe the purpose of the VI or object and to give users instructions for using the VI or object.

Use the following functions, located on the **Help** palette, to programmatically show or hide the **Context Help** window and link from VIs to HTML files or compiled help files:

- Use the Get Help Window Status function to return the status and position of the **Context Help window**.
- Use the Control Help Window function to show, hide, or reposition the **Context Help** window.

- Use the Control Online Help function to display the table of contents, jump to a specific topic in the file, or close the online help.
- Use the Open URL in Default Browser VI to display a URL or HTML file in the default Web browser.

C. Building the Application and Installer

Build Specifications in LabVIEW create stand-alone applications and installers.

- Stand-alone applications—Use stand-alone applications to provide other users with executable versions of VIs. Applications are useful when you want users to run VIs without installing the LabVIEW development system. **(Windows)** Applications have a `.exe` extension. **(Mac OS)** Applications have a `.app` extension.
- Installers—**(Windows)** Use installers to distribute stand-alone applications, shared libraries, and source distributions that you create with the Application Builder. Installers that include the LabVIEW Run-Time Engine are useful if you want users to be able to run applications or use shared libraries without installing LabVIEW.

LabVIEW Build Specifications

Use **Build Specifications** in the Project Explorer window to create build specifications for source distributions and other types of LabVIEW builds. A build specification contains all the settings for the build, such as files to include, directories to create, and settings for VIs.



Note If you previously hid **Build Specifications** in the **Project Explorer** window, you must display the item again to access it in the **Project Explorer** window.



Tip **(Windows and UNIX)** Depending upon the nature of your application, it may require the presence of non-VI files to function correctly. Files commonly needed include a preferences (`.ini`) file for the application, and any help files that your VIs call.

System Requirements

Applications that you create with Build Specifications generally have the same system requirements as the LabVIEW development system. Memory requirements vary depending on the size of the application created.

You can distribute these files without the LabVIEW development system; however, stand-alone application and shared library users must have the LabVIEW Run-Time Engine installed.

Implementing Build Specifications

You must create build specifications in the Project Explorer window. Expand My Computer, right-click **Build Specifications**, select **New** and the type of build you want to configure from the shortcut menu. Use the pages in the Source Distribution Properties, Application Properties, Shared Library Properties, **(Windows)** Installer Properties, or Zip File Properties dialog boxes to configure settings for the build specification. After you define these settings, click the **OK** button to close the dialog box and update the build specification in the project. The build specification appears under Build Specifications. Right-click a specification and select **Build** from the shortcut menu to complete the build. You also can select **Build All** from the shortcut menu to build all specifications under Build Specifications.



Note You can duplicate build specifications in the Project Explorer window. Right-click the build specification item you want to duplicate and select **Duplicate** from the shortcut menu to create a copy of the **Build Specification** item. You also can drag and drop build specification items to rearrange the build order within the same **Build Specification**.

Review the caveats and recommendations for applications and shared libraries and for installers before you create build specifications with the Application Builder.

Refer to the *LabVIEW Help* for more information about the caveats and recommendations for applications and installers.

Exercise 7-2 Concept: Creating a Stand-Alone Application

Goal

Create a stand-alone application with LabVIEW.

Description

Creating a stand-alone application and an installer simplifies deploying an application on multiple machines. In order to deploy the application, you first prepare the code, create an Application (`.exe`) Build Specification, and then create an Installer Build Specification.

Set Top-Level Application Window

1. Open the Weather Station UI VI.

   - ❑ Open `Weather Station.lvproj` in the `<Exercises>\LabVIEW Basics II\Course Project` directory.
   - ❑ In the Project Explorer window, double-click `Weather Station UI.vi`.

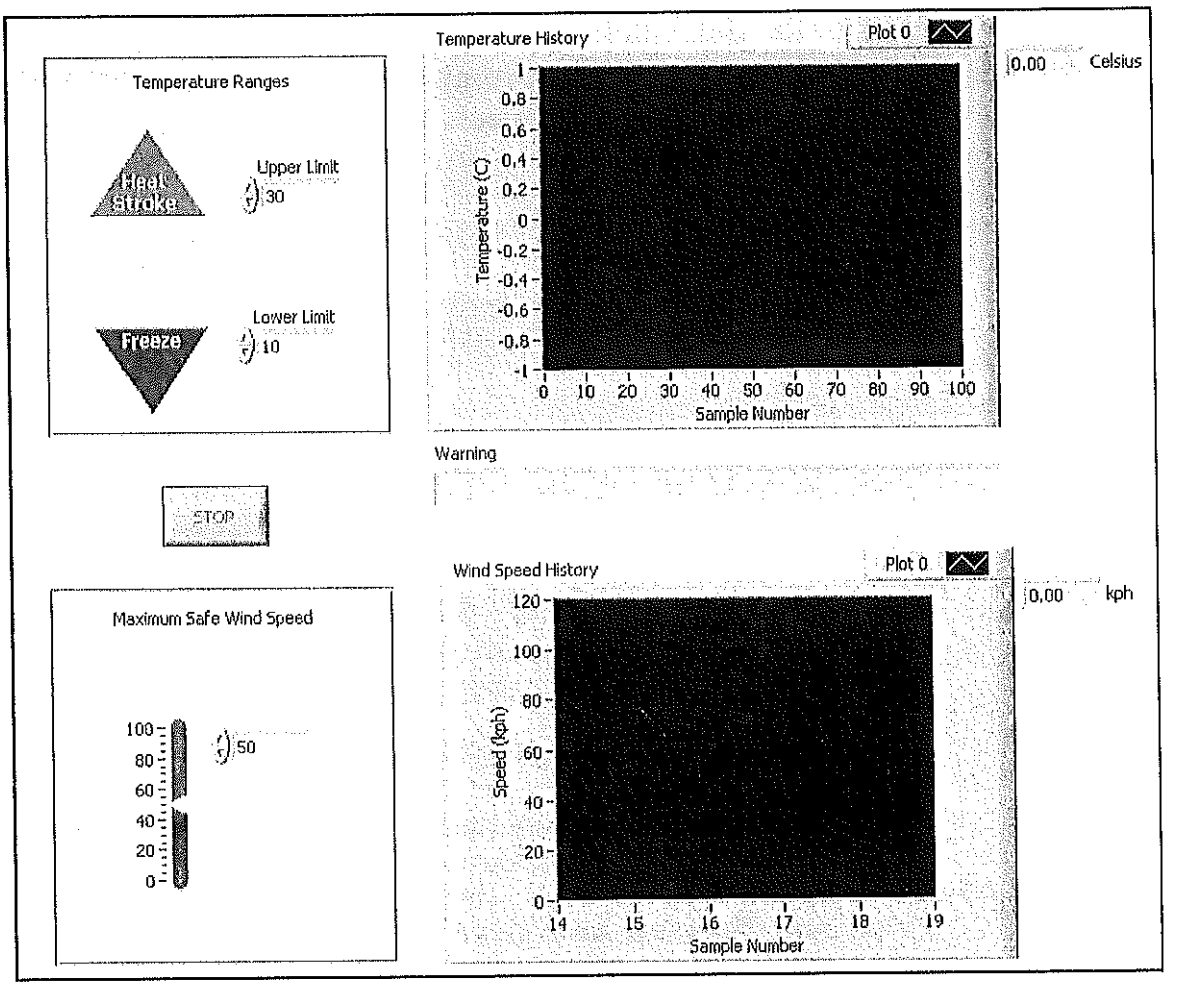


Figure 7-2. Front Panel

2. Select **File»VI Properties** to display the VI Properties dialog box.
3. Select **Window Appearance** from the top pull-down menu.
4. Give the window a name, such as `Weather Station`.
5. Select **Top-level application window**. This gives the front panel a professional appearance when it opens as an executable.
6. Save the VI.

Call the Quit LabVIEW Function

1. Open and modify the block diagram to call the Quit LabVIEW function when the application finishes.

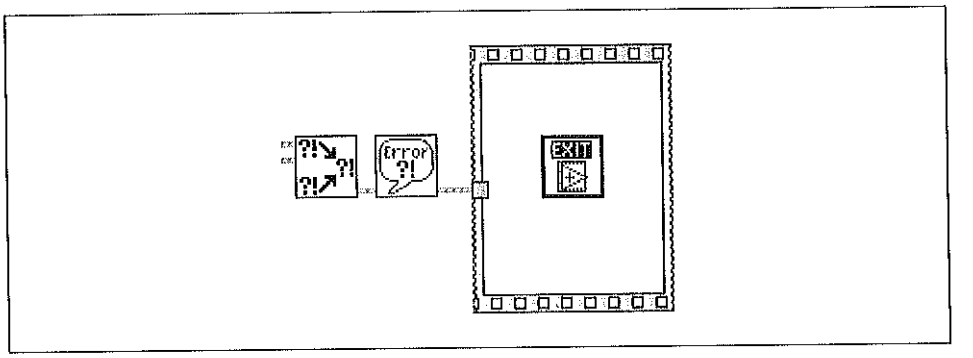


Figure 7-3. Block Diagram



- ❑ Add the Quit LabVIEW function to the block diagram so that it is the last function that executes. This function quits LabVIEW and quits the application after it has executed.
- ❑ Enclose the Quit LabVIEW function in a Flat Sequence structure.
- ❑ Wire the Simple Error Handler VI to the border of the Sequence structure to force the execution order.

2. Select **File»Save All** to save all the VIs.
3. Open the front panel and run the VI. When you click the **Stop** button, the VI stops and LabVIEW quits.
4. Restart LabVIEW and open `Weather Station.lvproj` located in the `<Exercises>\LabVIEW Basics II\Course Project` directory.

Modify File Path

1. Modify the relative path to have the same functionality after the executable is built by stripping an additional component of the path.

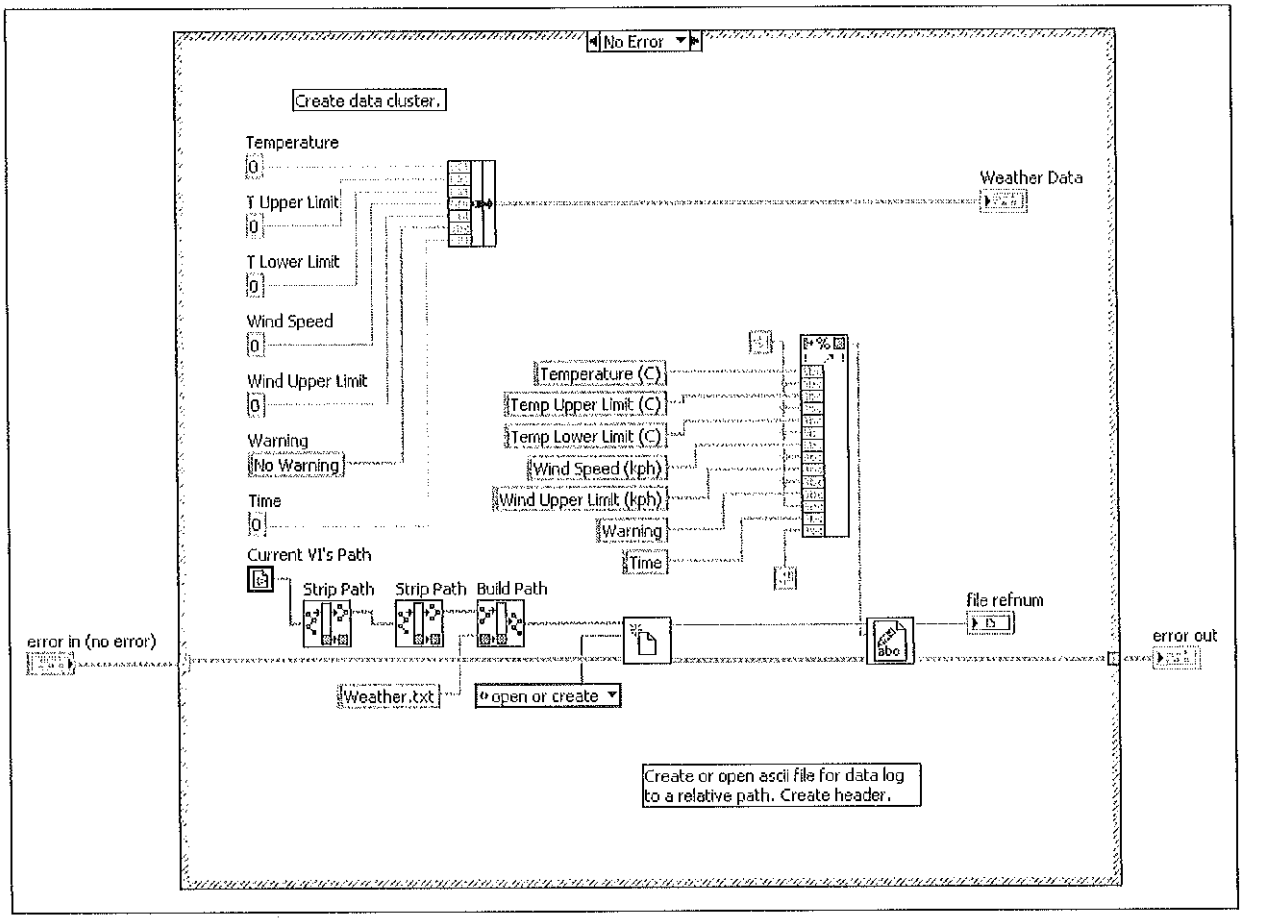


Figure 7-4. Additional Strip Path Function

- ❑ Open the `Initialize Weather Station.vi` from the Project Explorer window.
- ❑ Switch to the block diagram.
- ❑ Add an additional Strip Path function to the block diagram.
- ❑ Wire the block diagram as shown in Figure 7-4.
- ❑ Save and close the VI.

Application (EXE) Build Specification

1. Right-click **Build Specifications** in the Project Explorer window and select **New»Application (EXE)** from the shortcut menu.
2. Modify the filename of the target and destination directory for the application in the **Information** category.
   - ❑ Select the **Information** category.
   - ❑ Change the **Target filename** to `WeatherStation.exe`.
   - ❑ Enter `<Exercises>\LabVIEW Basics II\Course Project\Executable` in the **Destination directory**.



Tip You do not need to create the directory. LabVIEW creates any directories that you specify.

3. Specify the top-level VI.
   - ❑ Select the **Source Files** category.
   - ❑ Select the **Weather Station UI.vi** in the **Project Files** tree.
   - ❑ Click the arrow next to the **Startup VIs** listbox to add the selected VI to the **Startup VIs** listbox.
   - ❑ Click **OK**.
4. In the Project Explorer window, right-click the **My Application** build specification that you just created, and select **Build** from the shortcut menu.
5. Click **Done** in the Build Status window.
6. Navigate to `<Exercises>\LabVIEW Basics II\Course Project\Executable` in Windows Explorer and run `WeatherStation`. Stop when done.

Installer Build Specification

1. Right-click **Build Specifications** in the Project Explorer window and select **New»Installer** from the shortcut menu.
2. Modify the **Installer destination** in the **Product Information** category.
   - ❑ Select the **Product Information** category.

- ❑ Enter `<Exercises>\LabVIEW Basics II\Course Project\Installer` as the **Installer destination**.

3. Specify the **Executable Build Specification**.
   - ❑ Click the **Source Files** category.
   - ❑ Select the **My Application** build specification.
   - ❑ Select the **Weather Station** in the **ProgramFilesFolder** in the **Destination View** tree.
   - ❑ Click the arrow next to the **Project View** tree to place the executable and the executable support files under the ProgramFilesFolder in the Weather Station directory as shown in Figure 7-5.

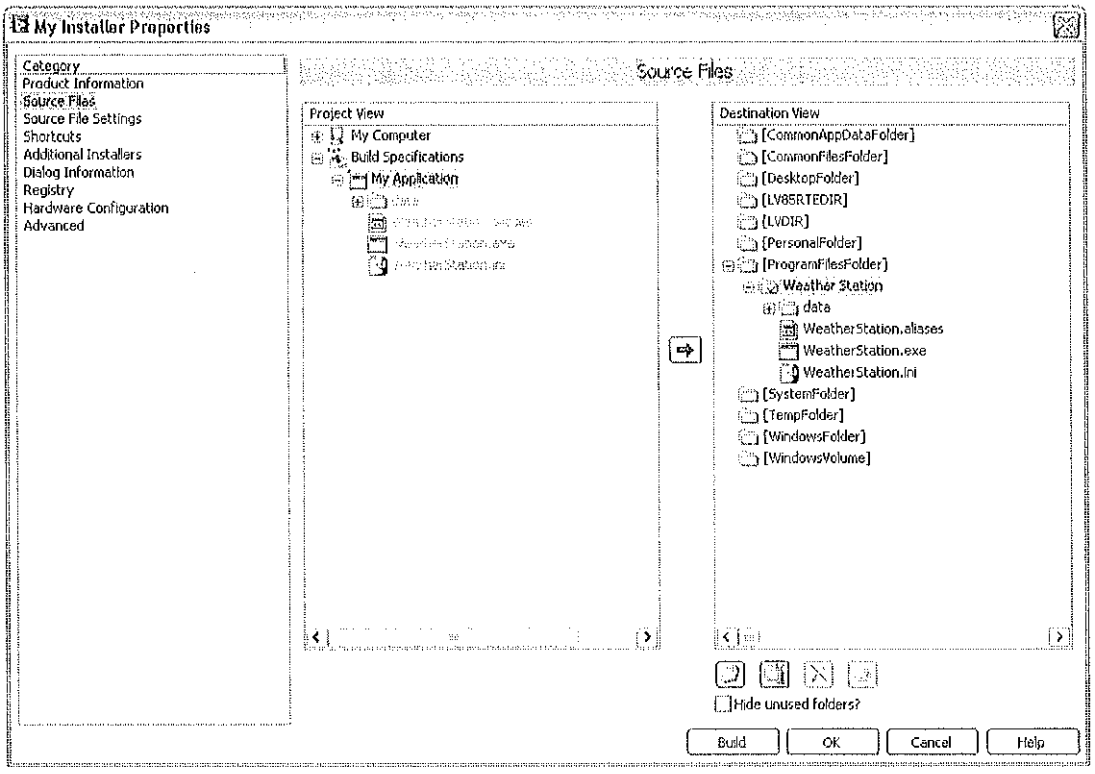


Figure 7-5. Installer Source Files Category

4. Add the NI LabVIEW Run-Time Engine to the installer by modifying the **Additional Installers** category.
   - ❑ Select the **Additional Installers** category.
   - ❑ Select the **NI LabVIEW Run-Time Engine 8.xx** installer.

5. Add a shortcut to the Start menu, by modifying the **Shortcuts** category.
   - ❑ Select the **Shortcuts** category.
   - ❑ Click the + button to add a shortcut.
   - ❑ Select **Weather Station.exe** in the **Select Target File** dialog box and click **OK**.
   - ❑ Click **OK**.
6. In the Project Explorer window, right-click the **My Installer** build specification and select **Build** from the shortcut menu.
7. Click **Done**.

Testing

1. Run the `setup.exe` file in the `<Exercises>\LabVIEW Basics II\Course Project\Installer\Volume` directory. You should be guided through a setup process. The executable is created inside the `<Program Files>\Weather Station` directory.
2. To run the application, select **Start»Programs»Weather Station» Weather Station**.

End of Exercise 7-2

Summary

- LabVIEW features the Application Builder, which enables you to create stand-alone executables and installers. The Application Builder is available in the Professional Development Systems or as an add-on package.
- Creating a professional, stand-alone application with your VIs involves understanding the following:
  - The architecture of your VI
  - The programming issues particular to the VI
  - The application building process
  - The installer building process
- Use the VI Revision History window to record comments and modifications to a VI and the user login, which, when used with VI Revision History, records who made changes to a VI. You can access the VI Revision History window at any time by selecting **Tools» VI Revision History**.
- The VI Hierarchy window provides a quick, concise overview of the VIs used in your project.

Notes

A

Additional Information and Resources

This appendix contains additional information about National Instruments technical support options and LabVIEW resources.

National Instruments Technical Support Options

Visit the following sections of the National Instruments Web site at `ni.com` for technical support and professional services.

- **Support**—Online technical support resources at `ni.com/support` include the following:
  - **Self-Help Resources**—For answers and solutions, visit the award-winning National Instruments Web site for software drivers and updates, a searchable KnowledgeBase, product manuals, step-by-step troubleshooting wizards, thousands of example programs, tutorials, application notes, instrument drivers, and so on.
  - **Free Technical Support**—All registered users receive free Basic Service, which includes access to hundreds of Application Engineers worldwide in the NI Discussion Forums at `ni.com/forums`. National Instruments Application Engineers make sure every question receives an answer.

    For information about other technical support options in your area, visit `ni.com/services` or contact your local office at `ni.com/contact`.
- **System Integration**—If you have time constraints, limited in-house technical resources, or other project challenges, National Instruments Alliance Partner members can help. The NI Alliance Partners joins system integrators, consultants, and hardware vendors to provide comprehensive service and expertise to customers. The program ensures qualified, specialized assistance for application and system development. To learn more, call your local NI office or visit `ni.com/alliance`.

If you searched `ni.com` and could not find the answers you need, contact your local office or NI corporate headquarters. Phone numbers for our worldwide offices are listed at the front of this manual. You also can visit the Worldwide Offices section of `ni.com/niglobal` to access the branch office Web sites, which provide up-to-date contact information, support phone numbers, email addresses, and current events.

Other National Instruments Training Courses

National Instruments offers several training courses for LabVIEW users. These courses continue the training you received here and expand it to other areas. Visit `ni.com/training` to purchase course materials or sign up for instructor-led, hands-on courses at locations around the world.

National Instruments Certification

Earning an NI certification acknowledges your expertise in working with NI products and technologies. The measurement and automation industry, your employer, clients, and peers recognize your NI certification credential as a symbol of the skills and knowledge you have gained through experience. areas. Visit `ni.com/training` for more information about the NI certification program.

LabVIEW Resources

This section describes how you can receive more information regarding LabVIEW.

LabVIEW Books

Many books have been written about LabVIEW programming and applications. The National Instruments Web site contains a list of all the LabVIEW books and links to places to purchase these books.

info-labview Listserve

info-labview is an email group of users from around the world who discuss LabVIEW issues. The list members can answer questions about building LabVIEW systems for particular applications, where to get instrument drivers or help with a device, and problems that appear.

To subscribe to info-labview, send email to:

info-labview-on@labview.nhmfl.gov

To subscribe to the digest version of info-labview, send email to:

info-labview-digest@labview.nhmfl.gov

To unsubscribe to info-labview, send email to:

info-labview-off@labview.nhmfl.gov

To post a message to subscribers, send email to:

info-labview@labview.nhmfl.gov

To send other administrative messages to the info-labview list manager, send email to:

info-labview-owner@nhmfl.gov

You might also want to search previous email messages at:

www.searchVIEW.net

The info-labview web page is available at:

www.info-labview.org

Course Evaluation

Course _____

Location _____

Instructor _____ Date _____

Student Information (optional)

Name _____

Company _____ Phone _____

Instructor

Please evaluate the instructor by checking the appropriate circle.	Unsatisfactory	Poor	Satisfactory	Good	Excellent
Instructor's ability to communicate course concepts	O	O	O	O	O
Instructor's knowledge of the subject matter	O	O	O	O	O
Instructor's presentation skills	O	O	O	O	O
Instructor's sensitivity to class needs	O	O	O	O	O
Instructor's preparation for the class	O	O	O	O	O

Course

Training facility quality	O	O	O	O	O
Training equipment quality	O	O	O	O	O

Was the hardware set up correctly? O Yes O No

The course length was O Too long O Just right O Too short

The detail of topics covered in the course was O Too much O Just right O Not enough

The course material was clear and easy to follow. O Yes O No O Sometimes

Did the course cover material as advertised? O Yes O No

I had the skills or knowledge I needed to attend this course. O Yes O No If no, how could you have been better prepared for the course? _____

_____

What were the strong points of the course? _____

_____

What topics would you add to the course? _____

_____

What part(s) of the course need to be condensed or removed? _____

_____

What needs to be added to the course to make it better? _____

_____

How did you benefit from taking this course? _____

_____

Are there others at your company who have training needs? Please list. _____

_____

_____

Do you have other training needs that we could assist you with? _____

_____

How did you hear about this course? O NI Web site O NI Sales Representative O Mailing O Co-worker O Other _____